CSSMedia User Guide



Copyright © 2026 Maxprograms

Table of Contents

Features **1**

Get started **2**
- Install CSSMedia 2
- Prepare your XML 2
- Run CSSMedia 3
- Your first stylesheet 3
- Try the samples 4

Selectors **7**
- Basic selectors and namespaces 7
- Combinators 8
- Pseudo-classes 10
- Pseudo-elements 11
- Specificity 12
- Selector reference 13

Cascade and declarations **21**
- Decide which rule wins 21
- Inherit, reset and roll back values 25
- Set many properties with a shorthand 27
- Reuse values with custom properties and functions 28
- Apply rules on conditions 30
- Cascade and declarations reference 33

Boxes, backgrounds and borders **42**
- Margins and padding 43
- Box sizes 45
- Borders and rounded corners 47
- Backgrounds 50
- Boxes, backgrounds and borders reference 53

Text and fonts **70**
- Font selection and sizing 70
- Font resources and descriptors 73
- Font variants and features 77
- Variable and color fonts 80
- Text and fonts reference 82

Languages and hyphenation **92**
- Document language 92
- Hyphenation 94
- Quotation marks 95
- Languages and hyphenation reference 96

Ruby and vertical text **99**

Tables **100**
Turn elements into a table 101
Size a table and its columns 104
Draw the borders and backgrounds of a table 106
Break tables across pages 108
Tables reference 109

Multi-column layout **112**

Flexbox and grid **113**

Shadows, opacity, transforms and masks **114**

Define pages **115**
Page setup reference 118

Page orientation, bleed and marks **122**

Named strings **123**
Named strings reference 124

Links **126**
Link reference 128

Footnotes **129**
Footnote reference 133

Table of contents **135**
Table of contents reference 138

Images **141**
Place an image 141
Fit an image in its box 143
Images reference 144

SVG **147**
Put a drawing in your document 148
Style a drawing 150
Draw shapes and reuse them 151
Paint shapes 154
Clip, mask and blend 157
Write text in a drawing 159
SVG reference 161

MathML **168**

Complete reference 169
Selector reference 169
Cascade reference 177
Margins, padding and borders 185
Box sizes 195
Backgrounds 198
Display and positioning 201
Tables 203
Text and fonts 205
Languages and hyphenation 214
Page reference 216
Links and generated content reference 220
Named strings 228
Links 229
Footnotes 230
Table of contents 231
Images 234
SVG 235

Licenses 241

Features

CSSMedia offers these main features.

- **XML format agnostic.** CSSMedia works with any well-formed XML. It does not assume anything about element names or attributes.
- **Any grammar.** XML is parsed with TypesXML, which handles any vocabulary. You can pass the grammars of your documents to CSSMedia with an OASIS Catalog.
- **CSS based.** The stylesheet is the only source of styling. Selector matching, the cascade, inheritance and value resolution follow the CSS specifications.
- **PDF for paged media.** Page sizes, margins, page breaks, named pages, headers, footers and page numbers are controlled with CSS paged media features.
- **Multilingual support.** Documents can be in any language and script, including right-to-left languages and CJK languages.
- **Hyphenation by language.** Text is hyphenated with the patterns of the language declared with `xml:lang`.
- **Fonts.** OpenType, TrueType, font collections, WOFF and WOFF 2.0 fonts, loaded from `@font-face` rules or a fonts folder, plus the 14 standard PDF fonts.
- **Shared fonts folder.** You can pass a central fonts folder to CSSMedia and reuse it from any project.
- **Generated content.** Counters, quotes, list markers, named strings, running elements, cross-references and tables of contents.
- **Links and bookmarks.** Internal and external links and PDF bookmarks.
- **Footnotes.** Any element can be a footnote. CSS moves it to the bottom of the page, numbered with a counter that you can restart on each page.
- **Images.** PNG, JPEG, JPEG 2000, GIF, BMP, TIFF and WebP images.
- **SVG and MathML.** SVG and MathML found in a document are drawn as images.

Get started

CSSMedia turns an XML file and a CSS file into a PDF.

To produce a PDF, install CSSMedia, prepare your XML and run it from the command line with your stylesheet. These topics show how:

- Install CSSMedia
- Prepare your XML
- Run CSSMedia
- Your first stylesheet
- Try the samples

Install CSSMedia

Follow these steps to build CSSMedia from source or to install it as a global command.

Procedure

1. Clone the CSSMedia project and build it with npm:

   ```
   git clone https://github.com/maxprograms-com/CSSMedia.git
   cd CSSMedia
   npm install
   npm run build
   ```

2. Check that the build works by running CSSMedia:

   ```
   node dist/cssmedia.js
   ```

   Without parameters, CSSMedia displays its help.

3. To use the `cssmedia` command from any folder, install the package globally:

   ```
   npm i -g file:cssmedia-x.x.x.tgz
   ```

   Download or build the package first, and replace `x.x.x` with its version.

Prepare your XML

CSSMedia generates a PDF from almost any well-formed XML file, provided that you give it an appropriate CSS file.

Some XML projects consist of multiple files that need pre-processing before they are ready for publishing. Projects based on DITA or DocBook need to merge all the files that make up a project into one XML file.

- If you work with DITA, use TypesDitac to process your topics and generate one combined XML file for CSSMedia.
- If you work with DocBook, TypesBook helps you merge your files into a single document ready for CSSMedia.

Run CSSMedia

Run CSSMedia from the command line with an XML file, a CSS file and the name of the PDF to write.

This command generates a PDF from your XML and your stylesheet:

```
cssmedia -x document.xml -c document.css -p document.pdf
```

If you built CSSMedia from source and did not install it globally, run `node dist/cssmedia.js` instead of `cssmedia`.

Running CSSMedia without parameters displays this help:

```
Usage: cssmedia -x <file> -c <file> -p <file> [-f <fontsFolder>] [-g <catalog>]

  -x, --xml <file>       XML file to format
  -c, --css <file>       CSS stylesheet to apply
  -p, --pdf <file>       PDF file to write
  -f, --fonts <folder>   (optional) alternative fonts folder
  -g, --catalog <file>   (optional) OASIS catalog to use when parsing XML
```

Use `-f` to pass a fonts folder that you reuse from any project. Use `-g` to pass an OASIS Catalog when your XML refers to grammars.

When something in the XML or the stylesheet cannot be used, for example an image that cannot be read, CSSMedia still writes the PDF and prints a warning for each problem after the run. If a file is missing or an error stops the run, CSSMedia prints the error and does not write the PDF.

Your first stylesheet

CSSMedia takes two files: an XML document and a CSS stylesheet. XML contains the document's elements and text. CSS contains rules that describe how selected elements should be styled.

Procedure

1. Look at the XML

   Suppose your XML contains this element:

   ```
   <chapter>
       <title>Using the device</title>
       <p>Follow these steps before starting.</p>
   </chapter>
   ```

   The names `chapter`, `title`, and `p` belong to this XML document. They are not special CSS names. You can use the names from your own XML in a stylesheet.

2. Write a rule

   A CSS rule has a selector and a declaration block:

   ```
   title {
       font-size: 18pt;
   }
   ```

The selector `title` chooses XML elements with that name. The declaration `font-size: 18pt;` sets a property to a value. The braces contain the declarations for the selected elements.

You can add more declarations to the same rule:

```
title {
    font-family: serif;
    font-size: 18pt;
    color: navy;
}
```

Each declaration has a property name, a colon, a value, and a semicolon. CSSMedia checks property values against the CSS syntax it supports. A selector only matches when it describes elements in your XML.

3. Use names from your own XML

   The element names in the examples are placeholders. If your XML uses `heading` instead of `title`, write `heading` as the selector:

   ```
   heading {
       font-size: 18pt;
   }
   ```

   CSSMedia does not infer that `heading` is a title or that `p` is a paragraph. You decide which XML names to select and which CSS properties to set.

4. Generate the PDF

   Save the XML as `document.xml` and the stylesheet as `document.css`, then run CSSMedia:

   ```
   cssmedia -x document.xml -c document.css -p document.pdf
   ```

   Open `document.pdf`. The `title` element is styled by the rule you wrote.

Try the samples

The CSSMedia repository includes sample documents, stylesheets, fonts and the PDFs they produce. Follow these steps to generate a PDF from a sample and change its stylesheet.

About this task

The `samples` folder of the repository has these samples:

Sample	Content
xml_gap	A DocBook article in English, Arabic, Japanese and Russian, formatted with one stylesheet, `xml_gap.css`.
Swordfish	The User Guide of a real product, written in DITA and merged into one XML file with TypesDitac and with the DITA Open Toolkit. The stylesheets `ditac.css`, `ditac_hyphens.css` and `dita-ot.css` format the merged files.
fonts	The Noto fonts that the stylesheets load with `@font-face` rules.

Each sample comes with the PDF that CSSMedia generated from it.

Procedure

1. Open a terminal in the `samples` folder.
2. Generate the PDF of the English article:

   ```
   cssmedia -x xml_gap/xml_gap.xml -c xml_gap.css -p xml_gap/xml_gap.pdf
   ```

   The XML files declare a DocBook DTD. Without a catalog, CSSMedia prints the warning `External subset not found: docbook/docbookx.dtd` and still generates the PDF. To avoid the warning, pass a catalog that resolves the DTD with `-g`, such as the one in TypesBook.
3. Open the PDF and compare it with the one in the repository.
4. Edit `xml_gap.css` and run the command again.

   For example, change the `font-size` of `article` or the `color` of `section > title`, and see what changes in the PDF.
5. Generate the Swordfish User Guide:

   ```
   cssmedia -x Swordfish/typesditac/swordfish.xml -c ditac.css -p Swordfish/
   typesditac/swordfish.pdf
   ```

Results

The `xml_gap.css` stylesheet uses these techniques:

Result	CSS
Local fonts	`@font-face` rules with `src: url("./fonts/NotoSans-Regular.ttf")`
Page size, margins and a footer with the document title and the page number	`@page` with `@bottom-left` and `@bottom-right`, `string-set: doc-title content()` on the title, and `counter(page)`
A different font for each language	Selectors such as `article[lang|="ar"]` and `article[lang|="ja"]`
Hyphenation in English and Russian	`hyphens: auto` on `article:lang(en)` and `article:lang(ru)`
Right-to-left text in Arabic	`direction: rtl`
Stricter line breaking in Japanese	`line-break: strict`
Links from an XML attribute	`-cssmedia-link: attr(url)` on `ulink`
An image from an XML attribute	`content: attr(fileref url)` on `imagedata`
The photo at the start side of the text, on the right in Arabic and on the left in the other languages	`float: inline-start` on `mediaobject`

Result	CSS
An arrow after each link	`ulink::after { content: " \2197"; }`
Code listings in a monospace font	`font-family: "Noto Sans Mono", monospace` with `white-space: pre`

Selectors

A selector tells CSSMedia which element or elements in the XML file a CSS rule applies to. A rule has a selector followed by braces. Inside the braces, write declarations that set property values:

```
chapter {
    color: navy;
}
```

Here, `chapter` is the selector and `color: navy;` is a declaration. CSSMedia reads XML names and attributes as data. It does not assign vocabulary-specific meanings to them.

The guides are grouped by selector type and matching behavior:

- Selector reference: a compact reference to the selector forms and their PDF behavior.
- Basic selectors and namespaces: element names, IDs, classes, attributes and XML namespaces.
- Combinators: selecting elements by their relationship to other elements.
- Pseudo-classes: structural tests, selector logic, language and other states.
- Pseudo-elements: selecting generated content, text parts and other special targets.
- Specificity: how selectors help resolve competing declarations.

Basic selectors and namespaces

Basic selectors match an element by its name, attributes or namespace. Combine selector parts when you want to require more than one condition.

Element names and any element

A type selector is an element name, such as `chapter` or `title`. It matches elements with that name. The universal selector `*` matches any element.

```
chapter { color: navy; }
* { font-family: serif; }
```

An unprefixed name uses the default namespace declared with `@namespace`. Without a declared default namespace, it matches an element with no namespace.

IDs and classes

An ID identifies an element. Write # followed by the ID value to select it. CSSMedia checks the `id` and `xml:id` attributes.

```
#overview { color: navy; }
```

A class selector starts with a period, as in `.warning`. Selectors Level 4 leaves the meaning of class membership to the document language, and XML has no universal class attribute. For that reason, class selectors only match inside SVG and MathML content, which define a `class` attribute.

If your XML has a `class` attribute with space-separated names, select it with an attribute selector. The ~= operator matches one whole word of the value:

```
[class~="warning"] { color: red; }
```

Attributes

An attribute selector checks an XML attribute. Put its name in square brackets to require that the attribute exists. Add an operator and value to compare its contents.

```
[role] { color: navy; }
[role="notice"] { color: navy; }
[class~="warning"] { color: red; }
[lang|="en"] { font-family: serif; }
[href^="https"] { color: blue; }
[href$=".pdf"] { color: purple; }
[data-kind*="reference"] { color: teal; }
[role="notice" i] { color: navy; }
```

= compares the whole value. ~= checks for a space-separated word. |= matches the value itself or a value followed by a hyphen. ^=, $= and *= check for text at the start, end or anywhere in the value. The `i` flag ignores ASCII letter case. The `s` flag requests case-sensitive comparison.

An attribute selector without a namespace prefix checks an attribute with no namespace. Use a declared prefix to select an attribute in a namespace.

XML namespaces

XML elements and attributes can belong to a namespace. The `@namespace` rule of CSS assigns a prefix to a namespace URI. Use that prefix in a selector to choose elements in that namespace.

```
@namespace svg url("http://www.w3.org/2000/svg");

svg|rect { color: navy; }
*|rect { color: teal; }
|rect { color: gray; }
```

`svg|rect` selects `rect` elements in the declared SVG namespace. `*|rect` selects `rect` elements in any namespace. `|rect` selects `rect` elements with no namespace. The CSS prefix is an alias for the URI. It does not need to match the prefix used in the XML.

A bare attribute selector such as `[id]` always selects an attribute with no namespace. An undeclared prefix makes that selector invalid. CSSMedia ignores a style rule if any selector in its selector list is invalid.

Combinators

A combinator describes the relationship between two elements. Put the related element's selector on the left and the element you want to match on the right.

Descendant and sibling relationships

Use a space to select a descendant at any depth. Use > for a direct child, + for the next element sibling and ~ for any later element sibling with the same parent.

```
chapter title { color: navy; }
chapter > title { color: teal; }
title + paragraph { color: gray; }
title ~ paragraph { color: purple; }
```

Example:

```
<chapter>
    <title>Setup</title>
    <paragraph>Install the tool.</paragraph>
    <paragraph>Start the tool.</paragraph>
    <section>
        <title>Options</title>
        <paragraph>Choose an option.</paragraph>
    </section>
</chapter>
```

- `chapter title` selects both titles, `Setup` and `Options`, because the second one is inside a `section` that is inside the chapter.
- `chapter > title` selects only `Setup`, the title that is a direct child of the chapter.
- `title + paragraph` selects `Install the tool.` and `Choose an option.`, the paragraphs that come right after a title.
- `title ~ paragraph` selects all three paragraphs, because each one has a title before it among its siblings.

Column combinator

The column combinator `||`, from Selectors Level 5, selects the cells of the column that a column element describes. CSSMedia finds columns and cells by the `display` values `table-column`, `table-column-group` and `table-cell`, not by element names. Columns must come before the first row. CSS gives a cell no span, so each cell takes the next column of its row.

```
colspec || entry { color: navy; }
```

This selects every `entry` in a column described by a `colspec`, when the stylesheet gives `colspec` the display value `table-column` and `entry` the value `table-cell`.

Reference combinators

A reference combinator, from Selectors Level 5, selects an element that is referred to by an attribute of another element.

```
note /target/ step { color: navy; }
```

The name between slashes is the attribute that holds the reference. The rule selects every `step` whose `id` or `xml:id` equals the `target` attribute of a `note`. The attribute value can start with `#`.

Example:

```
<note target="s2">Check the cable first.</note>
<step id="s1">Open the cover.</step>
<step id="s2">Unplug the cable.</step>
```

The rule selects only the second step, `Unplug the cable.`, because the note refers to `s2`.

Pseudo-classes

A pseudo-class starts with `:` and tests something about an element that its name and attributes do not tell, such as its position among its siblings or its language.

Write a pseudo-class after a selector. The rule applies only to elements that pass the test:

```
chapter:first-child { color: navy; }
```

The selector reference lists every pseudo-class that CSSMedia accepts.

Child and sibling position

Structural pseudo-classes select elements by their position among the elements that share a parent. Text, comments and processing instructions are not counted.

```
chapter:first-child { color: navy; }
chapter:last-child { color: teal; }
chapter:only-child { color: gray; }
chapter:first-of-type { color: purple; }
```

`:first-child`, `:last-child` and `:only-child` count all sibling elements. The `-of-type` forms count only siblings with the same element name and namespace.

Nth-position selectors

An `:nth-*` pseudo-class selects elements by their numbered position. Positions start at one. `odd` and `even` select alternating elements, and `An+B` describes a repeating sequence.

```
chapter:nth-child(odd) { color: navy; }
chapter:nth-child(2n + 1) { color: teal; }
chapter:nth-child(2 of [status="active"]) { color: purple; }
```

The `of` clause of `:nth-child()` and `:nth-last-child()` limits the count to siblings that match a selector. The `-last-` forms count from the end.

`:nth-col()` and `:nth-last-col()` select table cells by the position of their column. Columns are found as described in Combinators.

Root, empty and scope

`:root` selects the root element. `:empty` selects an element with no child elements and no text other than white space. `:scope` selects the scoping element, which is the root element unless a rule sets another scope.

```
:root { color: navy; }
chapter:empty { display: none; }
:scope > chapter { color: teal; }
```

Selector-list and relational pseudo-classes

These pseudo-classes combine selectors. `:is()` matches an element that matches at least one selector of its list. `:where()` does the same but adds no specificity. `:not()` excludes the elements that match its list. `:has()` selects an element by the elements related to it.

```
chapter:not([status="draft"]) { color: navy; }
:is(chapter, appendix) > title { color: teal; }
:where([compact]) paragraph { font-size: 10pt; }
section:has(> figure) { color: purple; }
title:has(+ paragraph) { color: gray; }
```

An invalid selector in a `:not()` list makes the whole selector invalid. `:is()` and `:where()` ignore invalid selectors. `:matches()` is accepted as another name for `:is()`.

Language and direction

`:lang()` selects elements by language. The language of an element is its nearest `xml:lang` value, or the value that the `-cssmedia-lang` property gives. See Document language.

```
:lang(fr) { font-family: serif; }
```

`:dir(ltr)` and `:dir(rtl)` select elements by text direction. CSSMedia takes the direction from the first strong character in the element's text, as the Unicode Bidirectional Algorithm does. Without one, it uses the direction in which the element's language is written, and left-to-right when there is no language. The `direction` property does not change the result.

Links and document location

An XML element is a link only when you make it one with `-cssmedia-link`, a CSSMedia extension that sets the destination of an element. The destination can be a place in the same document or an external address. See Links.

`:link` and `:any-link` select the elements that have a destination. `:local-link` selects the links that point into the same document. An optional integer limits the number of path segments that are compared.

```
xref { -cssmedia-link: attr(target); }
:any-link { color: blue; }
:local-link { color: navy; }
```

States that do not occur in a PDF

A PDF has no pointer, keyboard focus, media playback, form controls or browsing history. Pseudo-classes that test those states are valid selectors, but they never match an element. Examples are `:hover`, `:focus`, `:visited`, `:target`, `:playing`, `:checked` and `:disabled`. The same holds for `:state()` and `:heading`, because an XML element has no custom state and no heading role.

`:read-only` and `:defined` match every element. The selector reference gives the result for each pseudo-class.

Pseudo-elements

A pseudo-element starts with `::` and selects a part of an element or a piece of generated content. It follows the element selector.

```
paragraph::first-letter { color: navy; }
paragraph::before { content: "Note: "; }
```

The selector reference lists every pseudo-element that CSSMedia accepts.

Text parts

`::first-line` selects the first formatted line of a block. `::first-letter` selects its first letter, together with the punctuation around it. `::first-letter::prefix` and `::first-letter::suffix` select that punctuation.

```
paragraph::first-line { font-weight: bold; }
paragraph::first-letter { font-size: 150%; }
```

Generated content, markers and footnotes

`::before` and `::after` are positions before and after the content of an element. They exist only when the `content` property gives them something to show.

```
paragraph::before { content: "Note: "; }
item::marker { color: navy; }
```

`::marker` selects the marker of a list item. `::before::marker` and `::after::marker` select the marker of generated content that is a list item.

`::footnote-call` and `::footnote-marker` select the reference that replaces a footnote in the text and the marker at the start of the footnote.

To copy the text of an element into a page margin box, use `string-set`. See Named strings.

Pseudo-elements without a target in a PDF

Some pseudo-elements select ranges or widgets that exist only on screen. `::selection`, `::search-text`, `::target-text`, `::spelling-error`, `::grammar-error` and `::highlight()` select highlighted text. `::placeholder`, `::file-selector-button`, `::details-content` and `::backdrop` select parts of form controls and interactive elements. CSSMedia accepts them, but an XML document has nothing for them to select.

Specificity

Specificity helps the cascade choose between declarations from matching rules. It is one part of the cascade, after origin, importance and layer priority.

Comparing selector specificity

Each part of a selector adds to its specificity, as Selectors Level 4 defines. An ID selector counts more than an attribute selector or a pseudo-class. Those count more than an element name or a pseudo-element.

```
chapter { color: gray; }
[role="notice"] { color: teal; }
#important-notice { color: navy; }
```

If all other cascade priorities are equal, the ID rule wins over the attribute rule, and the attribute rule wins over the element-name rule. If competing rules have the same specificity, source order decides and the later declaration wins.

`:where()` contributes no specificity. For `:is()`, `:not()` and `:has()`, specificity is based on the most specific selector in the argument. The `&` nesting selector contributes specificity according to the parent selector list.

An attribute selector such as `[class~="warning"]` has the specificity of an attribute selector. See Basic selectors and namespaces for how to select a `class` attribute in XML.

Selector reference

These tables list every selector that CSSMedia accepts. The `In a PDF` column says whether the selector matches elements while CSSMedia generates a PDF.

Basic selectors

Basic selectors match an element by its name, ID or attributes. See Basic selectors and namespaces.

Name	Description	Values	In a PDF	Specification
`E`	Type selector. Matches elements named `E`.	An element name.	Matches	Selectors 4 §5.1
`*`	Universal selector. Matches any element.		Matches	Selectors 4 §5.2
`#id`	ID selector. Matches the element whose `id` or `xml:id` attribute has this value.	An identifier.	Matches	Selectors 4 §6.7
`.name`	Class selector. Matches inside SVG and MathML content, which define a `class` attribute.	An identifier.	Matches in SVG and MathML only	Selectors 4 §6.6
`[attr]`	Matches elements that have the attribute.	An attribute name, with an optional namespace prefix.	Matches	Selectors 4 §6
`[attr=value]`	Matches when the attribute value is exactly `value`.	A string or an identifier. Add `i` to ignore ASCII case or `s` to require it.	Matches	Selectors 4 §6.1

Name	Description	Values	In a PDF	Specification
`[attr~=value]`	Matches when the value is one of the white-space separated words of the attribute.	A word without white space.	Matches	Selectors 4 §6.1
`[attr|=value]`	Matches when the attribute is `value` or starts with `value-`.	A string or an identifier.	Matches	Selectors 4 §6.1
`[attr^=value]`	Matches when the attribute starts with `value`.	A non-empty string.	Matches	Selectors 4 §6.2
`[attr$=value]`	Matches when the attribute ends with `value`.	A non-empty string.	Matches	Selectors 4 §6.2
`[attr*=value]`	Matches when the attribute contains `value`.	A non-empty string.	Matches	Selectors 4 §6.2
`ns|E`, `*|E`, `|E`	Limits a type selector to a declared namespace, to any namespace, or to no namespace. The same prefixes work in attribute selectors.	A prefix declared with `@namespace`.	Matches	Selectors 4 §5.3, CSS Namespaces 3 §3
`&`	Nesting selector. Stands for the selector of the parent rule.		Matches	CSS Nesting 1

Combinators

A combinator relates two selectors. See Combinators.

Name	Description	Values	In a PDF	Specification
`A B`	Descendant combinator. Matches `B` inside `A` at any depth.	Two selectors separated by white space.	Matches	Selectors 4 §14.1
`A > B`	Child combinator. Matches `B` when its parent is `A`.	Two selectors.	Matches	Selectors 4 §14.2

Name	Description	Values	In a PDF	Specification
`A + B`	Next-sibling combinator. Matches `B` when it directly follows `A`.	Two selectors.	Matches	Selectors 4 §14.3
`A ~ B`	Subsequent-sibling combinator. Matches `B` when `A` is an earlier sibling.	Two selectors.	Matches	Selectors 4 §14.4
`A || B`	Column combinator. Matches the cells `B` of the column that `A` describes.	Two selectors.	Matches	Selectors 5 §8.1
`A / attr/ B`	Reference combinator. Matches `B` when the attribute `attr` of `A` refers to its ID.	Two selectors and an attribute name between slashes.	Matches	Selectors 5 §9.1

Pseudo-classes

A pseudo-class tests a property of an element other than its name and attributes. See Pseudo-classes.

Name	Description	Values	In a PDF	Specification
`:root`	The root element of the document.		Matches	Selectors 4 §13.1
`:empty`	An element with no child elements and no text other than white space.		Matches	Selectors 4 §13.2
`:scope`	The scoping element. Without another scope, the root element.		Matches	Selectors 4 §8.4
`:first-child`, `:last-child`, `:only-child`	The first, last or only element among its siblings.		Matches	Selectors 4 §13.3
`:first-of-type`, `:last-of-type`, `:only-of-type`	The first, last or only element of its type among its siblings.		Matches	Selectors 4 §13.4

Name	Description	Values	In a PDF	Specification
`:nth-child()`, `:nth-last-child()`	Elements at a position among their siblings, counted from the start or from the end.	odd, even or An+B, optionally followed by `of` and a selector list.	Matches	Selectors 4 §13.3
`:nth-of-type()`, `:nth-last-of-type()`	Elements at a position among the siblings of the same type.	odd, even or An+B.	Matches	Selectors 4 §13.4
`:nth-col()`, `:nth-last-col()`	Table cells at a position among the columns of their table.	odd, even or An+B.	Matches	Selectors 5 §8.2, §8.3
`:is()`, `:matches()`	An element that matches any selector of the list. `:matches()` is the old name of `:is()`.	A selector list. Invalid selectors are ignored.	Matches	Selectors 4 §4.2
`:where()`	Works as `:is()` and adds no specificity.	A selector list. Invalid selectors are ignored.	Matches	Selectors 4 §4.4
`:not()`	An element that matches none of the selectors of the list.	A selector list. An invalid selector makes the whole selector invalid.	Matches	Selectors 4 §4.3
`:has()`	An element that has a related element matching the selector.	A relative selector list, such as `> figure`. It cannot contain another `:has()` or a pseudo-element.	Matches	Selectors 4 §4.5
`:lang()`	Elements in a language. The language comes from `xml:lang` or the `-cssmedia-lang` property.	One or more language ranges, as identifiers or strings.	Matches	Selectors 4 §7.2

Name	Description	Values	In a PDF	Specification
`:dir()`	Elements by text direction, read from the text and the language, not from the `direction` property.	`ltr` or `rtl`.	Matches	Selectors 4 §7.1
`:link`, `:any-link`	Elements that are hyperlinks, because `-cssmedia-link` gives them a destination.		Matches	Selectors 4 §8.1, §8.2
`:local-link`	Hyperlinks that point to the same document. The optional number limits how many path segments are compared.	An optional non-negative integer in parentheses.	Matches	Selectors 5 §2.1
`:read-only`, `:defined`	Every element is read-only and defined, because an XML element has no editable or undefined state.		Always matches	Selectors 4 §12.1.2, §5.4
`:hover`, `:active`, `:focus`, `:focus-visible`, `:focus-within`	User action states, such as pointing at an element or giving it keyboard focus.		Never matches	Selectors 4 §9
`:interest-source`, `:interest-target`	Source and target of a user's declared interest.		Never matches	Selectors 5 §3.1

Name	Description	Values	In a PDF	Specification
`:visited`, `:target`	Links that were visited, and the element that a link points to. A PDF has no browsing history and no current target.		Never matches	Selectors 4 §8.2, §8.3
`:playing`, `:paused`, `:seeking`, `:buffering`, `:stalled`, `:muted`, `:volume-locked`	Media playback, loading and sound states.		Never matches	Selectors 4 §10
`:current`, `:current()`, `:past`, `:future`	Elements in relation to a timeline, such as speech or video playback.	A selector list for `:current()`.	Never matches	Selectors 5 §5
`:open`, `:popover-open`, `:modal`, `:fullscreen`, `:picture-in-picture`	Display states of interactive elements.		Never matches	Selectors 4 §11
`:enabled`, `:disabled`, `:read-write`, `:placeholder-shown`, `:autofill`, `:default`, `:checked`, `:unchecked`, `:indeterminate`, `:blank`	Form control states. An XML element is never a form control.		Never matches	Selectors 4 §12, Selectors 5 §4.1
`:valid`, `:invalid`, `:in-range`, `:out-of-range`, `:required`, `:optional`, `:user-valid`, `:user-invalid`	Form validation states.		Never matches	Selectors 4 §12.3
`:state()`	A custom state that a document language exposes. XML has none.	An identifier.	Never matches	Selectors 5 §6

Name	Description	Values	In a PDF	Specification
`:heading`, `:heading()`	Headings, as a document language defines them. CSSMedia does not give XML elements a heading role.	A list of levels for `:heading()`.	Never matches	Selectors 5 §7

Pseudo-elements

A pseudo-element selects a part of an element or generated content. See Pseudo-elements.

Name	Description	Values	In a PDF	Specification
`::before`, `::after`	Generated content before or after the content of an element. The `content` property gives it.		Matches	CSS Pseudo-Elements 4 §4.1
`::marker`	The marker of a list item. Also `::before::marker` and `::after::marker`.		Matches	CSS Pseudo-Elements 4 §4.2
`::first-line`	The first formatted line of a block.		Matches	CSS Pseudo-Elements 4 §2.1
`::first-letter`	The first letter of a block, with the punctuation around it. `::first-letter::prefix` and `::first-letter::suffix` select that punctuation.		Matches	CSS Pseudo-Elements 4 §2.2
`::footnote-call`	The reference that replaces a footnote in the text. By default it is the value of the `footnote` counter, as a superscript.		Matches	CSS GCPM 3 §2.6

Name	Description	Values	In a PDF	Specification
`::footnote-marker`	The marker at the start of a footnote body. By default it is the value of the `footnote` counter, followed by a period and a space.		Matches	CSS GCPM 3 §2.7
`::selection`, `::search-text`, `::target-text`, `::spelling-error`, `::grammar-error`, `::highlight()`	Highlighted ranges created by selection, search, navigation, spelling and grammar checking, or scripts.	A custom identifier for `::highlight()`.	No range exists	CSS Pseudo-Elements 4 §3.1
`::placeholder`, `::file-selector-button`, `::details-content`	Parts of form controls and disclosure widgets.		No such part exists	CSS Pseudo-Elements 4 §4.3, §5.1, §5.2
`::backdrop`	The backdrop of an element shown in the top layer.		No such part exists	CSS Position 4 §3.2

Cascade and declarations

A stylesheet is a list of rules, and several rules often apply to the same element. This chapter explains how CSSMedia decides which rule wins, how elements get values from their parents, and how to write less CSS with shorthands, custom properties and conditions. The cascade reference at the end lists the at-rules and media features.

A CSS rule has a selector and a list of declarations between braces. The selector says which elements the rule is for. Each declaration has a property, a colon, a value and a semicolon:

```
chapter {
    color: navy;
    margin: 1em;
}
```

CSSMedia reads your stylesheets and applies them to your XML. It has almost no built-in rules of its own, so what you see in the PDF is what your rules say. A declaration that CSSMedia does not understand, because the property is unknown or the value is not valid, is ignored, and the other declarations still apply.

One element can match many rules, and two rules can set the same property with different values. The *cascade* is the set of steps that picks one value for each property of each element. It works on every property alone: one rule can win for `color` while another wins for `margin`.

The cascade picks a value when at least one rule sets the property. When no rule does, the element takes the value of its parent or the initial value of the property. After that, CSSMedia computes the final value. For example, it turns `2em` into a length in pixels and replaces `var()` with the value of a custom property.

The topics of this chapter follow what you need to know:

- Decide which rule wins
- Inherit, reset and roll back values
- Set many properties with a shorthand
- Reuse values with custom properties and functions
- Apply rules on conditions

Decide which rule wins

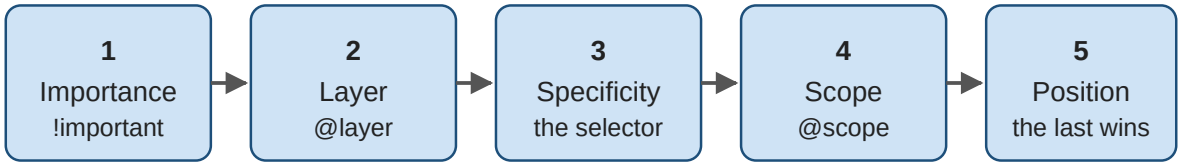
When two declarations set the same property on the same element, CSSMedia compares them one criterion at a time. The first criterion that tells them apart decides which one wins.

These are the criteria, in the order CSSMedia checks them:

1. Whether the declaration is `!important`.
2. The cascade layer of the rule.
3. The specificity of the selector.
4. The distance to the root of a scope.
5. The position in the stylesheet.

The order of the criteria

The criteria, in the order CSSMedia checks them



The first criterion in which two declarations differ decides.
The criteria after it are not used.

A criterion matters only when the declarations are equal in all the criteria before it. A rule that comes later in the stylesheet does not win against a more specific rule, and a more specific rule does not win against an important one.

CSSMedia has a few rules of its own for the markers of lists, the footnote calls and the attribute `xml:space`. Any rule you write wins over them, so you never need `!important` to override them.

Important declarations

Write `!important` after the value to make a declaration win against the normal declarations of other rules, whatever their layer, specificity or position:

```
title {
    color: red !important;
}

chapter title {
    color: blue;
}
```

The title is red, although the second selector is more specific. Between two important declarations, the usual criteria decide. `!important` belongs to the one declaration that carries it, not to the rule.

Cascade layers

A cascade layer is a group of rules that ranks together. Layers let you order whole groups of rules, for example the rules of a base design and the rules of a customer, without raising the specificity of any selector. Write the rules in `@layer` blocks, and name the layers first to set their order:

```
@layer base, theme;

@layer base {
    chapter { color: black; }
}

@layer theme {
    chapter { color: navy; }
}
```

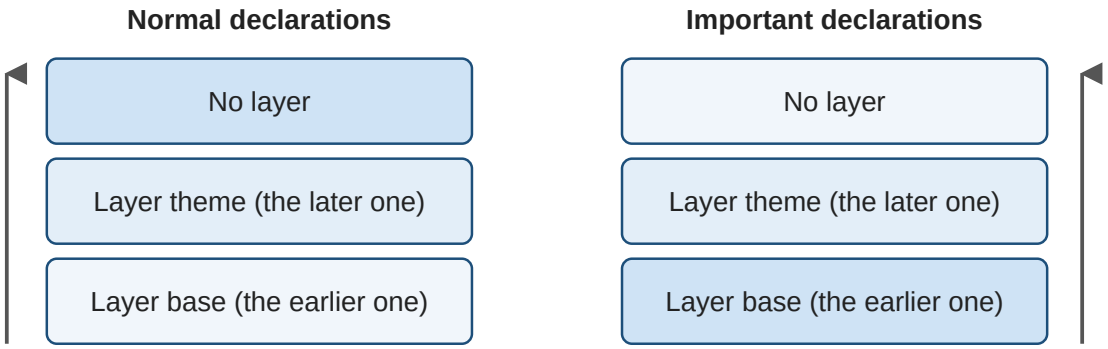
The layers rank in the order in which their names first appear, so `theme` is above `base`. A normal declaration in a later layer wins against a normal declaration in an earlier layer, even if the selector of the earlier one is more specific. Rules that are not in a layer rank above every layer:

```
@layer base {
    chapter title { color: red; }
}

title {
    color: blue;
}
```

The title is blue.

The order of layers



The arrows point up to the declaration that wins, in the darkest box.
With @layer base, theme; the layer base comes first.

For important declarations the order is reversed: an important declaration in an earlier layer wins against one in a later layer, and an important declaration in a layer wins against an important declaration outside the layers. This lets the first layer, which is usually the base design, protect the rules that must not change.

A layer can have layers inside it. Write the inner layer in the block of the outer one, or name it with a dot, as in `@layer outer.inner`. A block without a name makes a layer that no other rule can add to. Writing the name of a layer again adds more rules to the same layer. The `@import` rule can put a whole stylesheet in a layer, as Apply rules on conditions explains.

Specificity

When the importance and the layer are the same, the more specific selector wins. A selector with an ID counts more than a selector with a class or an attribute, and that counts more than a selector with only element names. Specificity explains how CSSMedia counts.

```
title {
    color: red;
}

chapter title {
    color: blue;
}
```

Both rules come from the same stylesheet and neither is important, so the specificity decides: the title is blue, even if you write the rules in the opposite order.

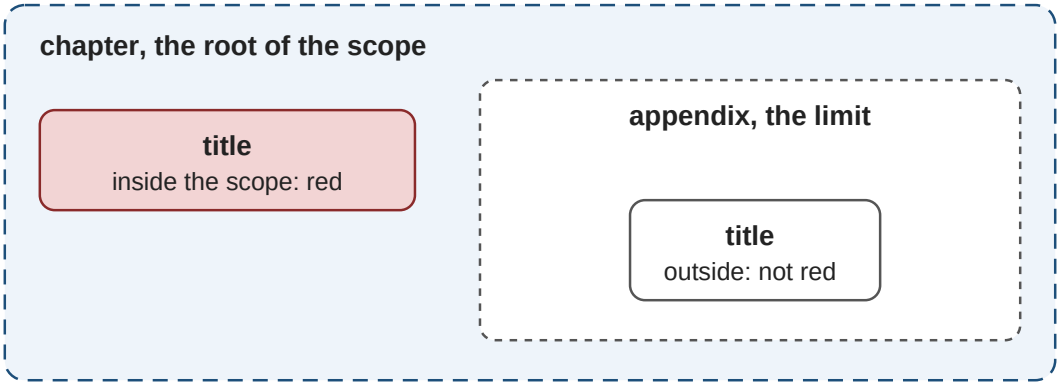
Scopes

An `@scope` block limits the rules inside it to the descendants of a root element. The selector in parentheses is the root. An optional `to` selector marks a limit: the elements that match it, and everything inside them, are outside the scope:

```
@scope (chapter) to (appendix) {
    title { color: navy; }
}
```

This rule colors the titles inside a `chapter`, except the titles inside an `appendix` that is in the chapter. If you leave out the root, the scope is the whole document.

A scope with a limit



@scope (chapter) to (appendix) { title { color: red; } }

A scope also breaks ties. When two declarations have the same importance, layer and specificity, the one from the root that is nearer to the element wins, whatever the position of the rules. A rule in a scope wins against a rule outside every scope. The selectors inside an `@scope` do not get more specificity from the root.

Position in the stylesheet

If everything else is equal, the declaration that comes last wins. The same holds inside one block: when a block sets a property twice, the second declaration wins. An imported stylesheet counts as being at the position of its `@import` rule, which is always before the rules of the stylesheet that imports it.

Declarations of pages

The declarations in `@page` rules and in the rules of the page-margin boxes cascade among themselves with the same criteria. Their selectors have a specificity of their own, which Page setup explains. They never compete with the declarations that style the elements of your XML.

Inherit, reset and roll back values

An element that no rule sets a property for still gets a value. It takes the value of its parent or the initial value of the property. You can ask for either, or for a value from an earlier step of the cascade, with the keywords `inherit`, `initial`, `unset`, `revert` and `revert-layer`.

Inheritance

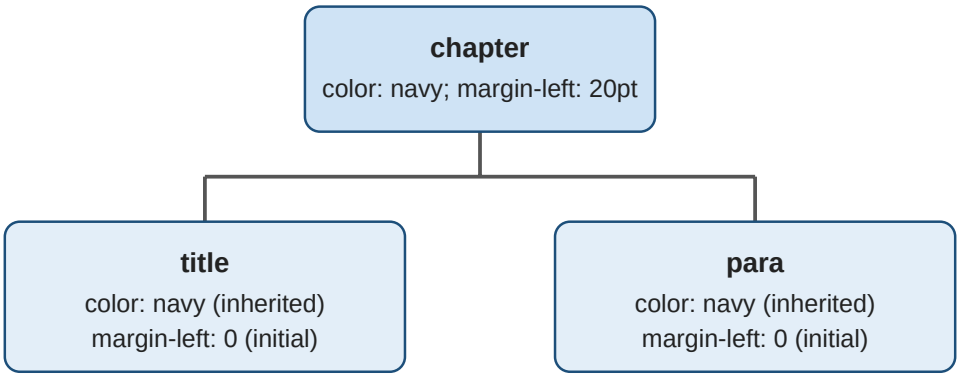
Some properties are inherited: an element takes the value that its parent has, unless a rule gives it another one. Text properties are inherited, such as `color`, `font-family`, `font-size`, `font-weight`, `line-height` and `text-align`. So you can set them once on an element high in the tree:

```
chapter {
    color: navy;
    font-size: 11pt;
}

title {
    color: teal;
}
```

The title is teal. All the other descendants of the chapter are navy and 11 points large.

An inherited and a not inherited property



The color flows down the tree. The margin does not:
margin-left is not inherited, so the children use its initial value.

Properties that describe boxes are not inherited, such as `margin`, `padding`, `border`, `background-color` and `display`. If a rule gives the chapter a border, its titles have no border. An element without a value for such a property uses the initial value of the property.

Inheritance follows the tree of your XML. The reference tables of this guide give the initial value of each property in the column **Default**.

Keywords that choose where a value comes from

You can write these keywords as the whole value of any property:

- `inherit` takes the value of the parent, even for a property that is not inherited. On the root element, which has no parent, it gives the initial value.

- `initial` takes the initial value of the property, even for a property that is inherited.
- `unset` is `inherit` for an inherited property, and `initial` for the others. It is the natural state of a property that no rule sets.
- `revert` removes your declarations for the property from this element, and lets the property have the value it would have without your stylesheet. CSSMedia has almost no rules of its own, so this is usually the same as `unset`.
- `revert-layer` removes the declarations of the layer of the rule, and lets the layers below it decide.

```
title {
    margin-left: inherit;
}

note {
    color: initial;
}
```

The title has the same left margin as its parent. The note is black, even if its parent is navy.

The keyword `revert-layer` is useful in a layer that adds to the one below it:

```
@layer base, theme;

@layer base {
    title { color: black; }
}

@layer theme {
    title { color: revert-layer; }
}
```

The rule of the layer `theme` gives the title the value that the layer `base` gives it. In a layer with nothing below, `revert-layer` acts as `revert`.

Resetting everything with all

The property `all` takes one of the keywords above and applies it to every property at once, except `direction`, `unicode-bidi` and custom properties:

```
note {
    all: initial;
    display: block;
}
```

This note starts without any style from the rules of its ancestors, and the declarations after `all` build it again. Remember that `all: initial` also resets `display`, which is `inline` initially, so write it again for an element that must be a block.

A value that is invalid when it is computed

A declaration with a value that CSSMedia cannot read is ignored before the cascade starts, so another declaration for the property can win. A declaration whose value uses `var()` is different, because CSSMedia can check it only after the cascade has chosen it. If the substituted value is

not valid for the property, for example because the custom property does not exist, the declaration acts as `unset`. CSSMedia does not go back to a declaration that lost:

```
title {
    color: red;
    color: var(--missing);
}
```

The title does not become red. The second declaration won, it is not valid, and the title takes the color of its parent. Write a fallback, as `var(--missing, red)`, to avoid this. Reuse values with custom properties and functions explains `var()`.

Set many properties with a shorthand

A shorthand property sets several properties in one declaration. Use it to write less CSS, and know what it does to the properties you leave out.

The properties that a shorthand sets are its longhands. The shorthand `margin`, for example, sets the four longhands `margin-top`, `margin-right`, `margin-bottom` and `margin-left`. The number of values you write decides how they are shared:

```
chapter { margin: 1em; }
chapter { margin: 1em 2em; }
chapter { margin: 1em 2em 3em; }
chapter { margin: 1em 2em 3em 4em; }
```

One value is for all four sides. Two values are for top and bottom, then right and left. Three values are for top, right and left, then bottom. Four values go clockwise from the top: top, right, bottom, left.

The shorthands `border`, `background`, `font`, `padding` and many others work in the same way, each with its own order of values. The topics Margins and padding, Borders and rounded corners, Backgrounds and Font selection and sizing explain the shorthands of their properties.

A shorthand resets what you leave out

A shorthand always sets all its longhands. The longhands that you do not write get their initial value, and that can replace a value that an earlier rule gave:

```
title {
    background-image: linear-gradient(white, silver);
}

title {
    background: yellow;
}
```

The second rule sets `background-image` to none too, so the title has a plain yellow background. Write the longhand after the shorthand when you need both.

Shorthands in the cascade

CSSMedia replaces a shorthand with its longhands at the position of the declaration. They compete with the other declarations like longhands that you wrote one by one:

```
chapter {
    margin-left: 2em;
    margin: 1em;
}

title {
    margin: 1em;
    margin-left: 2em;
}
```

In the chapter, the shorthand comes later, so the left margin is 1em. In the title, the longhand comes later, so the left margin is 2em and the other margins are 1em.

If you write `!important` after a shorthand, all its longhands are important. A declaration with a value that is not valid for the shorthand sets none of its longhands, and the earlier declarations stay. If you write a keyword such as `inherit` as the whole value of a shorthand, it applies to every longhand:

```
title {
    margin: inherit;
}
```

The shorthand for everything

The property `all` is a shorthand for all the properties. Inherit, reset and roll back values explains it.

Reuse values with custom properties and functions

A custom property holds a value that you write once and use in many declarations. You can give it a type, read values from the attributes of your XML, and define your own functions.

Procedure

1. Define a custom property

   The name of a custom property starts with two hyphens, as in `--accent`. Names are case sensitive, so `--Accent` and `--accent` are two properties. The value can be any text. A custom property is inherited, so define it on the root element or on any element, and every element below it can use it:

   ```
   doc {
       --accent: navy;
       --gap: 1em;
   }
   ```

2. Use the value with var()

   Write `var( )` with the name of the property. A second argument, after a comma, is the fallback for when the property has no value on that element. The fallback can use `var( )` too:

```
title {
    color: var(--accent, black);
    margin-left: calc(var(--gap) * 2);
}
```

CSSMedia replaces `var()` with the text of the property, so a custom property can hold part of a declaration, such as `20pt 0` for `margin`. Custom properties can use each other. If they use each other in a circle, none of them has a value, and the fallbacks are used.

A `var()` that has no value and no fallback makes the declaration invalid when it is computed, and the property takes the value `unset`. Inherit, reset and roll back values explains it.

3. Give a custom property a type

   An `@property` rule registers a custom property. It has these descriptors:

   - `syntax` is the type of the value, written as a string: `"<length>"`, `"<color>"`, `"<number>"`, `"<percentage>"` and the other types of CSS. A bar between types, as in `"<length> | auto"`, allows either. A plus sign after a type, as in `"<length>+"`, allows a list. `"*"` allows any value.
   - `inherits` is `true` or `false`. With `false` the elements below do not take the value, and they use the initial value.
   - `initial-value` is the value used when no rule sets the property.

```
@property --size {
    syntax: "<length>";
    inherits: false;
    initial-value: 20pt;
}

title {
    margin-left: var(--size);
}
```

A value that does not match the type is replaced by the initial value. A registered length is computed where you write it: if you write `--size: 2em` on the chapter, the titles inherit the length in pixels that 2em is for the chapter, whatever the font size of the title is. If you write the same `@property` rule more than once, the last one is used.

7. Read the attributes of your XML with attr()

   The function `attr()` takes the value of an attribute of the element that you style. Without a type it gives a string, which fits `content`. After the name of the attribute you can write the type of the value, in angle brackets, and CSSMedia reads the attribute as that type. The last argument is the fallback, used when the attribute is missing or cannot be read as the type:

```
figure::before {
    content: attr(title);
}

figure {
    width: attr(data-width <length>, 100pt);
    border-color: attr(tint <color>, black);
}
```

A string without a fallback is empty when the attribute is missing, and a value with a type has no fallback unless you write one. Place an image shows `attr()` with the type `url`.

8. Define your own functions with @function

   A `@function` rule defines a function whose name starts with two hyphens. The arguments have a name, and they can have a type and a default value. The descriptor `result` gives the value that the function returns, and it can use the arguments with `var()`. After `returns` you can write the type of the result. Inside the rule you can define custom properties for the steps of a calculation:

```
@function --double(--x <length>) returns <length> {
    result: calc(var(--x) * 2);
}

@function --pad(--x <length>: 4pt) returns <length> {
    --y: calc(var(--x) * 2);
    result: var(--y);
}

title {
    margin-left: --double(10pt);
    margin-right: --pad();
}
```

   A function can take several arguments, separated by commas, and can call other functions. An argument can be the value of a custom property, as in `--double(var(--gap))`. If an argument does not match its type, the declaration is invalid when it is computed.

Apply rules on conditions

A conditional rule holds rules that apply only when a condition is true. Use conditions to keep several designs in one stylesheet, to use a feature only where it exists, and to import stylesheets.

Procedure

1. Test the medium with @media

   An `@media` rule holds rules for a medium. CSSMedia makes a PDF, a paged medium, so the type `print` and the type `all` are true and the type `screen` is false. After the type you can test features of the medium in parentheses, joined with `and`. A comma separates alternatives, `not` reverses a whole query, and a query with no type has the type `all`:

```
@media print and (width >= 6in) {
    chapter { color: black; }
}

@media screen, (overflow-block: paged) {
    title { break-before: page; }
}
```

A feature can compare with >, <, >= and <=, and you can write a range such as `(100pt <= width <= 700pt)`. The features of the width and the height use the size of an A4 page, whatever size your @page rule sets. The cascade reference lists every media feature with the value that CSSMedia gives it.

2. Test what CSSMedia understands with @supports

   An `@supports` rule applies when CSSMedia accepts the declaration or selector in its condition. A stylesheet can use a new property and keep a fallback for the readers that do not know it. Write a declaration in parentheses, or a selector in `selector()`, and combine tests with `and`, `or` and `not`. The function `at-rule()` tests an at-rule:

   ```
   @supports (color: navy) and selector(chapter > title) {
       chapter > title { color: navy; }
   }

   @supports not (frobnicate: 1) {
       title { margin-left: 1em; }
   }
   ```

   The test is about the syntax, not about your document: `selector(chapter > title)` is true even if the document has no `chapter`.

   An `@supports-condition` rule gives a name to a group of tests, which you use afterwards as `(--name)` inside `@supports`. Define the name before you use it. If you define a name twice, the last definition is used:

   ```
   @supports-condition --basic-layout {
       display: block;
       color: navy;
   }

   @supports (--basic-layout) {
       chapter { display: block; }
   }
   ```

3. Combine tests with @when and @else

   An @when rule joins media tests and support tests in one condition. Write `media()` with one media feature, without a type, and `supports()` with a declaration. Combine them with `and`, `or` and `not`, and group them with parentheses. An `@else` rule that follows @when, `@media`, `@supports` or `@container` applies only when no branch before it applied. It can have a condition of its own, and without one it is the last fallback:

```
@when media(overflow-block: paged) and supports(color: navy) {
    title { color: navy; }
} @else supports(color: black) {
    title { color: black; }
} @else {
    title { color: inherit; }
}
```

Only the first branch with a true condition applies. An `@else` belongs to the conditional rule right before it, and any other rule between them ends the chain.

4. Name a media query with @custom-media

   An `@custom-media` rule gives a name that starts with two hyphens to a media query, or to `true` or `false`. Use the name in parentheses in a media query. A definition can use other names. If you define a name twice, the last definition is used:

   ```
   @custom-media --wide-page (width >= 6in);

   @media (--wide-page) {
       chapter { color: black; }
   }
   ```

5. Test an ancestor with @container

   An `@container` rule tests an ancestor of the element instead of the page. It applies to the nearest ancestor that qualifies. A size test, such as `(width >= 100pt)`, needs an ancestor with `container-type`. A style test, written with `style()`, compares a custom property of the ancestor, and any element can be the container for it. To choose one ancestor, give it a `container-name` and write the name before the condition:

   ```
   chapter {
       container-name: page-box;
       --theme: dark;
   }

   @container page-box style(--theme: dark) {
       title { color: white; }
   }
   ```

6. Import another stylesheet with @import

   An `@import` rule loads the rules of another stylesheet at that place. Write the URL, or a quoted string, first. After it, you can put the imported rules in a layer with `layer()`, test the support of a feature with `supports()` and name the media:

   ```
   @import "print.css" layer(print-theme) supports(color: navy) print;
   ```

   If the condition of an import is false, its rules do not apply. Write all the `@import` rules at the top of the stylesheet, before any other rule except `@charset` and layer statements. An `@import` after another rule is ignored. The rules you write after an import win against the imported rules of the same specificity, because they come later.

7. Declare namespaces with @namespace

An `@namespace` rule gives a prefix to the namespace of some of your elements, so that selectors can use it, as in `a|title`. It goes after the imports and before the other rules. Basic selectors and namespaces explains the selectors.

```
@namespace a url("urn:example:a");

a|title {
    color: navy;
}
```

The rules in a `@layer` block are not conditional, and Decide which rule wins explains them.

Cascade and declarations reference

This reference lists the property `all`, the at-rules of a stylesheet, the media features and the container features.

Property and at-rule reference

Name	Description	Values	Default	Specification
`all`	Resets all properties except `direction` and `unicode-bidi`.	`initial` | `inherit` | `unset` | `revert` | `revert-layer`		CSS Cascade 5 §3.2
`@charset`	Declares the stylesheet's fallback character encoding.	`@charset "<label>";`		CSS Syntax 3 §8.3
`@import`	Loads another stylesheet, optionally conditioned on a layer, support test or media query.	`@import <url-or-string> [layer | layer(<layer-name>)]? [supports(<condition>)]? <media-query-list>?;`		CSS Cascade 5 §2
`@namespace`	Declares a namespace prefix for selectors.	`@namespace <namespace-prefix>? <string-or-url>;`		CSS Namespaces 3 §3

Name	Description	Values	Default	Specification
`@layer`	Declares layer order or groups rules in a cascade layer.	`@layer <layer-name>? { <stylesheet> }` or `@layer <layer-name-list>;`		CSS Cascade 5 §6.4
`@property`	Registers a custom property with syntax, inheritance and optional initial value.	`@property <custom-property-name> { <declaration-list> }`		CSS Properties and Values API 1 §3
`syntax`	Defines the grammar of a registered custom property.	`<string>`		CSS Properties and Values API 1 §3.1
`inherits`	Sets whether a registered custom property inherits by default.	`true` | `false`		CSS Properties and Values API 1 §3.2
`initial-value`	Sets the registered custom property's initial value.	`<declaration-value>`	Guaranteed-invalid for `syntax: "*";` otherwise required	CSS Properties and Values API 1 §3.3
`@media`	Includes rules when a media query matches the PDF environment.	`@media <media-query-list> { <rule-list> }`		CSS Conditional 3 §5
`@supports`	Includes rules when CSSMedia accepts a declaration, selector or supported feature test.	`@supports <supports-condition> { <rule-list> }`		CSS Conditional 3 §6

Name	Description	Values	Default	Specification
`@supports-condition`	Defines a named group of support tests.	`@supports-condition <supports-condition-name> { <block-contents> }`		CSS Conditional 5 §8
`@container`	Includes rules when a selected ancestor container passes the query.	`@container <container-condition># { <block-contents> }`		CSS Conditional 5 §5.4
`@scope`	Limits the selectors in a rule list to a scope root and optional scope limit.	`@scope [(<scope-start>)]? [to (<scope-end>)]? { <rule-list> }`		CSS Cascade 6 §2.5
`@custom-media`	Gives a name to a media query list or boolean value.	`@custom-media <extension-name> [<media-query-list> | true | false];`		Media Queries 5 §10
`@when`	Includes rules when a generalized boolean condition is true.	`@when <boolean-condition> { <rule-list> }`		CSS Conditional 5 §3
`@else`	Supplies the next branch of a conditional chain.	`@else <boolean-condition>? { <rule-list> }`		CSS Conditional 5 §4
`@function`	Defines a custom CSS function.	`@function <name>(<parameters>) [returns <type>]? { <declaration-rule-list> }`		CSS Functions and Mixins 1 §2.1
`result`	Supplies a custom function's return value.	`<declaration-value>?`		CSS Functions and Mixins 1 §2.2

- ? means optional, | means choose one, # means one or more comma-separated items, || means one or more options in any order, and brackets group parts of the syntax.
- <label> is an encoding name.
- <url-or-string> and <string-or-url> each mean a URL or quoted string.
- <namespace-prefix> is the prefix bound to a namespace.
- <layer-name> names a cascade layer. <layer-name-list> is a comma-separated list of layer names.
- <stylesheet> is a CSS stylesheet.
- <custom-property-name> is a custom property name beginning with --.
- <declaration-list> is a list of declarations.
- <block-contents> is content inside a rule's braces, including declarations and nested rules.
- <declaration-value> is the value part of a declaration: one or more CSS tokens, such as navy, calc(1em + 2px) or var(--size). It cannot contain a top-level semicolon or !, since those delimit declarations and !important. Semicolons and exclamation marks inside functions are not top-level. See CSS Syntax 3.
- <string> is a quoted CSS string.
- <media-query-list> is one or more comma-separated media queries.
- <rule-list> is a sequence of rules.
- <condition> and <supports-condition> are support conditions made from feature tests and boolean operators.
- <supports-condition-name> names a reusable support condition.
- <container-condition> optionally names a query container and gives its query.
- <boolean-condition> combines media() and supports() tests.
- <scope-start> and <scope-end> are selector lists that set the scope roots and limits of @scope.
- <extension-name> names a custom media query.
- <name> in the function syntax is a function name beginning with --.
- <parameters> is a comma-separated list of function parameters.
- <type> is the optional syntax expected for the function result.
- <declaration-rule-list> contains declarations and nested rules.
- <number> in the function example is a CSS number, which can be an integer or include a fractional part.

The @charset rule must be the first rule in a stylesheet to affect byte decoding. Use @namespace before ordinary rules, and place @import rules before namespace declarations and ordinary rules.

Function names start with --. Parameters can have a syntax type and a default value. Use the result descriptor for the returned value:

```
@function --double(--value <number>) returns <number> {
    result: calc(var(--value) * 2);
}
```

`@supports` declaration tests use parentheses. `selector()` tests one complex selector. `font-tech()`, `font-format()` and `at-rule()` test supported features. A named condition is tested by placing its name inside parentheses:

```
@supports (color: navy) and selector(chapter > title) {
    chapter > title { color: navy; }
}

@supports-condition --has-navy {
    color: navy;
}

@supports (--has-navy) {
    chapter { color: navy; }
}
```

`@when` conditions combine `media()` and `supports()` tests with `and`, `or` and `not`. Parentheses group mixed operators:

```
@when media(width >= 6in) and supports(color: navy) {
    chapter { color: navy; }
}
```

Use `@else` immediately after an `@when`, `@media`, `@supports` or `@container` branch. An unrelated rule ends the chain.

Media features

Media feature values describe the page or PDF environment used for conditional rule evaluation. They do not describe a screen.

Name	Description	Values	Default	Specification
`any-hover`	Whether any pointing device can hover.	`none` | `hover`	none	Media Queries 5
`any-pointer`	The accuracy of any available pointing device.	`none` | `coarse` | `fine`	none	Media Queries 5
`aspect-ratio`	The ratio of page width to page height.	`<ratio>`	A4 page ratio	Media Queries 5
`color`	Bits per color component in the output.	`<integer>`	8	Media Queries 5
`color-gamut`	The approximate output color gamut.	`srgb` | `p3` | `rec2020`	`srgb`	Media Queries 5

Name	Description	Values	Default	Specification
`color-index`	Number of entries in the output color lookup table.	`<integer>`	`0`	Media Queries 5
`device-aspect-ratio`	The ratio of output width to output height.	`<ratio>`	A4 page ratio	Media Queries 5
`device-height`	Output height.	`<length>`	A4 page height	Media Queries 5
`device-width`	Output width.	`<length>`	A4 page width	Media Queries 5
`display-mode`	The presentation mode.	`fullscreen` | `standalone` | `minimal-ui` | `browser` | `picture-in-picture`	Not set	Media Queries 5
`dynamic-range`	The output's dynamic range.	`standard` | `high`	`standard`	Media Queries 5
`environment-blending`	Whether output blends with the surrounding environment.	`opaque` | `additive` | `subtractive`	`opaque`	Media Queries 5
`forced-colors`	Whether the output forces a limited color palette.	`none` | `active`	`none`	Media Queries 5
`grid`	Whether the output is grid-based.	`<mq-boolean>` (`0` or `1`)	`0`	Media Queries 5
`height`	Page height.	`<length>`	A4 page height	Media Queries 5
`horizontal-viewport-segments`	Number of horizontal viewport segments.	`<integer>`	`1`	Media Queries 5
`hover`	Whether the primary pointing device can hover.	`none` | `hover`	`none`	Media Queries 5

Name	Description	Values	Default	Specification
`inverted-colors`	Whether the output inverts colors.	`none` | `inverted`	`none`	Media Queries 5
`monochrome`	Number of bits per pixel in a monochrome output.	`<integer>`	`0`	Media Queries 5
`nav-controls`	Availability of navigation controls.	`none` | `back`	`none`	Media Queries 5
`orientation`	Whether the page is portrait or landscape.	`portrait` | `landscape`	`portrait`	Media Queries 5
`overflow-block`	How the output handles block-axis overflow.	`none` | `scroll` | `paged`	`paged`	Media Queries 5
`overflow-inline`	How the output handles inline-axis overflow.	`none` | `scroll`	`none`	Media Queries 5
`pointer`	The accuracy of the primary pointing device.	`none` | `coarse` | `fine`	`none`	Media Queries 5
`prefers-color-scheme`	The preferred color scheme.	`light` | `dark`	`light`	Media Queries 5
`prefers-contrast`	The preferred contrast level.	`no-preference` | `less` | `more` | `custom`	`no-preference`	Media Queries 5
`prefers-reduced-data`	Whether the user prefers reduced data use.	`no-preference` | `reduce`	`no-preference`	Media Queries 5
`prefers-reduced-motion`	Whether the user prefers reduced motion.	`no-preference` | `reduce`	`no-preference`	Media Queries 5
`prefers-reduced-transparency`	Whether the user prefers reduced transparency.	`no-preference` | `reduce`	`no-preference`	Media Queries 5
`resolution`	Output pixel density.	`<resolution>` | `infinite`	`infinite`	Media Queries 5

Name	Description	Values	Default	Specification
`scan`	The output's scanning process.	`interlace` | `progressive`	Not set	Media Queries 5
`scripting`	Whether scripting is available.	`none` | `initial-only` | `enabled`	`none`	Media Queries 5
`update`	How frequently the output can update.	`none` | `slow` | `fast`	`none`	Media Queries 5
`vertical-viewport-segments`	Number of vertical viewport segments.	`<integer>`	`1`	Media Queries 5
`video-color-gamut`	The color gamut of the output video.	`srgb` | `p3` | `rec2020`	`srgb`	Media Queries 5
`video-dynamic-range`	The dynamic range of the output video.	`standard` | `high`	`standard`	Media Queries 5
`width`	Page width.	`<length>`	A4 page width	Media Queries 5

- `<ratio>` is a ratio of two numbers.
- `<integer>` is a whole number.
- `<length>` is a CSS length, such as `12pt` or `2em`.
- `<mq-boolean>` is an integer with the value `0` or `1`.
- `<resolution>` is a pixel density, such as `300dpi`.

Container size features

A size query in `@container` tests the size of the nearest ancestor that has `container-type`. An ancestor with `size` answers all these features, and an ancestor with `inline-size` answers only the features of the inline axis.

Name	Description	Values	Default	Specification
`width`	The width of the content box of the container.	A length, alone or in a comparison or a range	The size of the container	CSS Conditional 5 §6.1
`height`	The height of the content box of the container.	A length, alone or in a comparison or a range	The size of the container	CSS Conditional 5 §6.1

Name	Description	Values	Default	Specification
`inline-size`	The size of the content box of the container along the lines of text.	A length, alone or in a comparison or a range	The size of the container	CSS Conditional 5 §6.1
`block-size`	The size of the content box of the container across the lines of text.	A length, alone or in a comparison or a range	The size of the container	CSS Conditional 5 §6.1
`aspect-ratio`	The ratio of the width to the height of the container.	A ratio, alone or in a comparison or a range	The ratio of the container	CSS Conditional 5 §6.1
`orientation`	Whether the container is wider than high.	`landscape` | `portrait`	The orientation of the container	CSS Conditional 5 §6.1

Container style queries

A container style query uses `style()` to test a computed style or custom property on an ancestor. See Conditional rules.

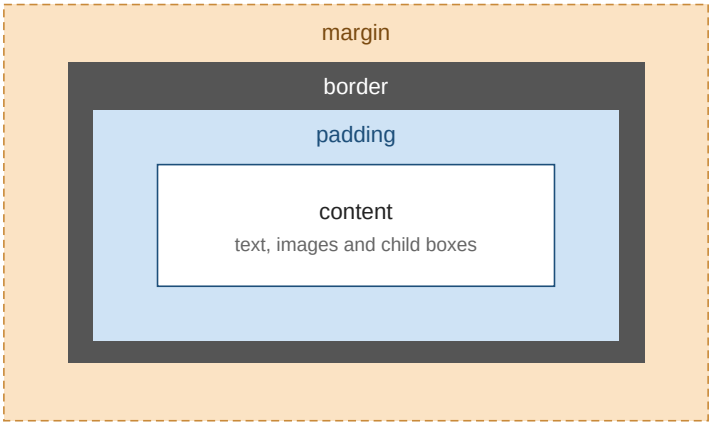
Boxes, backgrounds and borders

Every element of your document makes a box on the page. This chapter explains how to space and size boxes, draw their borders and fill them with colors and images. The boxes reference at the end lists every property.

A box is the rectangle that CSSMedia reserves on the page for an element and its content. It has four areas, from the inside to the outside:

- The content is the text, the images and the child boxes.
- The padding is the space around the content, inside the border.
- The border is the line around the padding.
- The margin is the space outside the border, between this box and the boxes around it.

The four areas of a box



Any of the areas can be zero. The border, the padding and the margin are zero until you set them, and a box has no color until you give it a background.

The element names in the examples of this chapter, such as `note`, are placeholders. Use the names that occur in your XML.

Lengths and percentages

Most of the properties of this chapter take a length, which is a number with a unit, such as `12pt`. A length can be negative only where a property says so. A `0` needs no unit. These are the units that you will use most:

Unit	Meaning
`pt`	Point. 72pt are 1 inch.
`pc`	Pica. 1pc is 12pt.
`px`	CSS pixel. 96px are 1 inch, so 1px is 0.75pt.
`in`, `cm`, `mm`, `Q`	Inch, centimeter, millimeter and quarter millimeter.

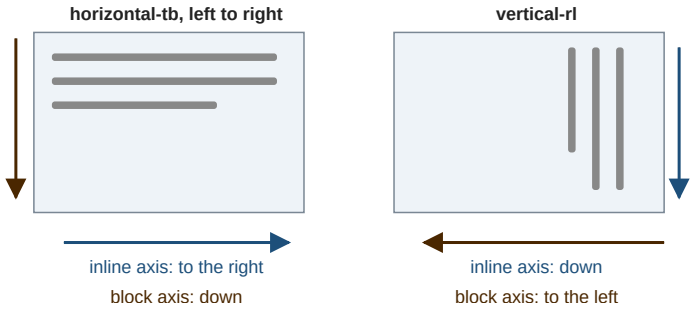
Unit	Meaning
`em`	The font size of the element. `1em` is a different length in elements with different font sizes.
`rem`	The font size of the root element.

A percentage is a part of another size, and the property decides which one. For the width of a box it is a part of the width of the containing block. The margins and the padding of all four sides also use the width of the containing block.

Physical and logical sides

Many properties come in two forms. The physical form names a side of the page: `margin-left` is always on the left. The logical form names a side of the text: `margin-inline-start` is where a line of text starts. The inline axis is the direction of a line of text, and the block axis is the direction in which lines are stacked.

The inline and block axes



In horizontal text that reads from left to right, the inline start is the left and the block start is the top. The writing mode and the direction of the text change that. Use the logical properties when a document has text in more than one writing mode or direction, so that the boxes follow the text.

The topics of this chapter follow the areas of a box:

- Margins and padding
- Box sizes
- Borders and rounded corners
- Backgrounds

Margins and padding

Margins put space between a box and the boxes around it. Padding puts space between the content of a box and its border.

Procedure

1. Set the margins

   The `margin` property sets the space outside the border. It takes one to four values:

   - One value sets all four sides.

- Two values set the top and the bottom, then the left and the right.
- Three values set the top, then the left and the right, then the bottom.
- Four values set the top, the right, the bottom and the left, clockwise from the top.

```
note {
    margin: 12pt 24pt;
}
```

Each side also has its own property: `margin-top`, `margin-right`, `margin-bottom` and `margin-left`. A margin can be a length, a percentage or `auto`, and it can be negative. The initial value is 0.

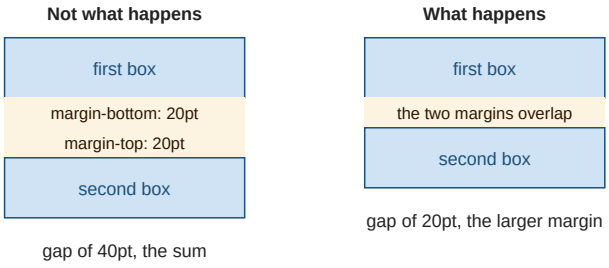
A percentage is a part of the width of the containing block, also for the top and the bottom margins.

An `auto` margin takes the free space that is left in the containing block. When a box has a width and both side margins are `auto`, the box is centered:

```
figure {
    width: 300pt;
    margin: 0 auto;
}
```

> **Note**
>
> The vertical margins of boxes that follow each other collapse. The space between them is the larger of the two margins, not their sum. Two margins of 20pt leave 20pt, not 40pt.

Collapsing margins



6. Set the margins by the direction of the text

   The logical margin properties name the sides by the direction of the text, so they follow the writing mode of the element. `margin-block-start` and `margin-block-end` are the sides at the start and the end of the block axis. `margin-inline-start` and `margin-inline-end` are the sides at the start and the end of a line. `margin-block` and `margin-inline` set the two sides of an axis, the start first. With one value, both sides are equal.

```
step {
    margin-block: 6pt 12pt;
    margin-inline: 18pt;
}
```

In horizontal text that reads from left to right, this step has 6pt above, 12pt below and 18pt on the left and on the right.

7. Set the padding

   The `padding` property takes the same one to four values as `margin`, but it cannot be negative, `auto` does not exist for it, and the padding of two boxes never collapses. The background of a box is painted under its padding, and not under its margin.

```
note {
    padding: 6pt 12pt;
}
```

   The properties `padding-top`, `padding-right`, `padding-bottom` and `padding-left` set one side. The logical ones are `padding-block-start`, `padding-block-end`, `padding-inline-start`, `padding-inline-end`, and the pairs `padding-block` and `padding-inline`. The initial value is 0. A percentage is a part of the width of the containing block.

8. Choose what happens to a margin at a page break

   A box can break across two pages. The `margin-break` property decides what happens to the margins that touch the break:

   - `auto` is the initial value. At a break that CSSMedia chooses, a margin before the break is cut to the space that is left on the page and a margin after it is zero. At a break that you force, the margin before the break is cut and the margin after it stays.
   - `keep` keeps the margins.
   - `discard` removes the margins at the break, also at the top of the first page and at the bottom of the last one.

```
chapter {
    margin-break: discard;
}
```

Box sizes

By default a box is as wide as the box that contains it and as high as its content. Use the size properties to give a box another size, or to limit its size.

Procedure

1. Set the width and the height

   The `width` and `height` properties set the preferred size of a box. Their initial value is `auto`, which lets layout choose the size. A size can be a length, such as `200pt`, or a percentage of the size of the containing block.

```
figure {
    width: 60%;
    height: 120pt;
}
```

   The properties `inline-size` and `block-size` do the same by the direction of the text: the inline size is the width and the block size is the height in horizontal text.

2. Choose what the size measures

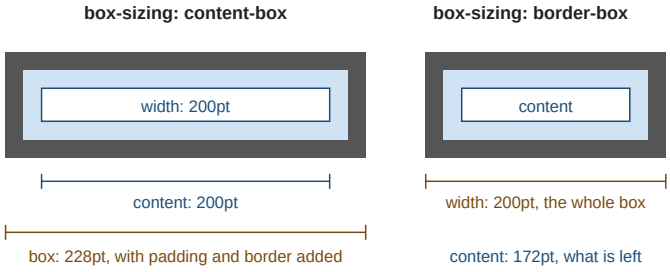
   The `box-sizing` property tells what a width or a height measures:

   - `content-box` is the initial value. The size is the size of the content. The padding and the border are added outside it.
   - `border-box` makes the size include the padding and the border, so the content is smaller.

```
note {
    width: 200pt;
    padding: 12pt;
    border: 2pt solid black;
    box-sizing: border-box;
}
```

   This note is 200pt wide from the outside of one border to the outside of the other. With `content-box`, it would be 228pt wide.

   content-box and border-box



5. Size a box from its content

   These keywords can replace a length in any size property:

   - `min-content` is the smallest size that the content can have without overflowing. For text, it is the width of the longest word.
   - `max-content` is the size that the content has when nothing wraps.
   - `fit-content` is the available space, but not less than `min-content` and not more than `max-content`. `fit-content(limit)` uses a length or a percentage as the available space.
   - `stretch` makes the margin box fill the available space.
   - `contain` fills the available space and keeps the aspect ratio of the box, if it has one.
   - `calc-size()` calculates a size from another size. `calc-size(auto, size / 2)` is half of the size that `auto` gives.

```
label {
    display: inline-block;
    width: fit-content(150pt);
}
```

12. Limit the size

Minimum and maximum sizes limit the preferred size. `min-width`, `min-height`, `min-inline-size` and `min-block-size` take the same values as `width`. Their initial value is `auto`. `max-width`, `max-height`, `max-inline-size` and `max-block-size` take the same values, and none too, which is their initial value. When a minimum is larger than a maximum, the minimum wins.

```
illustration {
    width: 100%;
    max-width: 40em;
}
```

This illustration fills the line, but is never wider than 40em.

13. Set both sizes at once

    The `size` property sets the width and the height. `min-size` and `max-size` set the two minimum sizes and the two maximum sizes. They take one or two values, and one value applies to both. With two values, the width comes first.

    ```
    icon {
        size: 24pt;
    }
    ```

14. Keep a proportion

    The `aspect-ratio` property gives a box a ratio between its width and its height. When you set one of the two sizes, CSSMedia calculates the other. Write the ratio as two numbers separated by a slash, such as `16 / 9`, or as one number. A single number is a ratio to 1. The initial value is `auto`, which uses the ratio of an image if the box has one.

    ```
    video-frame {
        width: 320pt;
        aspect-ratio: 16 / 9;
    }
    ```

    This frame is 320pt wide and 180pt high.

15. Give a size to a box that ignores its content

    A box with `contain: size` is sized as if it were empty. Use `contain-intrinsic-width`, `contain-intrinsic-height`, `contain-intrinsic-inline-size`, `contain-intrinsic-block-size`, or `contain-intrinsic-size` for both, to give it the size that it should have instead. They take a length, or none, which is the initial value.

Borders and rounded corners

A border is a line around the padding of a box. Each side has a style, a width and a color, and the corners can be rounded.

Procedure

1. Draw a border

   The `border` property sets the width, the style and the color of the four borders. Write the three values in any order:

```
note {
    border: 1pt solid #336699;
}
```

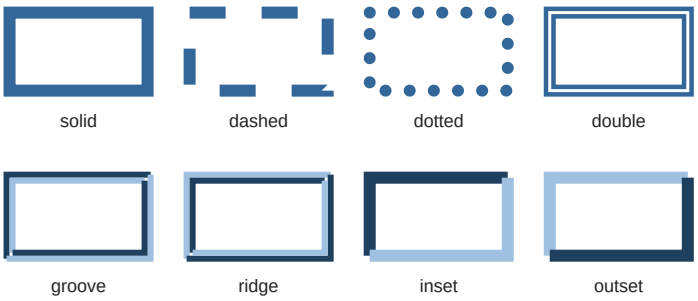
The style is the value that turns a border on. The initial style is none, and a border with that style has no width, whatever width you set. If a border does not show, check that it has a style.

2. Choose a style

   The `border-style` property takes one to four of these values:

   - none and `hidden` draw nothing.
   - `solid` is a continuous line.
   - `dashed` is a line of dashes, and `dotted` a line of round dots.
   - `double` is two lines with a space between them. The lines and the space are a third of the width each.
   - `groove` and `ridge` are two lines of a lighter and a darker shade of the color. A groove looks cut into the page and a ridge looks raised.
   - `inset` and `outset` are one line, darker on the top and the left for an inset, and on the bottom and the right for an outset.

   The border styles



   With two to four values, the sides take them as in `margin`: one value sets all sides, two values set top and bottom, then left and right, and so on.

```
note {
    border-style: solid none;
}
```

This note has a border at the top and at the bottom only.

9. Set the width

   The `border-width` property takes one to four widths. A width is a length that is not negative, or one of three keywords: `thin` is 1px, `medium` is 3px and `thick` is 5px. A pixel is 0.75pt. The initial width is `medium`.

```
note {
    border-style: solid;
    border-width: 0.5pt 3pt;
}
```

10. Set the color

    The `border-color` property takes one to four colors. Its initial value is `currentcolor`, which is the color of the text of the element, so a border that has no color is drawn in the color of the text.

    ```
    note {
        border: 2pt solid;
        border-color: #336699 #999999;
    }
    ```

    Where two sides of different colors meet, the change of color is on the diagonal from the outer corner to the inner corner.

11. Set one side

    Each side has properties of its own. The physical sides are `top`, `right`, `bottom` and `left`, and the logical sides, which follow the direction of the text, are `block-start`, `block-end`, `inline-start` and `inline-end`. For each side there are four properties:

    - `border-side` sets the width, the style and the color, like `border`.
    - `border-side-width`, `border-side-style` and `border-side-color` set one part.

    The properties `border-block` and `border-inline` set the two sides of an axis. They also have the three parts, such as `border-inline-width`, which takes one or two values.

    ```
    heading {
        border-block-end: 0.5pt solid gray;
    }
    ```

    This heading has a thin line under it in horizontal text, and a line at the end of the block axis in any other writing mode.

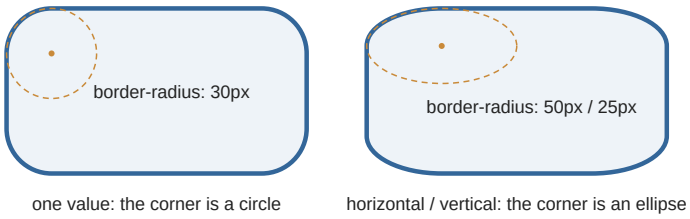
14. Round the corners

    The `border-radius` property sets the radius of the four corners. A radius is a length or a percentage that is not negative. A percentage is a part of the width of the box for the horizontal radius and of its height for the vertical one. The values are read like those of `margin`: one value sets all corners, and with more values the first is the top left corner and the rest follow clockwise.

    ```
    note {
        border: 1pt solid black;
        border-radius: 6pt;
    }
    ```

    A corner can be an ellipse. Write the horizontal radii, a slash and the vertical radii. `border-radius: 12pt / 6pt` gives each corner a horizontal radius of 12pt and a vertical radius of 6pt.

A circular and an elliptical corner



To set one corner, use `border-top-left-radius`, `border-top-right-radius`, `border-bottom-right-radius` or `border-bottom-left-radius`. They take one value for a circle or two for an ellipse. The properties `border-start-start-radius`, `border-start-end-radius`, `border-end-start-radius` and `border-end-end-radius` name the corners by the direction of the text. The properties `border-top-radius`, `border-right-radius`, `border-bottom-radius`, `border-left-radius`, `border-block-start-radius`, `border-block-end-radius`, `border-inline-start-radius` and `border-inline-end-radius` set the two corners of one side.

The `corner` property and the properties `corner-top-left`, `corner-top` and the like set the radius of a corner or of a side too.

The rounded corner clips the background of the box, and the border follows the curve. If the radii of a side are larger than the side, CSSMedia makes all the radii of the box smaller in the same proportion, until they fit. The inner edge of a border has a radius that is the outer radius less the width of the border.

Backgrounds

A background is a color, one or more images, or both, painted behind the content of a box. Use it to fill a box, to add a texture or a watermark, and to repeat a drawing on every page.

About this task

A background has up to two kinds of parts. The color is a single layer below everything else. The images are layers above the color, and you can have as many of them as you need. The first image of a list is the top layer.

Procedure

1. Set the color

   The `background-color` property paints the whole background area with one color. It takes any CSS color, such as a name, a hexadecimal value or `rgb()`. The initial value is `transparent`.

   ```
   note {
       background-color: #f2f2f2;
   }
   ```

   The background of the root element fills the page area, which is the part of the page inside the page margins. To paint the page margins too, set a background in an @page rule.

2. Add images

   The `background-image` property takes a list of images separated by commas. An image can be a file, written as `url()`, or a gradient such as `linear-gradient()`, `radial-gradient()`, `conic-gradient()` and their repeating forms. The value `none` is a layer without an image, and it is the initial value.

   ```
   chapter {
       background-image: url("watermark.svg"), linear-gradient(white, #dddddd);
   }
   ```

   The watermark is above the gradient, and the background color is below both. A relative `url()` is found from the location of the stylesheet.

3. Place an image

   An image is placed in the positioning area of the box. The `background-origin` property chooses it: `padding-box`, which is the initial value, `border-box` or `content-box`.

   The `background-position` property moves the image inside that area. A value can be a keyword (`left`, `center`, `right`, `top` and `bottom`), a length or a percentage. With one value, the second is `center`. With two values, the first is the horizontal position and the second is the vertical one. A keyword followed by a length is an offset from that side. The initial value is `0% 0%`, the top left corner.

   A percentage aligns the same point of the image and of the area. `50%` centers the image and `100%` puts its right edge on the right edge of the area.

   ```
   chapter {
       background-position: right 12pt bottom 12pt;
   }
   ```

   The properties `background-position-x` and `background-position-y` set one axis. `background-position-inline` and `background-position-block` set an axis by the direction of the text, with `start` and end instead of the side keywords.

4. Size an image

   The `background-size` property sets the size of an image. The initial value, `auto`, is the size of the image. These are the other values:

   - A length or a percentage of the positioning area sets the width. A second value sets the height. If you give only one, the height is `auto`, and CSSMedia keeps the proportions of the image.
   - `cover` makes the image as small as possible so that it covers the whole area. A part of the image can be left out.
   - `contain` makes the image as large as possible so that all of it fits in the area. A part of the area can stay empty.

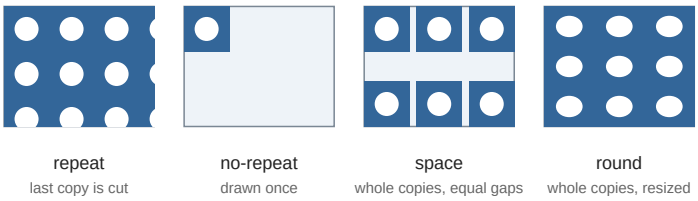
   ```
   chapter {
       background-size: cover;
   }
   ```

8. Repeat an image

An image repeats until it fills the area. The `background-repeat` property sets how, with one value for both axes, or two values for the horizontal and the vertical axis:

- `repeat` is the initial value. It repeats the image and cuts the last copy at the edge.
- `no-repeat` draws the image once.
- `space` draws as many whole copies as fit and spreads them with equal space between them.
- `round` draws as many whole copies as fit, and resizes them so that they fill the area exactly.

The repeat styles



The values `repeat-x` and `repeat-y` repeat on one axis only, and `repeat-inline` and `repeat-block` do it by the direction of the text. The properties `background-repeat-x`, `background-repeat-y`, `background-repeat-inline` and `background-repeat-block` set one axis.

```
chapter {
    background-image: url("watermark.svg");
    background-repeat: no-repeat;
}
```

13. Attach an image to the page

    The `background-attachment` property is `scroll` by default, which attaches the image to its box. With `fixed`, the image is attached to the page, so it is placed the same way on every page where the box has a background. A value of `local` works as `scroll`, because a page does not scroll.

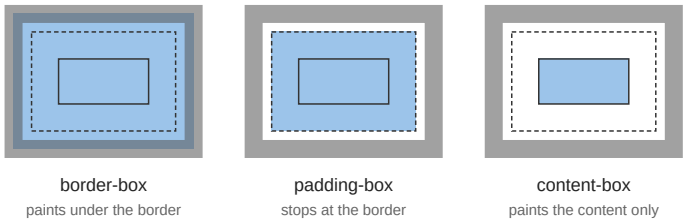
    ```
    chapter {
        background-attachment: fixed;
    }
    ```

14. Clip the background

    The `background-clip` property chooses the area where the background is painted:

    - `border-box` is the initial value. The background goes under the border.
    - `padding-box` stops it at the inner edge of the border.
    - `content-box` paints only the content area.
    - `border-area` paints only under the border.
    - `text` paints only inside the shapes of the letters of the box, which makes text that is filled with an image or a gradient. You can combine `border-area` and `text`.

The areas of background-origin and background-clip




```
title {
    background-image: linear-gradient(#336699, #cc3333);
    background-clip: text;
    color: transparent;
}
```

20. Blend the layers

    The `background-blend-mode` property sets how each image layer mixes with the layers below it and with the color. The modes are `normal`, which is the initial value, `multiply`, `screen`, `overlay`, `darken`, `lighten`, `color-dodge`, `color-burn`, `hard-light`, `soft-light`, `difference`, `exclusion`, hue, `saturation`, `color` and `luminosity`. Give one mode for each layer, separated by commas. When the list is shorter than the list of images, it repeats.

    ```
    chapter {
        background-color: #ffeecc;
        background-image: url("texture.png");
        background-blend-mode: multiply;
    }
    ```

21. Set everything at once

    The `background` property sets all the background properties in one declaration. Separate the layers with commas, and put the color in the last layer. In a layer, write a position, then a slash and a size, and in any order the image, the repeat style, the attachment and one or two boxes. One box sets the origin and the clip. Two boxes set the origin first and the clip second. A property that you leave out goes back to its initial value, `background-blend-mode` included.

    ```
    chapter {
        background: url("watermark.svg") right bottom / 120pt no-repeat, #f2f2f2;
    }
    ```

    The `background-tbd` property is like `background`, but it takes no color and leaves the color as it is.

Boxes, backgrounds and borders reference

This reference lists the properties that CSSMedia uses for the margins, padding, size, borders and backgrounds of a box.

Margins

Name	Description	Values	Default	Specification
`margin`	Sets the space outside the four sides of the box. One value applies to all sides. Two values set top and bottom, then left and right. Three set top, then left and right, then bottom. Four set top, right, bottom and left.	One to four lengths or percentages, or `auto`	`0`	CSS Box Model 4 §3.2
`margin-top`	Sets the space above the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1
`margin-right`	Sets the space at the right of the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1
`margin-bottom`	Sets the space below the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1
`margin-left`	Sets the space at the left of the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1
`margin-block`	Sets the margins at the start and the end of the block axis.	One or two lengths, percentages or `auto`	`0`	CSS Logical Properties 1 §4.2
`margin-block-start`	Sets the margin at the start of the block axis.	A length, a percentage or `auto`	`0`	CSS Logical Properties 1 §4.2
`margin-block-end`	Sets the margin at the end of the block axis.	A length, a percentage or `auto`	`0`	CSS Logical Properties 1 §4.2
`margin-inline`	Sets the margins at the start and the end of the inline axis.	One or two lengths, percentages or `auto`	`0`	CSS Logical Properties 1 §4.2
`margin-inline-start`	Sets the margin at the start of the inline axis.	A length, a percentage or `auto`	`0`	CSS Logical Properties 1 §4.2
`margin-inline-end`	Sets the margin at the end of the inline axis.	A length, a percentage or `auto`	`0`	CSS Logical Properties 1 §4.2

Name	Description	Values	Default	Specification
`margin-break`	Chooses what happens to the margins of a box where a page or column break touches them.	`auto`, `keep` or `discard`	`auto`	CSS Fragmentation 4 §5.2
`margin-trim`	Removes the margins of the children that touch the edges of the container.	`none`, `block`, `inline`, or one or more of `block-start`, `block-end`, `inline-start` and `inline-end`	`none`	CSS Box Model 4 §3.3

Padding

Name	Description	Values	Default	Specification
`padding`	Sets the space between the content and the border on the four sides. The values are read like those of `margin`.	One to four non-negative lengths or percentages	`0`	CSS Box Model 4 §4.2
`padding-top`	Sets the space between the content and the top border.	A non-negative length or percentage	`0`	CSS Box Model 4 §4.1
`padding-right`	Sets the space between the content and the right border.	A non-negative length or percentage	`0`	CSS Box Model 4 §4.1
`padding-bottom`	Sets the space between the content and the bottom border.	A non-negative length or percentage	`0`	CSS Box Model 4 §4.1
`padding-left`	Sets the space between the content and the left border.	A non-negative length or percentage	`0`	CSS Box Model 4 §4.1
`padding-block`	Sets the padding at the start and the end of the block axis.	One or two non-negative lengths or percentages	`0`	CSS Logical Properties 1 §4.4
`padding-block-start`	Sets the padding at the start of the block axis.	A non-negative length or percentage	`0`	CSS Logical Properties 1 §4.4
`padding-block-end`	Sets the padding at the end of the block axis.	A non-negative length or percentage	`0`	CSS Logical Properties 1 §4.4

Name	Description	Values	Default	Specification
`padding-inline`	Sets the padding at the start and the end of the inline axis.	One or two non-negative lengths or percentages	`0`	CSS Logical Properties 1 §4.4
`padding-inline-start`	Sets the padding at the start of the inline axis.	A non-negative length or percentage	`0`	CSS Logical Properties 1 §4.4
`padding-inline-end`	Sets the padding at the end of the inline axis.	A non-negative length or percentage	`0`	CSS Logical Properties 1 §4.4

Preferred sizes

Name	Description	Values	Default	Specification
`box-sizing`	Chooses what `width` and `height` measure. `content-box` measures the content, and the padding and the border are added. `border-box` measures the box through the border.	`content-box` or `border-box`	`content-box`	CSS Sizing 3 §3.3
`width`	Sets the preferred width of the box.	`auto`, a non-negative length or percentage, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, `calc-size()` or `anchor-size()`	`auto`	CSS Sizing 3 §3.1
`height`	Sets the preferred height of the box.	The values of `width`	`auto`	CSS Sizing 3 §3.1
`inline-size`	Sets the preferred size along the inline axis, which is the width in horizontal text.	The values of `width`	`auto`	CSS Logical Properties 1 §4.1
`block-size`	Sets the preferred size along the block axis, which is the height in horizontal text.	The values of `width`	`auto`	CSS Logical Properties 1 §4.1

Name	Description	Values	Default	Specification
`size`	Sets `width` and `height`. One value applies to both.	One or two values of `width`	`auto`	CSS Sizing 4 §3.1

Minimum and maximum sizes

Name	Description	Values	Default	Specification
`min-width`	Sets the smallest width of the box.	The values of `width`	`auto`	CSS Sizing 3 §3.1
`min-height`	Sets the smallest height of the box.	The values of `width`	`auto`	CSS Sizing 3 §3.1
`min-inline-size`	Sets the smallest size along the inline axis.	The values of `width`	`auto`	CSS Logical Properties 1 §4.1
`min-block-size`	Sets the smallest size along the block axis.	The values of `width`	`auto`	CSS Logical Properties 1 §4.1
`min-size`	Sets `min-width` and `min-height`. One value applies to both.	One or two values of `min-width`	`auto`	CSS Sizing 4 §3.1
`max-width`	Sets the largest width of the box.	none, or the values of `width` except `auto`	none	CSS Sizing 3 §3.1
`max-height`	Sets the largest height of the box.	The values of `max-width`	none	CSS Sizing 3 §3.1
`max-inline-size`	Sets the largest size along the inline axis.	The values of `max-width`	none	CSS Logical Properties 1 §4.1
`max-block-size`	Sets the largest size along the block axis.	The values of `max-width`	none	CSS Logical Properties 1 §4.1
`max-size`	Sets `max-width` and `max-height`. One value applies to both.	One or two values of `max-width`	none	CSS Sizing 4 §3.1

Aspect ratio and intrinsic sizes

Name	Description	Values	Default	Specification
`aspect-ratio`	Gives the box a preferred ratio between its width and its height. The first number is the width and the second is the height. A single number is a ratio to 1.	`auto`, a non-negative number or two non-negative numbers separated by `/`, such as `16 / 9`. `auto` can come before or after the ratio.	`auto`	CSS Sizing 4 §4.1
`contain-intrinsic-width`	Sets the width that a box with size containment has instead of the size of its content.	An optional `auto`, then none or a non-negative length	none	CSS Sizing 4 §5.2
`contain-intrinsic-height`	Sets the height that a box with size containment has instead of the size of its content.	The values of `contain-intrinsic-width`	none	CSS Sizing 4 §5.2
`contain-intrinsic-inline-size`	Sets the inline size that a box with size containment has instead of the size of its content.	The values of `contain-intrinsic-width`	none	CSS Sizing 4 §5.2
`contain-intrinsic-block-size`	Sets the block size that a box with size containment has instead of the size of its content.	The values of `contain-intrinsic-width`	none	CSS Sizing 4 §5.2
`contain-intrinsic-size`	Sets `contain-intrinsic-width` and `contain-intrinsic-height`. One value applies to both.	One or two values of `contain-intrinsic-width`	none	CSS Sizing 4 §5.2
`frame-sizing`	Chooses the size of a replaced element that is used for its sizing.	`auto`, `content-width`, `content-height`, `content-block-size` or `content-inline-size`	`auto`	CSS Sizing 4 §5.3

Name	Description	Values	Default	Specification
`min-intrinsic-sizing`	Chooses when the minimum size that a box contributes to its container is compressed.	`legacy`, `zero-if-scroll`, `zero-if-extrinsic` or both of the last two	`legacy`	CSS Sizing 4 §5.5

Scaling

Name	Description	Values	Default	Specification
`zoom`	Scales all the lengths of an element, such as its font size, margins and width, by a factor. `2` and `200%` double them. The property is not inherited.	A number or a percentage of 0 or more	`1`	CSS Viewport 1 §4

Borders

Name	Description	Values	Default	Specification
`border`	Sets the width, style and color of the four borders. It also resets `border-image`.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-width`	Sets the width of the four borders. The values are read like those of `margin`.	One to four widths	`medium`	CSS Borders 4 §2.3
`border-style`	Sets the line style of the four borders. The values are read like those of `margin`.	One to four styles	`none`	CSS Borders 4 §2.2
`border-color`	Sets the color of the four borders. The values are read like those of `margin`.	One to four colors	`currentcolor`	CSS Borders 4 §2.1
`border-top`	Sets the width, style and color of the top border.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-right`	Sets the width, style and color of the right border.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-bottom`	Sets the width, style and color of the bottom border.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4

Name	Description	Values	Default	Specification
`border-left`	Sets the width, style and color of the left border.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-block`	Sets the width, style and color of the borders at the start and the end of the block axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-block-start`	Sets the width, style and color of the border at the start of the block axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-block-end`	Sets the width, style and color of the border at the end of the block axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-inline`	Sets the width, style and color of the borders at the start and the end of the inline axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-inline-start`	Sets the width, style and color of the border at the start of the inline axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-inline-end`	Sets the width, style and color of the border at the end of the inline axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-top-width`	Sets the width of the top border.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-right-width`	Sets the width of the right border.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-bottom-width`	Sets the width of the bottom border.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-left-width`	Sets the width of the left border.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3

Name	Description	Values	Default	Specification
`border-block-width`	Sets the width of the borders at the start and the end of the block axis.	One or two widths	`medium`	CSS Borders 4 §2.3
`border-block-start-width`	Sets the width of the border at the start of the block axis.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-block-end-width`	Sets the width of the border at the end of the block axis.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-inline-width`	Sets the width of the borders at the start and the end of the inline axis.	One or two widths	`medium`	CSS Borders 4 §2.3
`border-inline-start-width`	Sets the width of the border at the start of the inline axis.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-inline-end-width`	Sets the width of the border at the end of the inline axis.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-top-style`	Sets the line style of the top border.	`none`, `hidden`, `dotted`, `dashed`, `solid`, `double`, `groove`, `ridge`, `inset` or `outset`	none	CSS Borders 4 §2.2
`border-right-style`	Sets the line style of the right border.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-bottom-style`	Sets the line style of the bottom border.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-left-style`	Sets the line style of the left border.	The values of `border-top-style`	none	CSS Borders 4 §2.2

Name	Description	Values	Default	Specification
`border-block-style`	Sets the line style of the borders at the start and the end of the block axis.	One or two styles	none	CSS Borders 4 §2.2
`border-block-start-style`	Sets the line style of the border at the start of the block axis.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-block-end-style`	Sets the line style of the border at the end of the block axis.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-inline-style`	Sets the line style of the borders at the start and the end of the inline axis.	One or two styles	none	CSS Borders 4 §2.2
`border-inline-start-style`	Sets the line style of the border at the start of the inline axis.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-inline-end-style`	Sets the line style of the border at the end of the inline axis.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-top-color`	Sets the color of the top border.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-right-color`	Sets the color of the right border.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-bottom-color`	Sets the color of the bottom border.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-left-color`	Sets the color of the left border.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-block-color`	Sets the color of the borders at the start and the end of the block axis.	One or two colors	`currentcolor`	CSS Borders 4 §2.1

Name	Description	Values	Default	Specification
`border-block-start-color`	Sets the color of the border at the start of the block axis.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-block-end-color`	Sets the color of the border at the end of the block axis.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-inline-color`	Sets the color of the borders at the start and the end of the inline axis.	One or two colors	`currentcolor`	CSS Borders 4 §2.1
`border-inline-start-color`	Sets the color of the border at the start of the inline axis.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-inline-end-color`	Sets the color of the border at the end of the inline axis.	A color	`currentcolor`	CSS Borders 4 §2.1

Rounded corners

Name	Description	Values	Default	Specification
`border-radius`	Sets the radius of the four corners. Values after a slash set the vertical radii.	One to four non-negative lengths or percentages, optionally followed by `/` and one to four more	`0`	CSS Borders 4 §3.5
`border-top-left-radius`	Sets the radius of the top left corner. One value is a circle and two values are an ellipse.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-top-right-radius`	Sets the radius of the top right corner.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-bottom-right-radius`	Sets the radius of the bottom right corner.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5

Name	Description	Values	Default	Specification
`border-bottom-left-radius`	Sets the radius of the bottom left corner.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-start-start-radius`	Sets the radius of the corner at the start of the block axis and the start of the inline axis.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-start-end-radius`	Sets the radius of the corner at the start of the block axis and the end of the inline axis.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-end-start-radius`	Sets the radius of the corner at the end of the block axis and the start of the inline axis.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-end-end-radius`	Sets the radius of the corner at the end of the block axis and the end of the inline axis.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-top-radius`	Sets the radius of the two top corners.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-right-radius`	Sets the radius of the two right corners.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-bottom-radius`	Sets the radius of the two bottom corners.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-left-radius`	Sets the radius of the two left corners.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-block-start-radius`	Sets the radius of the two corners at the start of the block axis.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5

Name	Description	Values	Default	Specification
`border-block-end-radius`	Sets the radius of the two corners at the end of the block axis.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-inline-start-radius`	Sets the radius of the two corners at the start of the inline axis.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-inline-end-radius`	Sets the radius of the two corners at the end of the inline axis.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`corner`	Sets the radius and the shape of the four corners.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-top-left`	Sets the radius and the shape of the top left corner.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-top-right`	Sets the radius and the shape of the top right corner.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-bottom-right`	Sets the radius and the shape of the bottom right corner.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-bottom-left`	Sets the radius and the shape of the bottom left corner.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-start-start`	Sets the radius and the shape of the corner at the start of the block axis and the start of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-start-end`	Sets the radius and the shape of the corner at the start of the block axis and the end of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9

Name	Description	Values	Default	Specification
`corner-end-start`	Sets the radius and the shape of the corner at the end of the block axis and the start of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-end-end`	Sets the radius and the shape of the corner at the end of the block axis and the end of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-top`	Sets the radius and the shape of the two top corners.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-right`	Sets the radius and the shape of the two right corners.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-bottom`	Sets the radius and the shape of the two bottom corners.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-left`	Sets the radius and the shape of the two left corners.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-block-start`	Sets the radius and the shape of the two corners at the start of the block axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-block-end`	Sets the radius and the shape of the two corners at the end of the block axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-inline-start`	Sets the radius and the shape of the two corners at the start of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-inline-end`	Sets the radius and the shape of the two corners at the end of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9

Color, images and shorthands

Name	Description	Values	Default	Specification
`background`	Sets all the background properties in one declaration. Layers are separated by commas, and the color goes in the last layer. It resets the properties that you leave out.	One or more layers, each with an image, a position, a size after `/`, a repeat style, an attachment and one or two boxes; and a color in the last layer	See the individual properties	CSS Backgrounds 4 §2.12
`background-tbd`	Sets the background layers like `background`, but takes no color and does not reset it.	One or more layers	See the individual properties	CSS Backgrounds 4 §2.11
`background-color`	Sets the solid color painted below the images.	A color	`transparent`	CSS Backgrounds 4 §2.1
`background-image`	Sets the images of the layers. The first image is the top layer.	A comma-separated list of none or images: `url()`, gradients, `image()`, `image-set()` and `cross-fade()`	none	CSS Backgrounds 4 §2.2

Position, size and repetition

Name	Description	Values	Default	Specification
`background-position`	Sets where each image is placed in its positioning area.	One to four values: `left`, `center`, `right`, `top`, `bottom`, lengths and percentages. A side keyword can be followed by an offset.	`0% 0%`	CSS Backgrounds 4 §2.7
`background-position-x`	Sets the horizontal position of each image.	`center`, `left`, `right`, `x-start`, `x-end`, a length or percentage, or a side keyword with an offset	`0%`	CSS Backgrounds 4 §2.6

Name	Description	Values	Default	Specification
`background-position-y`	Sets the vertical position of each image.	`center`, `top`, `bottom`, `y-start`, `y-end`, a length or percentage, or a side keyword with an offset	`0%`	CSS Backgrounds 4 §2.6
`background-position-inline`	Sets the position of each image along the inline axis.	`center`, `start` or `end`, a length or percentage, or a keyword with an offset	`0%`	CSS Backgrounds 4 §2.6
`background-position-block`	Sets the position of each image along the block axis.	The values of `background-position-inline`	`0%`	CSS Backgrounds 4 §2.6
`background-size`	Sets the size of each image. `cover` makes the image cover the area, and `contain` makes the whole image fit in it.	`cover`, `contain`, or one or two values that are `auto` or non-negative lengths or percentages	`auto`	CSS Backgrounds 4 §2.10
`background-repeat`	Sets how each image repeats on the two axes. One value applies to both axes, and two values are the horizontal and the vertical ones.	`repeat-x`, `repeat-y`, `repeat-inline`, `repeat-block`, or one or two of `repeat`, `no-repeat`, `space` and `round`	`repeat`	CSS Backgrounds 4 §2.4
`background-repeat-x`	Sets how each image repeats on the horizontal axis.	`repeat`, `no-repeat`, `space` or `round`	`repeat`	CSS Backgrounds 4 §2.3
`background-repeat-y`	Sets how each image repeats on the vertical axis.	The values of `background-repeat-x`	`repeat`	CSS Backgrounds 4 §2.3
`background-repeat-inline`	Sets how each image repeats along the inline axis.	The values of `background-repeat-x`	`repeat`	CSS Backgrounds 4 §2.3
`background-repeat-block`	Sets how each image repeats along the block axis.	The values of `background-repeat-x`	`repeat`	CSS Backgrounds 4 §2.3

Name	Description	Values	Default	Specification
`background-origin`	Chooses the box that is the positioning area of the images.	`border-box`, `padding-box` or `content-box`	`padding-box`	CSS Backgrounds 4 §2.9
`background-clip`	Chooses the area that the background is painted in.	`border-box`, `padding-box`, `content-box`, `border-area`, `text`, or `border-area` and `text` together	`border-box`	CSS Backgrounds 4 §2.8
`background-attachment`	Chooses what the images are attached to. In a PDF, `fixed` attaches them to the page, so that they are drawn again on every page.	`scroll`, `local` or `fixed`	`scroll`	CSS Backgrounds 4 §2.5

Blending

Name	Description	Values	Default	Specification
`background-blend-mode`	Sets how each image layer blends with the layers below it and with the background color.	`normal`, `multiply`, `screen`, `overlay`, `darken`, `lighten`, `color-dodge`, `color-burn`, `hard-light`, `soft-light`, `difference`, `exclusion`, hue, `saturation`, `color` or `luminosity`; a comma-separated list	`normal`	Compositing 1 §3.4.3

Text and fonts

The font properties of your stylesheet decide how text looks in the PDF. This chapter explains how to choose fonts, load your own and use their special features. The text and fonts reference at the end lists every property, at-rule and descriptor.

A font is a file that holds the shapes of the characters, called glyphs. A font family is a group of related font faces, such as the regular, bold and italic faces of one design. You choose a family with `font-family`, and CSSMedia picks the face that best matches the weight, width and style that you request.

CSSMedia gets fonts from three places:

- The 14 standard fonts of PDF: Times, Helvetica, Courier, Symbol and ZapfDingbats. They are built in.
- The fonts in the folder that you pass with the `-f` option.
- The fonts that your stylesheet declares with `@font-face`.

CSSMedia does not read the fonts installed in your computer, so a PDF looks the same on every computer. It embeds the fonts that it loads in the PDF.

The topics of this chapter follow the order in which you work:

- Font selection and sizing
- Font resources and descriptors
- Font variants and features
- Variable and color fonts

Font selection and sizing

Choose the font family, the size and the face of your text, and control the height of its lines. Put the declarations in rules that select the XML elements that you want to style.

About this task

The element names in the examples, such as `chapter` and `term`, are placeholders. Use the names that occur in your XML.

```
chapter {
    font-family: "Noto Serif", serif;
    font-size: 12pt;
    font-weight: 400;
}
```

Procedure

1. Choose the font family

   Use `font-family` to list the families to try, in order of preference. Separate them with commas and put a name that contains spaces in quotes:

```
body {
    font-family: "Noto Serif", "Times New Roman", serif;
}
```

The last name, `serif`, is a generic family. It is a keyword, so it has no quotes. CSSMedia maps three generic families to the standard fonts of PDF: `serif` is Times, `sans-serif` is Helvetica and `monospace` is Courier. The generic family `math` is the first font of the fonts folder that has a MATH table, or Times if there is none. The other generic families have no built-in font, so CSSMedia skips them and tries the next family in the list.

CSSMedia finds a family in these places, in this order: the `@font-face` rules of your stylesheet, the standard fonts of PDF, and the fonts folder that you pass with `-f`. It never uses the fonts installed in your computer. The next topic explains how to load your fonts.

A family may not have every character of your text. For each character, CSSMedia tries the families of the list in order. If none has the character, it tries Helvetica, Symbol and ZapfDingbats, and then it draws the missing-character glyph of Helvetica. List a family for every script that your documents use.

The standard fonts of PDF

Every PDF reader has the 14 standard fonts, and CSSMedia includes their metrics. You can name each of the five families:

Family	Faces	Generic family
`Times`	Times-Roman, Times-Bold, Times-Italic, Times-BoldItalic	`serif`
`Helvetica`	Helvetica, Helvetica-Bold, Helvetica-Oblique, Helvetica-BoldOblique	`sans-serif`
`Courier`	Courier, Courier-Bold, Courier-Oblique, Courier-BoldOblique	`monospace`
`Symbol`	Symbol	
`ZapfDingbats`	ZapfDingbats	

Times has serifs and Helvetica does not. Courier is monospaced: all its glyphs have the same width. Symbol has symbols, and ZapfDingbats has ornaments. The default font of CSSMedia, which is the font of text with no `font-family`, is Helvetica. ISO 32000-1, section 9.6.2, defines these fonts.

2. Set the size

   Use `font-size` to set the size of the text. The value is a length such as `12pt`, a percentage, or a keyword:

```
body { font-size: 12pt; }
h1 { font-size: 1.5em; }
small { font-size: smaller; }
```

An `em` or a percentage is relative to the font size of the parent element. The keywords `larger` and `smaller` make the size larger or smaller than the size of the parent. The keywords from `xx-small` to `xxx-large` are absolute sizes, and the initial value is `medium`.

When a fallback font looks smaller than the preferred font at the same size, use `font-size-adjust`. It scales the text so that one metric of the font has the same relative size in every font. The metric is `ex-height`, which is the height of the lowercase letters and is the default, or `cap-height`, `ch-width`, `ic-width` or `ic-height`. The value is a number, or `from-font` to use the metric of the first font that is available:

```
body { font-size-adjust: 0.5; }
```

3. Choose the face

   Within a family, CSSMedia chooses the face that best matches three properties, in this order: `font-width`, `font-style` and `font-weight`.

   font-weight

   Use `font-weight` to choose how heavy the letters look. The values are `normal`, which is 400, `bold`, which is 700, or a number from 1 to 1000. The values `bolder` and `lighter` are relative to the weight of the parent element:

   ```
   strong { font-weight: 700; }
   ```

   font-width

   Use `font-width` to ask for narrower or wider letters. The value is a percentage, where 100% is normal, or a keyword from `ultra-condensed`, which is 50%, to `ultra-expanded`, which is 200%. The old name `font-stretch` is an alias:

   ```
   heading { font-width: condensed; }
   label { font-width: 110%; }
   ```

   font-style

   Use `font-style` to ask for upright or slanted letters. The values are `normal`, `italic`, and `oblique` with an optional angle from -90deg to 90deg. If you do not write an angle, it is 14deg. The values `left` and `right` ask for an italic or oblique face that slants clockwise or counter-clockwise:

   ```
   em { font-style: italic; }
   ```

   If the family has no face for the weight, width or style that you request, CSSMedia uses the closest face that exists. The next topic explains how a declared face states what it provides.

4. Control synthesized faces

   When a family has no bold or slanted face, CSSMedia can create one. It draws a thin outline around the glyphs to make them bold, and it shears the glyphs to make them slanted. The standard fonts of PDF have bold and slanted faces, so they never need synthesis. CSSMedia can also create small capitals and the superscript and subscript forms that Font variants and features explains.

   Use `font-synthesis` to forbid these faces. The value none forbids all of them. Otherwise list the kinds that you allow: `weight`, `style`, `small-caps` and `position`. Four properties

control one kind each. Their value is `auto` to allow or `none` to forbid, and `font-synthesis-style` also accepts `oblique-only`, which allows oblique faces but not italic faces:

```
body { font-synthesis: none; }
code { font-synthesis-weight: none; }
```

The four longhand properties are `font-synthesis-weight`, `font-synthesis-style`, `font-synthesis-small-caps` and `font-synthesis-position`.

5. Use the shorthand and set the line height

   Use `font` to set several properties in one declaration. Write the optional style, variant, weight and width first, then the size, an optional slash and line height, and the family. The size and the family are required:

   ```
   chapter { font: italic bold 12pt/1.4 "Noto Serif", serif; }
   ```

   The shorthand resets to their initial values all the font properties that you do not write, including the variant and feature properties. To keep them, use separate declarations. The shorthand also accepts the system font keywords `caption`, `icon`, `menu`, `message-box`, `small-caption` and `status-bar`. A PDF has no system fonts, so CSSMedia gives the keyword the `system-ui` family and the initial values of the other properties.

   Use `line-height` to set the height of the lines. A number multiplies the font size of the element, a length gives the height directly, and a percentage is relative to the font size. The value `normal` uses the metrics of the font. A number is the best choice, because it adapts when the font size changes:

   ```
   body { font-size: 12pt; line-height: 1.4; }
   caption { line-height: 16pt; }
   ```

Font resources and descriptors

Load your own fonts with the `@font-face` rule or with a fonts folder. Each rule names a family, lists the sources of one face and describes what the face provides, so that CSSMedia can choose it.

About this task

Use a fonts folder when you reuse the same fonts in many projects. Pass the folder with the `-f` option. CSSMedia reads the files with the extensions `.ttf`, `.otf`, `.ttc` and `.otc` in the folder and in its subfolders, and takes the family names from the fonts. Then you name a family in `font-family`, with no other declaration.

Use `@font-face` when you want to load a font from a file or a web address, or to give a font another family name:

```
@font-face {
    font-family: "Example Serif";
    src: url("example-serif.woff2") format("woff2");
    font-weight: 400 700;
    font-style: normal;
}

chapter {
    font-family: "Example Serif", serif;
}
```

The descriptors inside the rule describe a font face, not an element of your XML. Write one rule for each face of the family, such as the regular, bold and italic faces. Each rule needs a `font-family` and a `src` descriptor. CSSMedia ignores an invalid declaration, and when a descriptor occurs more than once in a rule, it uses the last valid value.

A family that you declare with `@font-face` hides an installed or standard family with the same name. When CSSMedia cannot load any source of a rule, it prints a warning and the family has no face from that rule.

Procedure

1. Name the family and the sources

   The `font-family` descriptor gives the name that the `font-family` property uses. It replaces the names that are stored in the font file. Write the name as a string, or as one or more identifiers.

   The `src` descriptor is a comma-separated list of sources. CSSMedia tries them in order and uses the first that it can load:

   ```
   @font-face {
       font-family: "Example Serif";
       src: local("Example Serif Regular"),
            url("example-serif.woff2") format("woff2"),
            url("example-serif.otf") format("opentype");
   }
   ```

   - `url()` is the address of a font file. A relative address is resolved against the address of the stylesheet. It can be a file or a web address.
   - `local()` is the full name or the PostScript name of a face in the fonts folder. It does not name a family.

   CSSMedia reads OpenType and TrueType fonts, font collections, and WOFF and WOFF 2.0 files. A `url()` value can end with a fragment that names the face to use from a collection, such as `url("fonts.ttc#Example-Bold")`. The fragment is the PostScript name of the face.

   Hints: format() and tech()

   A `url()` source can have two hints. They let CSSMedia skip a source without loading it. Write `format()` first and then `tech()`.

   `format()` names the format of the file. CSSMedia supports `collection`, `opentype`, `truetype`, `woff` and `woff2`. It skips the sources with another format, such as `embedded-opentype` and `svg`. The strings `"woff2"`, `"woff"`, `"truetype"`, `"opentype"` and

`"collection"` are accepted as well. The strings that end in `-variations`, such as `"woff2-variations"`, mean the format plus `tech(variations)`.

`tech()` lists the technologies that the font needs, separated by commas. CSSMedia skips the source when it does not support one of them. It supports these:

- `features-opentype` and `features-aat`: layout features of OpenType and of Apple Advanced Typography.
- `variations` and `avar2`: variable fonts and their axis remapping.
- `palettes`, `color-COLRv0`, `color-COLRv1`, `color-SVG`, `color-sbix` and `color-CBDT`: color fonts.

The technologies `features-graphite` and `incremental` are not supported, so CSSMedia skips the sources that need them.

7. Describe the face

   The descriptors `font-weight`, `font-width` and `font-style` tell CSSMedia which requests the face can satisfy. Their initial value is `auto`, which reads the information from the font. Write one value, or two values to state a range that a variable font covers:

   ```
   @font-face {
       font-family: "Example Sans";
       src: url("example-sans-variable.woff2");
       font-weight: 300 800;
       font-width: 75% 125%;
       font-style: oblique 0deg 12deg;
   }
   ```

   - `font-weight` takes `normal`, `bold` or a number from 1 to 1000.
   - `font-width` takes a percentage or a width keyword. The keywords are 50%, 62.5%, 75%, 87.5%, 100%, 112.5%, 125%, 150% and 200%, from `ultra-condensed` to `ultra-expanded`. The old name `font-stretch` is an alias.
   - `font-style` takes `normal`, `italic`, `left`, `right`, or `oblique` with no angle, one angle or two angles from -90deg to 90deg.

   The `font-size` descriptor limits the sizes for which the face is used. It takes one number, or two numbers for a range. The numbers are font sizes in CSS pixels, without a unit. Use it for fonts that have a face for each size, such as a text face and a display face.

   The `font-named-instance` descriptor selects a named instance of a variable font, such as `"Condensed Bold"`. The value is a string that CSSMedia compares with the names of the instances, without regard to case. If no instance has the name, the descriptor has no effect. The weight, width and style that you request are applied first, and the axes of the named instance replace them.

11. Limit the face to some characters

    The `unicode-range` descriptor lists the characters for which the face can be used. The value is a comma-separated list of code points such as `U+0041`, ranges such as `U+0400-04FF`, and ranges with wildcards such as `U+4??`, where each `?` stands for one hexadecimal digit. The initial value covers all characters:

```
@font-face {
    font-family: "Document";
    src: url("latin.woff2");
    unicode-range: U+0000-024F;
}

@font-face {
    font-family: "Document";
    src: url("cyrillic.woff2");
    unicode-range: U+0400-04FF;
}
```

Two rules with the same family and the same style make one family. CSSMedia uses the face of each character from the rules whose range includes it. If the ranges overlap, the rule that comes last wins. A range does not guarantee that the font has the glyph. If it does not, CSSMedia goes on with the next family of the list.

12. Set the defaults of the face

    Three descriptors set what the face does by default. A property of the same name in your rules replaces the descriptor:

    - `font-feature-settings` turns OpenType features on or off by their tags. Font variants and features explains the value.
    - `font-variation-settings` sets the axes of a variable font. Variable and color fonts explains the value. Use `font-weight`, `font-width` and `font-style` for the standard axes.
    - `font-language-override` sets the OpenType language system of the face, as Font variants and features explains.

16. Adjust the metrics of the face

    Fonts of different families look different at the same size, and their lines have different heights. Adjust a face so that it fits with the others:

    - `size-adjust` scales the glyphs and the metrics of the face by a percentage. The initial value is `100%`. The font size does not change.
    - `ascent-override`, `descent-override` and `line-gap-override` replace the ascent, the descent and the line gap of the font. Each takes `normal` or a percentage of the font size. The first value is for horizontal text, and an optional second value is for vertical text.
    - `superscript-position-override`, `superscript-size-override`, `subscript-position-override` and `subscript-size-override` replace the position and the size of the glyphs that `font-variant-position` creates. Each takes `normal`, `from-font` or a percentage. The initial value is `normal`.

```
@font-face {
    font-family: "Example Sans";
    src: url("example-sans.woff2");
    size-adjust: 105%;
    ascent-override: 90%;
    descent-override: 22%;
    line-gap-override: 0%;
}
```

Font variants and features

Many fonts have alternate letter shapes, number styles, ligatures and other features. Use the properties of this topic to ask for them. The font must provide a feature for CSSMedia to use it.

About this task

An OpenType font stores its features in tables, and each feature has a four-letter tag, such as `liga` for the common ligatures. The properties `font-variant-*` and `font-kerning` ask for groups of features by name. The property `font-feature-settings` sets one feature by its tag.

The element names in the examples, such as `term` and `formula`, are placeholders. Use the names that occur in your XML.

Procedure

1. Control kerning and ligatures

   Use `font-kerning` to turn kerning on or off. The values `auto` and `normal` use the kerning of the font, and none turns it off:

   ```
   code { font-kerning: none; }
   ```

   Use `font-variant-ligatures` to control the ligatures. The value `normal` uses the defaults of the font: common ligatures and contextual forms are on, and discretionary and historical ligatures are off. The value none turns off all of them. You can also combine one value from each pair:

   - `common-ligatures` or `no-common-ligatures`, for ligatures such as "fi" and "fl".
   - `discretionary-ligatures` or `no-discretionary-ligatures`, for optional ligatures that suit a particular style.
   - `historical-ligatures` or `no-historical-ligatures`.
   - `contextual` or `no-contextual`, for glyphs that change with the characters around them.

   ```
   body { font-variant-ligatures: common-ligatures discretionary-ligatures; }
   code { font-variant-ligatures: none; }
   ```

   The ligatures that a script needs, such as those of Arabic, are always used.

6. Choose capitals, positions and numbers

   Use `font-variant-caps` to ask for small capitals and other forms of capital letters:

   - `small-caps`: small capitals instead of lowercase letters.
   - `all-small-caps`: small capitals instead of lowercase and uppercase letters.
   - `petite-caps` and `all-petite-caps`: the same with petite capitals, which are smaller.
   - `unicase`: small capitals for the uppercase letters, and normal lowercase letters.
   - `titling-caps`: capitals designed for titles.
   - `normal`: the usual letters.

   ```
   term { font-variant-caps: small-caps; }
   ```

When the font has no small capitals, CSSMedia creates them by scaling the capitals, unless `font-synthesis-small-caps` is none.

Use `font-variant-position` to ask for subscript glyphs with `sub` or superscript glyphs with `super`. The value `normal` uses the ordinary glyphs. When the font has no such glyphs, CSSMedia scales and moves the ordinary glyphs, unless `font-synthesis-position` is none. The `@font-face` descriptors that Font resources and descriptors explains can change their size and position.

```
formula { font-variant-position: sub; }
```

Use `font-variant-numeric` to choose the shape and the spacing of numbers. Combine one value from each group:

- `lining-nums` or `oldstyle-nums`: figures as high as capitals, or figures with ascenders and descenders.
- `proportional-nums` or `tabular-nums`: figures with different widths, or with equal widths. Use equal widths in tables so that the digits line up.
- `diagonal-fractions` or `stacked-fractions`: fractions with a diagonal slash, or with the numerator above the denominator.
- `ordinal`: the forms of the suffixes of ordinal numbers, such as "st" in "1st".
- `slashed-zero`: a zero with a slash.

```
table { font-variant-numeric: lining-nums tabular-nums; }
```

18. Use alternate glyphs

Fonts can have several alternate shapes for a character, which the font identifies by numbers. Use `font-variant-alternates` to ask for them. The value `normal` asks for none. Otherwise combine these values:

- `historical-forms`: historical glyph forms.
- `stylistic(name)`: a stylistic alternate.
- `styleset(name, ...)`: one or more stylistic sets.
- `character-variant(name, ...)`: variants of individual characters.
- `swash(name)`: a swash form.
- `ornaments(name)`: an ornament.
- `annotation(name)`: an annotation form, such as a circled digit.

The names are yours. You define each name for a font family with `@font-feature-values`. The prelude of the rule lists one or more family names, which cannot be generic families. Inside the block, a rule for each kind of alternate maps names to numbers:

```
@font-feature-values "Example Serif" {
    @styleset {
        headings: 1;
    }
}

h1 { font-variant-alternates: styleset(headings); }
```

A name that is not defined for the family of the text has no effect. Names are case-sensitive. If you define the same name twice, the last definition wins. These are the rules of the block:

- `@stylistic` maps a name to one number for the feature `salt`.
- `@historical-forms` maps a name to one or more numbers for `hist`.
- `@styleset` maps a name to one or more numbers from 1 to 99, for the features `ss01` to `ss99`. A 0 or a number above 99 selects no feature.
- `@character-variant` maps a name to one or two numbers. The first, from 1 to 99, selects `cv01` to `cv99`, and the optional second sets its value. A 0 or a number above 99 selects no feature.
- `@swash` maps a name to one number for `swsh` or `cswh`.
- `@ornaments` maps a name to one number for `ornm`.
- `@annotation` maps a name to one number for `nalt`.

33. Choose forms for a language or a script

    Use `font-variant-east-asian` to choose the forms of Chinese and Japanese characters. The value `normal` asks for none. Otherwise combine a glyph form, a width and `ruby`:

    - `jis78`, `jis83`, `jis90` and `jis04`: the Japanese forms of that edition of the JIS standard.
    - `simplified` and `traditional`: the forms of simplified and traditional Chinese.
    - `full-width` and `proportional-width`: glyphs of equal width, or with proportional spacing.
    - `ruby`: glyphs designed for the small size of ruby text.

    ```
    ruby { font-variant-east-asian: ruby; }
    ```

    Use `font-variant-emoji` to choose between the text and the emoji forms of the characters that have both. The value `text` asks for the text form, `emoji` for the emoji form and `unicode` for the default form that Unicode gives to each character. The value `normal` lets CSSMedia choose. A variation selector in the text, U+FE0E for text and U+FE0F for emoji, overrides the property.

    ```
    label { font-variant-emoji: text; }
    ```

    A font can have different glyphs for different languages. CSSMedia chooses the language system of the font from the language of the text, which comes from `xml:lang` or from `-cssmedia-lang`. Use `font-language-override` only when the font lacks the language system for your text but has another that suits it. The value is the OpenType language system tag in a string, such as `"SRB"` for Serbian. It is not a language tag such as `sr`. The tag has four characters, and CSSMedia pads shorter tags with spaces. A tag that the font does not have is ignored. The value `normal` uses the language of the text.

    ```
    chapter { font-language-override: "SRB"; }
    ```

38. Use the shorthand and the feature tags

    Use `font-variant` to set several variant properties in one declaration. It takes `normal`, none, or values of the longhand properties. The value none sets `font-variant-ligatures` to none. The properties that you do not write return to their initial values. The

shorthand does not change `font-language-override`, `font-feature-settings` or `font-variation-settings`:

```
body { font-variant: small-caps; }
table { font-variant: lining-nums tabular-nums; }
```

Use `font-feature-settings` when no other property describes the feature that you need. The value is `normal`, or a comma-separated list of tags. A tag is a string of four characters, followed by an optional number, on or `off`. Without a value, a tag is on. The value `0` or `off` turns the feature off, and a positive number turns it on and selects its alternate:

```
body { font-feature-settings: "liga" 0, "ss01"; }
```

Tags are case-sensitive. A tag that the font does not have is ignored. If a tag occurs more than once, the last value wins. The settings of this property come after the settings of the other properties, so they win. Prefer a higher-level property, such as `font-variant-ligatures`, when it describes the feature.

Variable and color fonts

A variable font changes its design along axes, such as weight, width and optical size. A color font draws its glyphs with the colors of a palette. The font file must provide the axis or the palette that you request.

About this task

Each axis of a variable font has a four-letter tag. The registered tags are `wght` for the weight, `wdth` for the width, `slnt` and `ital` for the slant, and `opsz` for the optical size. A font can also have axes with other tags. The font can have named instances, which are ready-made combinations of axis values, such as "Condensed Bold".

For the standard axes, use the properties that you use with any font: `font-weight` controls `wght`, `font-width` controls `wdth`, and `font-style` controls `slnt` and `ital`. They also take part in the choice of the face. Declare the range that the font covers in an `@font-face` rule, as Font resources and descriptors explains.

Procedure

1. Set the optical size

   A font with an optical size axis changes the details of its glyphs with the size of the text, for example thinner strokes at large sizes. Use `font-optical-sizing` to turn it on or off. With `auto`, which is the initial value, CSSMedia sets the `opsz` axis from the font size, within the range of the axis. With none, it leaves the axis at the value that you set or at its default:

   ```
   body { font-optical-sizing: auto; }
   ```

2. Set the axes directly

   Use `font-variation-settings` to set the axes by their tags. The value is `normal`, or a comma-separated list of an axis tag in quotes followed by a number. The number can be a fraction or negative:

```
heading { font-variation-settings: "wght" 650, "wdth" 90; }
```

CSSMedia clamps a number to the range of the axis. It ignores an axis that the font does not have, and if a tag occurs more than once, the last value wins. This property does not take part in the choice of the face. The `font-variation-settings` descriptor of `@font-face` sets the same values as the defaults of a face. The values of the property come last, so they win.

3. Choose a color palette

   A color font has one or more palettes, which are numbered lists of colors, starting at 0. Use `font-palette` to choose one:

   - `normal`: the usual palette of the font, which is palette 0 in most fonts.
   - `light` and `dark`: the first palette that the font marks for light or for dark backgrounds. If the font has none, CSSMedia uses `normal`.
   - A name that starts with two hyphens, such as `--night`: a palette that you define with `@font-palette-values`. If no rule defines it, CSSMedia uses `normal`.
   - `palette-mix()`: a palette that mixes the colors of other palettes.

   ```
   emoji { font-palette: dark; }
   ```

   `palette-mix()` takes a color interpolation method and two or more palettes, each with an optional percentage. The method is the one that `color-mix()` uses, such as `in oklab`. The palettes are any value of `font-palette`, including another `palette-mix()`. Palettes without a percentage share the remainder equally. If the percentages add up to more than 100%, CSSMedia scales them to 100%, and if they add up to less, the colors become more transparent by the missing share. This example mixes a quarter of the `dark` palette and three quarters of `--night`:

   ```
   emoji { font-palette: palette-mix(in oklab, dark 25%, --night); }
   ```

8. Define your own palette

   Use the `@font-palette-values` rule to give a name to a palette. The name starts with two hyphens. Three descriptors define the palette:

   - `font-family` lists the families that the palette is for. It is required. Generic families are not allowed.
   - `base-palette` is the palette that you start from: `light`, `dark` or a palette number. If you omit it, or the font has no such palette, it is palette 0.
   - `override-colors` replaces colors of the base palette. The value is a comma-separated list of pairs of a palette number and a color. An index that the palette does not have is ignored, and if an index occurs more than once, the last color wins.

```
@font-palette-values --night {
    font-family: "Example Emoji";
    base-palette: dark;
    override-colors: 0 navy, 1 white;
}

emoji {
    font-family: "Example Emoji";
    font-palette: --night;
}
```

The colors of `override-colors` must be absolute: named colors, hexadecimal colors and numeric functions such as `rgb()` in any color space. A color that depends on the context, such as `currentColor`, makes the descriptor invalid. If two rules have the same name, the last one wins.

Text and fonts reference

This reference lists the properties, at-rules and descriptors that CSSMedia uses to select, load and shape fonts.

Font selection and sizing

Name	Description	Values	Default	Specification
`font`	Sets the font style, variant, weight, width, size, line height and family in one declaration.	A system font keyword (`caption`, `icon`, `menu`, `message-box`, `small-caption` or `status-bar`), or optional style, `normal` or `small-caps`, weight and width, then the size, an optional `/` and line height, and the family	See the individual properties	CSS Fonts 4 §2.7
`font-family`	Lists the font families to try, in order. A family name with spaces needs quotes.	Family names, and `serif`, `sans-serif`, `monospace`, `system-ui`, `cursive`, `fantasy`, `math`, `ui-serif`, `ui-sans-serif`, `ui-monospace`, `ui-rounded` or `generic()` keywords	`sans-serif`, which is Helvetica	CSS Fonts 4 §2.1

Name	Description	Values	Default	Specification
`font-size`	Sets the size of the text.	A keyword from `xx-small` to `xxx-large`, `larger`, `smaller`, `math`, or a non-negative length or percentage	`medium`	CSS Fonts 4 §2.5
`font-size-adjust`	Scales the text so that a chosen metric of the font, such as the height of lowercase letters, has the same relative size in every font.	none, or an optional metric (`ex-height`, `cap-height`, `ch-width`, `ic-width` or `ic-height`) followed by `from-font` or a non-negative number	`none`	CSS Fonts 5 §3.4
`font-weight`	Selects the weight of the face.	`normal`, `bold`, `bolder`, `lighter` or a number from 1 to 1000	`normal`	CSS Fonts 4 §2.2
`font-width`	Selects the width of the face.	`normal`, a non-negative percentage, or `ultra-condensed`, `extra-condensed`, `condensed`, `semi-condensed`, `semi-expanded`, `expanded`, `extra-expanded` or `ultra-expanded`	`normal`	CSS Fonts 4 §2.3
`font-stretch`	Alias of `font-width`.	The values of `font-width`	`normal`	CSS Fonts 4 §2.3.1
`font-style`	Selects an upright, italic or oblique face.	`normal`, `italic`, `left`, `right`, or `oblique` with an optional angle from -90deg to 90deg	`normal`	CSS Fonts 4 §2.4
`font-synthesis`	Allows or forbids the faces that CSSMedia creates when the family has none: bold, slanted, small capitals, subscripts and superscripts.	none, or any of `weight`, `style`, `small-caps` and `position`	`weight style small-caps position`	CSS Fonts 4 §2.8.5

Name	Description	Values	Default	Specification
`font-synthesis-weight`	Allows or forbids synthesized bold.	`auto` or `none`	`auto`	CSS Fonts 4 §2.8.1
`font-synthesis-style`	Allows or forbids synthesized italic and oblique faces.	`auto`, `none` or `oblique-only`	`auto`	CSS Fonts 4 §2.8.2
`font-synthesis-small-caps`	Allows or forbids synthesized small capitals.	`auto` or `none`	`auto`	CSS Fonts 4 §2.8.3
`font-synthesis-position`	Allows or forbids synthesized subscripts and superscripts.	`auto` or `none`	`auto`	CSS Fonts 4 §2.8.4
`line-height`	Sets the height of a line of text.	`normal`, a non-negative number, or a non-negative length or percentage	`normal`	CSS Inline Layout 3 §5.1

Text color, alignment and lines

Name	Description	Values	Default	Specification
`color`	Sets the color of the text. Borders and other properties that use `currentcolor` take it too, and the property is inherited.	A color: a keyword, `#rgb` or `#rrggbb`, `rgb()`, `hsl()` or another color function, `currentcolor` or `transparent`	`CanvasText`, black	CSS Color 4 §3.2
`text-align`	Aligns the lines of a block. `justify` stretches every line but the last, and `justify-all` stretches the last one too. `start` and `end` follow the direction of the text.	`start`, `end`, `left`, `right`, `center`, `justify`, `justify-all` or `match-parent`	`start`	CSS Text 4 §7.1

Name	Description	Values	Default	Specification
`text-transform`	Changes the case of the text when it is drawn, without changing your XML. `full-width` replaces the letters and digits by their full-width forms, which CJK text uses.	none, or `capitalize`, `uppercase` or `lowercase`, with `full-width`	none	CSS Text 4 §2.1
`text-decoration`	Draws lines on the text. It is a shorthand for `text-decoration-line`, `text-decoration-style`, `text-decoration-color` and `text-decoration-thickness`. The lines run across the text of the children too.	A line: none or one or more of `underline`, `overline` and `line-through`; a style: `solid`, `double`, `dotted`, `dashed` or `wavy`; a color; a thickness	none	CSS Text Decoration 4 §2.6
`white-space`	Chooses whether spaces and line feeds collapse and whether lines wrap. It is a shorthand for `white-space-collapse` and `text-wrap-mode`. `pre` keeps spaces and line feeds and does not wrap, `pre-wrap` keeps them and wraps, `pre-line` keeps line feeds only, and `nowrap` collapses spaces and does not wrap. `break-spaces` keeps spaces and lets a line break after each.	`normal`, `pre`, `nowrap`, `pre-wrap`, `pre-line` or `break-spaces`	`normal`	CSS Text 4 §3
`overflow-wrap`	Lets a word that is longer than its line break at any letter, so that it does not overflow the box.	`normal`, `break-word` or `anywhere`	`normal`	CSS Text 4 §6.4

Font variants and features

Name	Description	Values	Default	Specification
`font-variant`	Shorthand for the `font-variant-*` properties.	`normal`, none, or values of the longhand properties	`normal`	CSS Fonts 4 §6.11
`font-variant-alternates`	Selects alternate glyphs that the font names, such as stylistic sets and swashes.	`normal`, or any of `stylistic()`, `historical-forms`, `styleset()`, `character-variant()`, `swash()`, `ornaments()` and `annotation()`	`normal`	CSS Fonts 4 §6.8
`font-variant-caps`	Selects small capitals and other capital forms.	`normal`, `small-caps`, `all-small-caps`, `petite-caps`, `all-petite-caps`, `unicase` or `titling-caps`	`normal`	CSS Fonts 4 §6.6
`font-variant-east-asian`	Selects glyph forms and widths for East Asian text.	`normal`, or any of `jis78`, `jis83`, `jis90`, `jis04`, `simplified`, `traditional`, `full-width`, `proportional-width` and `ruby`	`normal`	CSS Fonts 4 §6.10
`font-variant-emoji`	Chooses text or emoji presentation for characters that have both.	`normal`, `text`, `emoji` or `unicode`	`normal`	CSS Fonts 4 §9.3
`font-variant-ligatures`	Turns common, discretionary, historical and contextual ligatures on or off.	`normal`, none, or any of `common-ligatures`, `no-common-ligatures`, `discretionary-ligatures`, `no-discretionary-ligatures`, `historical-ligatures`, `no-historical-ligatures`, `contextual` and `no-contextual`	`normal`	CSS Fonts 4 §6.4

Name	Description	Values	Default	Specification
`font-variant-numeric`	Selects the shape and spacing of numbers, fractions and zeros.	`normal`, or any of `lining-nums`, `oldstyle-nums`, `proportional-nums`, `tabular-nums`, `diagonal-fractions`, `stacked-fractions`, `ordinal` and `slashed-zero`	`normal`	CSS Fonts 4 §6.7
`font-variant-position`	Selects subscript or superscript glyphs.	`normal`, `sub` or `super`	`normal`	CSS Fonts 4 §6.5
`font-kerning`	Turns the kerning of the font on or off.	`auto`, `normal` or `none`	`auto`	CSS Fonts 4 §6.3
`font-feature-settings`	Turns OpenType features on or off by their four-letter tags.	`normal`, or a comma-separated list of tags, each followed by an optional integer, on or `off`	`normal`	CSS Fonts 4 §6.12
`font-language-override`	Sets the OpenType language system of the text, in place of its language.	`normal` or a string	`normal`	CSS Fonts 4 §6.13

Variable and color fonts

Name	Description	Values	Default	Specification
`font-optical-sizing`	Sets the optical size axis of a variable font from the font size.	`auto` or none	`auto`	CSS Fonts 4 §8.1
`font-variation-settings`	Sets the axes of a variable font by their four-letter tags.	`normal`, or a comma-separated list of axis tags in quotes, each followed by a number	`normal`	CSS Fonts 4 §8.2
`font-palette`	Chooses the color palette of a color font.	`normal`, `light`, `dark`, the name of an `@font-palette-values` rule, or `palette-mix()`	`normal`	CSS Fonts 4 §9.1

The @font-face rule

Name	Description	Values	Default	Specification
`@font-face`	Makes a font face available to the stylesheet under a family name.	A block of the descriptors below		CSS Fonts 4 §4.1
`font-family` descriptor	Names the family of the face. This name hides an installed family with the same name.	A family name	Required	CSS Fonts 4 §4.2
`src` descriptor	Lists the sources of the face. CSSMedia uses the first one that it can load.	A comma-separated list of `url()` and `local()` sources, each with optional `format()` and `tech()` hints	Required	CSS Fonts 4 §4.3
`font-weight` descriptor	Gives the weight, or the range of weights, of the face.	`auto`, or one or two values, each `normal`, `bold` or a number from 1 to 1000	`auto`	CSS Fonts 4 §4.4
`font-width` descriptor	Gives the width, or the range of widths, of the face.	`auto`, or one or two values, each a width keyword or a non-negative percentage	`auto`	CSS Fonts 4 §4.4
`font-stretch` descriptor	Alias of the `font-width` descriptor.	The values of the `font-width` descriptor	`auto`	CSS Fonts 4 §4.4.1
`font-style` descriptor	Gives the style, or the range of oblique angles, of the face.	`auto`, `normal`, `italic`, `left`, `right`, or `oblique` with one or two angles from -90deg to 90deg	`auto`	CSS Fonts 4 §4.4
`font-size` descriptor	Limits the font sizes for which the face is used.	`auto` or one or two numbers	`auto`	CSS Fonts 5 §4.3

Name	Description	Values	Default	Specification
`font-named-instance` descriptor	Selects a named instance of a variable font.	`auto` or a string	`auto`	CSS Fonts 4 §4.7
`unicode-range` descriptor	Limits the characters for which the face is used.	A comma-separated list of code points, ranges and ranges with ? wildcards	`U+0-10FFFF`	CSS Fonts 4 §4.5
`font-feature-settings` descriptor	Sets the OpenType features that the face uses by default.	The values of the `font-feature-settings` property	`normal`	CSS Fonts 4 §4.6
`font-variation-settings` descriptor	Sets the variable font axes that the face uses by default.	The values of the `font-variation-settings` property	`normal`	CSS Fonts 4 §4.6
`font-language-override` descriptor	Sets the OpenType language system that the face uses by default.	`normal` or a string	`normal`	CSS Fonts 4 §4.10
`size-adjust` descriptor	Scales the glyphs and the metrics of the face.	A non-negative percentage	`100%`	CSS Fonts 5 §4.4
`ascent-override` descriptor	Replaces the ascent of the face.	One or two values, each `normal` or a non-negative percentage	`normal`	CSS Fonts 5 §4.5
`descent-override` descriptor	Replaces the descent of the face.	One or two values, each `normal` or a non-negative percentage	`normal`	CSS Fonts 5 §4.5
`line-gap-override` descriptor	Replaces the line gap of the face.	One or two values, each `normal` or a non-negative percentage	`normal`	CSS Fonts 5 §4.5
`superscript-position-override` descriptor	Replaces the position of the superscript glyphs.	One or two values, each `normal`, `from-font` or a percentage	`normal`	CSS Fonts 5 §4.6

Name	Description	Values	Default	Specification
`superscript-size-override` descriptor	Replaces the size of the superscript glyphs.	One or two values, each `normal`, `from-font` or a non-negative percentage	`normal`	CSS Fonts 5 §4.6
`subscript-position-override` descriptor	Replaces the position of the subscript glyphs.	One or two values, each `normal`, `from-font` or a percentage	`normal`	CSS Fonts 5 §4.6
`subscript-size-override` descriptor	Replaces the size of the subscript glyphs.	One or two values, each `normal`, `from-font` or a non-negative percentage	`normal`	CSS Fonts 5 §4.6

The @font-feature-values rule

Name	Description	Values	Default	Specification
`@font-feature-values`	Gives names to the alternate glyphs of one or more font families, for use in `font-variant-alternates`.	One or more family names, and a block with the rules below		CSS Fonts 4 §6.9
`@stylistic`	Names stylistic alternates (`salt`).	Names, each with one non-negative integer		CSS Fonts 4 §6.9.1
`@styleset`	Names stylistic sets (`ss01` to `ss99`).	Names, each with one or more integers from 1 to 99. 0 and larger values select no feature.		CSS Fonts 4 §6.9.2
`@character-variant`	Names character variants (`cv01` to `cv99`).	Names, each with one or two integers. The first, from 1 to 99, selects the feature, and the second sets its value. 0 and larger values select no feature.		CSS Fonts 4 §6.9.2
`@swash`	Names swashes (`swsh`, `cswh`).	Names, each with one non-negative integer		CSS Fonts 4 §6.9.1

Name	Description	Values	Default	Specification
`@ornaments`	Names ornaments (`ornm`).	Names, each with one non-negative integer		CSS Fonts 4 §6.9.1
`@annotation`	Names annotation forms (`nalt`).	Names, each with one non-negative integer		CSS Fonts 4 §6.9.1
`@historical-forms`	Names historical forms (`hist`).	Names, each with one or more non-negative integers		CSS Fonts 4 §6.9.1

The @font-palette-values rule

Name	Description	Values	Default	Specification
`@font-palette-values`	Defines a named color palette for a color font. Use its name in `font-palette`.	A name that starts with `--`, and a block of the descriptors below		CSS Fonts 4 §9.2
`font-family` descriptor	Names the families that the palette is for.	One or more family names	Required	CSS Fonts 4 §9.2.1
`base-palette` descriptor	Chooses the palette of the font that the new palette starts from.	`light`, `dark` or a non-negative integer	The first palette of the font	CSS Fonts 4 §9.2.2
`override-colors` descriptor	Replaces colors of the base palette.	A comma-separated list of pairs of a palette index and a color	No replacements	CSS Fonts 4 §9.2.3

Languages and hyphenation

CSSMedia formats text differently for each language. The languages and hyphenation reference at the end of this chapter lists the property, the selector and the properties that you use.

A document can have text in many languages, and each language has its own rules. The language of the text decides where CSSMedia can hyphenate a word, which quotation marks it writes, and which elements the `:lang()` selector matches.

CSSMedia reads the language from the `xml:lang` attribute when your XML uses it. When your vocabulary names the language in another way, you tell CSSMedia where to find it with the `-cssmedia-lang` property.

Hyphenation breaks a long word at the end of a line and writes a hyphen, so that lines are fuller and the spaces between words are more even. CSSMedia includes hyphenation patterns for many languages. The topics of this chapter follow the order in which you work:

- Document language
- Hyphenation
- Quotation marks

Document language

The language of the text decides how CSSMedia hyphenates words, which quotation marks it writes and which elements the `:lang()` selector matches. CSSMedia reads the language from the `xml:lang` attribute when your XML vocabulary uses it. If your vocabulary names the language in another attribute, use the CSSMedia `-cssmedia-lang` property to say which one.

About this task

A language tag is a short code that names a language, such as en for English, `fr` for French or `zh-Hans` for Chinese in simplified characters. The `xml:lang` attribute is defined by the XML specification. It holds a language tag and applies to the element that carries it and to everything inside that element.

XML can use any element and attribute names, so CSSMedia cannot know which other attribute holds a language. A vocabulary such as DocBook 4 uses a plain `lang` attribute. Names such as `para` and `lang` below are examples. Replace them with names used in your XML.

Procedure

1. Mark the language with `xml:lang`

   When your XML already has `xml:lang` attributes, you do not write anything in the stylesheet. An element has the language of the nearest `xml:lang` on itself or on an ancestor.

   ```
   <article xml:lang="en">
       <para>This paragraph is in English.</para>
       <para xml:lang="fr">Ce paragraphe est en français.</para>
   </article>
   ```

2. Take the language from another attribute

The `-cssmedia-lang` property is a CSSMedia extension. It is not a standard CSS property, so its name has the vendor prefix `-cssmedia-`. It sets the language of the elements it is applied to.

Value	Meaning
none	The property does not set the language of the element. This is the initial value.
`<string>`	A language tag written between quotation marks, such as `"fr"`. An empty string is the same as none.

The property is not inherited, but the language it sets reaches the content of the element, as `xml:lang` does. Use `attr()` to read the language from an attribute:

```
<book lang="en">
    <para>This paragraph is in English.</para>
    <para lang="de">Dieser Absatz ist auf Deutsch.</para>
</book>
```

```
* {
    -cssmedia-lang: attr(lang);
}
```

`*` selects every element. `attr(lang)` reads the value of the `lang` attribute of each element. The first `para` has no `lang` attribute, so `attr()` supplies an empty string and the paragraph keeps the language of `book`. The second `para` is in German.

You can also write the language in the stylesheet:

```
quote[role="greek"] {
    -cssmedia-lang: "el";
}
```

3. Know which language wins

   For each element, CSSMedia takes the first of these that exists:

   4. The language set by `-cssmedia-lang` on the element.
   5. The `xml:lang` attribute of the element itself.
   6. The language of its parent element.

   An element with both a `lang` attribute that your rule reads and an `xml:lang` attribute takes the language from `lang`.

   A rule that sets `-cssmedia-lang` cannot depend on the language. A selector such as `:lang(fr)` or `:dir(rtl)` never matches while CSSMedia decides the language, so those selectors have no effect on the `-cssmedia-lang` declarations. They work as usual on all other properties, and they see the language that `-cssmedia-lang` gave.

Hyphenation

Hyphenation breaks a word at the end of a line and writes a hyphen after the first part. Turn it on with the hyphens property, and set limits so that the breaks look good.

About this task

The element names in the examples, such as `para`, are placeholders. Use the names that occur in your XML.

Procedure

1. Turn hyphenation on

   The hyphens property has three values:

   - `manual` is the initial value. CSSMedia breaks a word only at a soft hyphen, which is the invisible character U+00AD that you put in the text where a word can break.
   - `auto` also breaks words with the hyphenation patterns of the language of the text.
   - none never breaks a word, not even at a soft hyphen.

   It is inherited, so set it once on the root element or on the element that holds the text:

   ```
   article {
       hyphens: auto;
   }
   ```

5. Check that the language has patterns

   CSSMedia hyphenates only text with a known language. Mark the language as Document language explains. A soft hyphen in a word makes the automatic breaks of that word a second choice, used only when nothing else fits.

   CSSMedia includes hyphenation patterns for these languages:Afrikaans, Assamese, Belarusian, Bulgarian, Bengali, Catalan, Church Slavonic, Welsh, Danish, German, Greek, English, Esperanto, Spanish, Estonian, Basque, Finnish, French, Friulian, Irish, Galician, Gujarati, Hindi, Croatian, Upper Sorbian, Hungarian, Interlingua, Icelandic, Italian, Georgian, Kazakh, Kannada, Latin, Lithuanian, Malayalam, Mongolian, Marathi, Norwegian Bokmål and Nynorsk, Dutch, Occitan, Oriya, Punjabi, Pali, Polish, Portuguese, Romansh, Romanian, Russian, Sanskrit, Slovak, Slovenian, Albanian, Serbian, Bosnian, Swedish, Tamil, Telugu, Thai, Turkmen, Turkish and Ukrainian. English has American and British patterns, for the tags `en-US` and `en-GB`.

   CSSMedia uses the patterns of the nearest tag, without regard to case. It looks for the whole tag first, such as `en-GB` or `de-CH`. If there are no patterns for it, it looks for the tag without its region, so `pt-AO` uses the patterns of `pt` when it has none of its own.

   CSSMedia does not drop a script or a variant, because they name another writing or another spelling. Serbian in Latin script, `sr-Latn`, has its own patterns, and so does `sr-Latn-ME`. A tag such as `mn-Mong` has none, because the patterns of Mongolian are for the Cyrillic script. BCP 47 allows a script subtag that the language does not need, such as `en-Latn-US`, but says that it should not be used, because English is almost always written in Latin script and the tag adds nothing to `en-US`. CSSMedia ignores such tags. A text with an ignored tag, or with a tag without patterns, is not hyphenated. To give it other patterns, set another tag with `-cssmedia-lang`:

```
article {
    -cssmedia-lang: "en-US";
}
```

6. Choose the hyphen

   Use `hyphenate-character` to choose the string that CSSMedia writes at the end of a broken line. The value `auto` is the hyphen U+2010 when the font of the text has it, and the hyphen-minus U+002D otherwise. You can also write your own string:

```
article {
    hyphenate-character: "-";
}
```

7. Limit hyphenation

   Four properties keep the hyphens from being too many or too short:

   - `hyphenate-limit-chars` sets the shortest word that CSSMedia hyphenates, the letters before the hyphen and the letters after it. The first value is the length of the word, and its default is 5. The two other values are the letters before and after the hyphen. When you write them as `auto` or omit them, CSSMedia uses the values of the patterns of the language. With one value you set only the length of the word, and with two values the third one is the same as the second.
   - `hyphenate-limit-lines` is the largest number of consecutive lines of an element that end with a hyphen. The initial value is `no-limit`.
   - `hyphenate-limit-last` keeps hyphens away from the last lines. With `always`, the last full line of an element and the last line before a page break do not end with a hyphen. With `page` or `column`, only the last line before a page break is kept free of a hyphen, and with `spread` only the last line before a spread break, which is the break after a right-hand page. A spread is two facing pages. The initial value is `none`.
   - `hyphenate-limit-zone` is the empty space that a line can have before CSSMedia hyphenates a word to fill it. It is a length, or a percentage of the width of the line. The initial value is 0.

```
article {
    hyphens: auto;
    hyphenate-limit-chars: 7 3 3;
    hyphenate-limit-lines: 2;
}
```

   This rule hyphenates words of at least 7 letters, leaves at least 3 letters on each side of the hyphen, and never has more than 2 hyphenated lines in a row.

Quotation marks

Each language has its own quotation marks. CSSMedia writes the marks of the language of the text when you generate quotes with `open-quote` and `close-quote`.

About this task

The element name q in the examples is a placeholder. Use the name of the elements that hold quotations in your XML.

Procedure

1. Generate the quotes

   Your XML does not have the quotation marks as text. Generate them with the `content` property of the pseudo-elements `::before` and `::after`:

   ```
   q::before {
       content: open-quote;
   }

   q::after {
       content: close-quote;
   }
   ```

   A q inside another q gets the marks of the next level. For English, the outer quotes are “ ” and the inner quotes are ‘ ’. German text gets „ “ and , ‘, and Japanese text gets 「 」.

2. Choose the marks yourself

   The `quotes` property has the initial value `auto`, which uses the marks of the language. To use other marks, write pairs of strings. The first pair is for the outer level, the second pair for the next level, and so on. The last pair is used for the deeper levels. The value none writes no marks, and `match-parent` uses the marks of the parent element:

   ```
   q {
       quotes: "«" "»" "‹" "›";
   }
   ```

Languages and hyphenation reference

This reference lists the property, the selector and the properties that CSSMedia uses for the language of the text, hyphenation and quotation marks.

Properties

Name	Description	Values	Default	Specification
`-cssmedia-lang`	Sets the language of the elements from a value of your XML, such as an attribute. CSSMedia extension.	none or a string with a language tag. `attr()` reads the tag from an attribute.	none	CSSMedia extension

Name	Description	Values	Default	Specification
`hyphens`	Chooses how words are broken at the end of a line. `manual` breaks only at the soft hyphens that the text has, `auto` also uses the hyphenation patterns of the language, and none never breaks inside a word.	none, `manual` or `auto`	`manual`	CSS Text 4 §6.3.1
`hyphenate-character`	Sets the string that CSSMedia writes at the end of a line that it breaks inside a word.	`auto` or a string. `auto` is U+2010 when the font has it, and U+002D otherwise.	`auto`	CSS Text 4 §6.3.2
`hyphenate-limit-chars`	Sets the shortest word that CSSMedia hyphenates, and the least number of letters before and after the hyphen.	One to three values, each `auto` or an integer. The values are the word length, the letters before and the letters after. A missing third value is the second, and a missing second value is `auto`.	`auto`	CSS Text 4 §6.3.4
`hyphenate-limit-lines`	Sets the largest number of consecutive lines of an element that end with a hyphen.	`no-limit` or an integer	`no-limit`	CSS Text 4 §6.3.5
`hyphenate-limit-last`	Stops CSSMedia from ending a line with a hyphen at the end of an element or before a page break. `always` applies to the last full line of the element and to the last line before a page break. `column` and page apply to the last line before a page break, and `spread` to the last line before a spread break.	none, `always`, `column`, page or `spread`	none	CSS Text 4 §6.3.5

Name	Description	Values	Default	Specification
`hyphenate-limit-zone`	Sets how much empty space a line can have before CSSMedia hyphenates a word to fill it. A percentage is relative to the width of the line.	A length or a percentage	`0`	CSS Text 4 §6.3.3
`quotes`	Selects the quotation marks used by generated `open-quote` and `close-quote` content. With `auto`, the marks depend on the language of the text.	`auto`, `none`, `match-parent`, or one or more pairs of quoted strings	`auto`	CSS Content 3 §2.4.1

Selector

Name	Description	Values	In a PDF	Specification
`:lang()`	Elements in a language. The language comes from `xml:lang` or the `-cssmedia-lang` property.	One or more language ranges, as identifiers or strings.	Matches	Selectors 4 §7.2

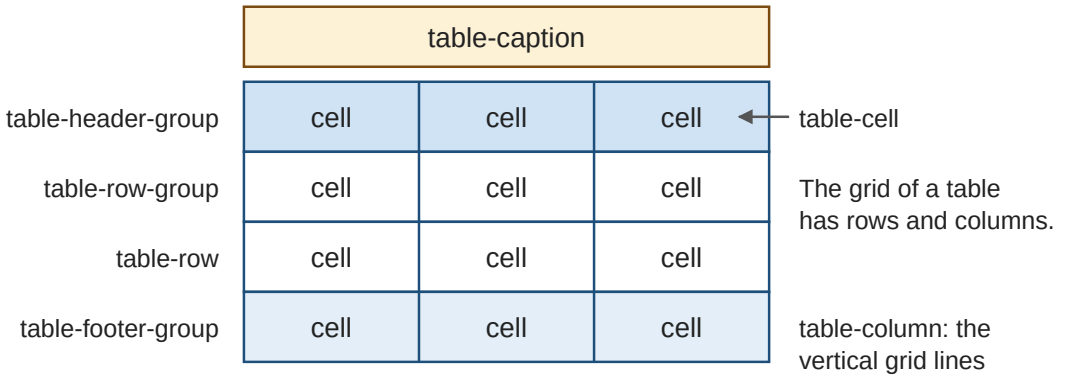
Ruby and vertical text

Tables

XML has no table element, so your stylesheet decides which elements are a table, its rows and its cells. This chapter explains how to build a table from your elements, how to size it, how to draw its borders and how it breaks across pages. The tables reference at the end lists the properties and display values.

A table in the PDF is a grid of cells. Every element that you want in the grid gets a `display` value that says what part of a table it is. The element names do not matter: a `table` element, a `list` of `item` elements or any other structure of your XML can be a table.

The parts of a table



A box with display: table holds the caption, the row groups and the rows.
The header group is always first and the footer group always last.

These are the display values that make a table:

Value	The element is
`table`	The table. It is a block, like a paragraph.
`inline-table`	A table that sits on a line of text, like a word.
`table-caption`	The title of the table.
`table-header-group`, `table-footer-group`	The rows that go first and last, and that can repeat on every page.
`table-row-group`	A group of rows, such as the body of the table.
`table-row`	A row of cells.
`table-cell`	A cell. It holds the content of the table.
`table-column-group`, `table-column`	A description of columns, to give them a width, a border or a background. They hold no content.

You do not need to give a value to every element. CSSMedia adds what is missing. A cell outside a row gets a row around it, text that is directly in a table or in a row gets a cell around it, and

rows that are outside a row group are fine. White space between the parts of a table is ignored, so you can indent your XML.

Each cell takes the next free column of its row, and a row that has fewer cells than the table has columns is completed with empty cells. CSS gives a cell no span: a stylesheet cannot make one cell cover two columns or two rows.

The topics of this chapter follow the steps of the work:

- Turn elements into a table
- Size a table and its columns
- Draw the borders and backgrounds of a table
- Break tables across pages

Turn elements into a table

You give each element of your table a display value, and then you add a caption, header rows, footer rows and descriptions of the columns. You also choose how the content of the cells is aligned.

Procedure

1. Give the elements a display value

   Write the table in your XML with elements of your choice. These are placeholders: use the names of your own document.

   ```
   <table>
       <caption>Prices</caption>
       <header>
           <row><cell>Item</cell><cell>Price</cell></row>
       </header>
       <body>
           <row><cell>Paper</cell><cell>4.00</cell></row>
           <row><cell>Ink</cell><cell>12.50</cell></row>
       </body>
   </table>
   ```

   Then write the rules that say what each element is. Without them, the elements are inline and their text runs together:

   ```
   table   { display: table; }
   caption { display: table-caption; }
   header  { display: table-header-group; }
   body    { display: table-row-group; }
   row     { display: table-row; }
   cell    { display: table-cell; padding: 2pt 6pt; }
   ```

2. Put the header and the footer rows in their groups

   The rows of a `table-header-group` are always the first rows of the table, and the rows of a `table-footer-group` are always the last, wherever the groups are in your XML. If your XML has the footer before the body, the PDF still shows it at the end. Only the first header group and the first footer group have this role. A second one is an ordinary row group.

   To repeat the header or the footer on every page, see Break tables across pages.

3. Add a caption

An element with `display: table-caption` is the caption. It is above the table, wherever you write it in the table. Use `caption-side: bottom` to put it below. The caption is as wide as the table and its text wraps to that width, so give the table a width, as Size a table and its columns explains, if a long caption must not wrap in a narrow table. The text of the caption uses the properties of the text, such as `text-align`:

```
caption {
    display: table-caption;
    caption-side: bottom;
    text-align: center;
    font-style: italic;
}
```

4. Describe the columns

   Elements with `display: table-column` stand for the columns, from left to right, in the order of your XML. They do not have content, and what is inside them is not drawn. Use them to give a column a width, a background or a border, so that you do not need to repeat it in every cell:

```
<table>
    <col name="item"/>
    <col name="price"/>
    <body>
        <row><cell>Paper</cell><cell>4.00</cell></row>
    </body>
</table>
```

```
col { display: table-column; }
col[name="item"]  { width: 120pt; }
col[name="price"] { background-color: #f5f5f5; }
```

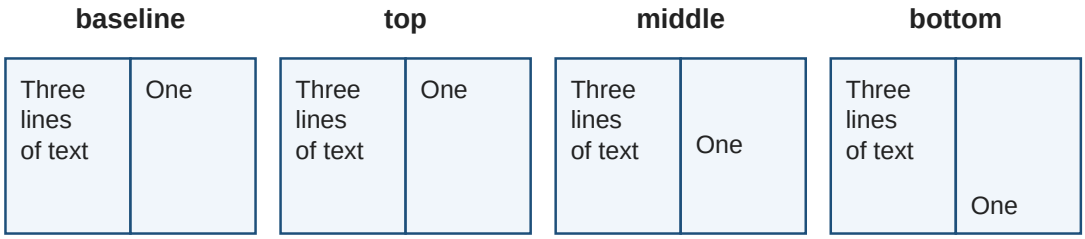
   An element with `display: table-column-group` that holds `table-column` elements groups them, for example to draw a border around several columns. A column group without columns inside stands for one column.

5. Align the content of the cells

   The properties of boxes work on cells: `padding`, `border`, `background-color` and `text-align`. When the cells of a row have different heights, `vertical-align` decides where the content of a cell goes in the height of the row:

   - `baseline` is the initial value. The first lines of the cells that use it are on one line.
   - `top` puts the content at the top of the cell.
   - `middle` centers the content in the height of the row.
   - `bottom` puts the content at the bottom of the cell.

The values of vertical-align



In each table the first cell has three lines, and the title is the vertical-align of the second cell. With baseline and top, the first lines of both cells are level.

```
cell {
    display: table-cell;
    vertical-align: top;
}
```

10. Imitate a cell that covers two columns

    CSS gives a cell no span, but when your XML marks a cell that covers two columns and leaves out the cell of the second column, you can keep the grid in line. In this XML the first cell has an attribute, `span`, and the row has no cell for the second column:

    ```
    <row>
        <cell span="2">Spans two columns</cell>
        <cell>Third</cell>
    </row>
    ```

    The cell that follows the marked one becomes two cells: an empty cell for the second column, and a cell with its own content for the third. The rule `display: contents` removes the box of the element, and the pseudo-elements `::before` and `::after` become the two cells. The value `contents` of `content` puts the content of the element in the second one. The style `hidden` on the border between the marked cell and the empty one makes them look like one cell, in a table with `border-collapse: collapse`:

```
cell[span] {
    border-right-style: hidden;
}

cell[span] + cell {
    display: contents;
    content: "";
}

cell[span] + cell::before,
cell[span] + cell::after {
    display: table-cell;
    border: 1pt solid;
    padding: 2pt 6pt;
}

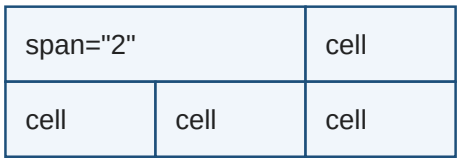
cell[span] + cell::before {
    content: "";
}

cell[span] + cell::after {
    content: contents;
}
```

The text of the marked cell stays at the start of its cell, over the width of both columns. Give the pseudo-elements the same padding, borders and background as the cells of your table.

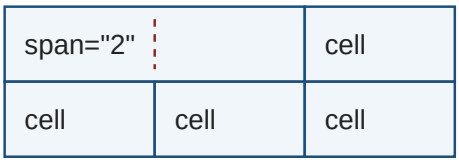
A cell that covers two columns

What the XML means



One cell covers two columns.

What the stylesheet makes



The grid has an empty cell there. The border between them is hidden, drawn here as a dashed line.

Size a table and its columns

By default a table is as wide as its content needs, up to the width of the page. You can give the table a width, give widths to its columns, and choose between two layouts.

Procedure

1. Set the width of the table

   A table with `width: auto`, the initial value, takes the width that its content wants. If the content is wider than the space it has, the table is as wide as the space and the text in the cells wraps. A table with a width has that width, and it can be a length or a percentage.

Margins, borders and padding work as for any box, and `margin-left: auto` with `margin-right: auto` centers the table:

```
table {
    display: table;
    width: 100%;
}
```

2. Let CSSMedia share the width between the columns

   With the initial layout, `table-layout: auto`, CSSMedia measures the content of all the cells of a column. A column is never narrower than its longest word or other unbreakable content, and it is as wide as its content when there is room. When the table is wider than the content needs, the extra width goes to the columns without a width of their own, in proportion to the width their content asks for. This layout looks at every row, so it gives the best result for most tables, but it must measure all the cells.

3. Give widths to columns

   A `width` on a cell makes its column that wide, or wider if another cell of the column needs more. You can also set the `width` or the `min-width` on the `table-column` element of the column, as Turn elements into a table shows. A column with a width does not take part in sharing the extra width of the table:

   ```
   cell:first-child {
       width: 80pt;
   }
   ```

4. Use a fixed layout when you know the widths

   With `table-layout: fixed` and a table width, CSSMedia does not look at the content to find the widths. A column takes the width of its `table-column` element, or else of its cell in the first row. The columns without a width share the rest of the width of the table equally. The result does not change with the content, and a long word in a cell that is too narrow does not make the column wider. If the table has no width, or its width is `auto`, the layout is automatic:

   ```
   table {
       display: table;
       table-layout: fixed;
       width: 100%;
   }

   col[name="item"] {
       width: 120pt;
   }
   ```

   Percentages work for the widths of cells and columns in a fixed layout.

The two layouts

table-layout: auto

No.	Description	Qty
1	Short	3

Each column is as wide as its content needs, in all the rows.

table-layout: fixed

No.	Description	Qty
1	Short	3

The columns share the width equally, unless the first row or a column gives one.

5. Set the heights

   The height of a row is the height of its tallest cell. A `height` on a row or on a cell is a minimum: the row is taller if its content needs it. If you give the table a `height` that is more than its rows need, the extra height goes to the rows.

Draw the borders and backgrounds of a table

You choose between two models for the borders of the cells: separate borders with a gap between the cells, or one shared border between neighbors. You can also paint backgrounds on every part of the table.

Procedure

1. Use separate borders for cells that stand apart

   The initial value of `border-collapse` is `separate`. Each cell has its own border, and the cells touch each other unless you set `border-spacing`, the gap between the borders of neighbors. One length is the gap in both directions, and two lengths are the horizontal gap and the vertical gap. The initial gap is 0. In this model the table can also have padding, and rows, row groups and columns have no border of their own.

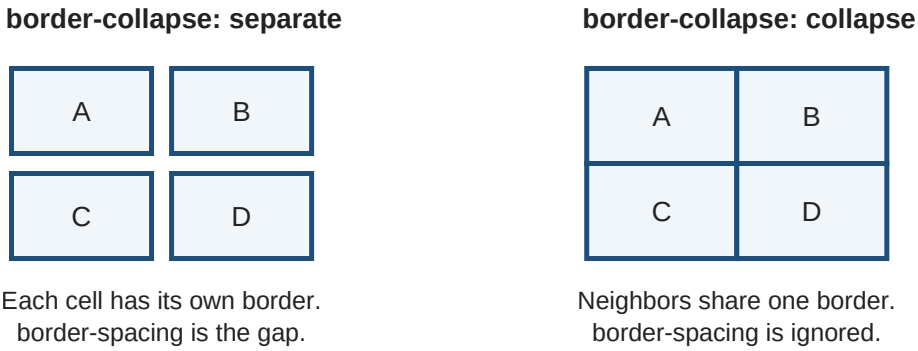
```
table {
    display: table;
    border-spacing: 4pt 2pt;
}

cell {
    display: table-cell;
    border: 1pt solid gray;
}
```

2. Use collapsed borders for a grid

   With `border-collapse: collapse` the neighbors share one border, which is centered on the line between them. `border-spacing` has no effect, and the table has no padding. The borders of the table, the row groups, the rows, the columns and the cells all take part, because they can meet on the same line.

The two border models



When two borders meet at a line, CSSMedia draws only one of them. It is chosen in this order:

3. A border with the style `hidden` always wins, and nothing is drawn on that line.
4. Otherwise the wider border wins, and a border with the style none counts as having no width.
5. When the widths are equal, the style decides: `double`, then `solid`, `dashed`, `dotted`, `ridge`, `outset`, `groove` and `inset`.
6. When the styles are equal too, the border of the cell wins over the border of the row, then the row group, the column, the column group and, last, the table.

```
table {
    display: table;
    border-collapse: collapse;
    border: 2pt solid black;
}

cell {
    display: table-cell;
    border: 0.5pt solid gray;
}
```

The outer edge of the table takes the width of the widest border along it, and half of it is inside the table.

7. Paint backgrounds

Every part of a table can have a background: the table, the column groups, the columns, the row groups, the rows and the cells. They are painted in layers in that order, each one over the one before, so the background of a cell hides the background of its row. Use a background on the header group to set it apart:

```
header {
    display: table-header-group;
    background-color: #e0e0e0;
    font-weight: bold;
}

row:nth-child(even) {
    background-color: #f5f5f5;
}
```

Break tables across pages

A long table continues on the next pages. You can repeat its header and its footer on every page, keep rows whole, and force a page break between rows.

Procedure

1. Know where a table breaks

   A table breaks between its rows. When a row does not fit in the space that is left on the page, CSSMedia decides by its height:

   - A row that is less than half as high as the page goes whole to the next page.
   - A taller row breaks inside. Each cell of the row continues on the next page where it stopped, and the cells that have ended stay empty.

   To keep a row whole in any case, give the rows `break-inside: avoid`. A row that is taller than a page cannot be kept whole, and it breaks anyway.

   ```
   row {
       display: table-row;
       break-inside: avoid;
   }
   ```

4. Repeat the header and the footer

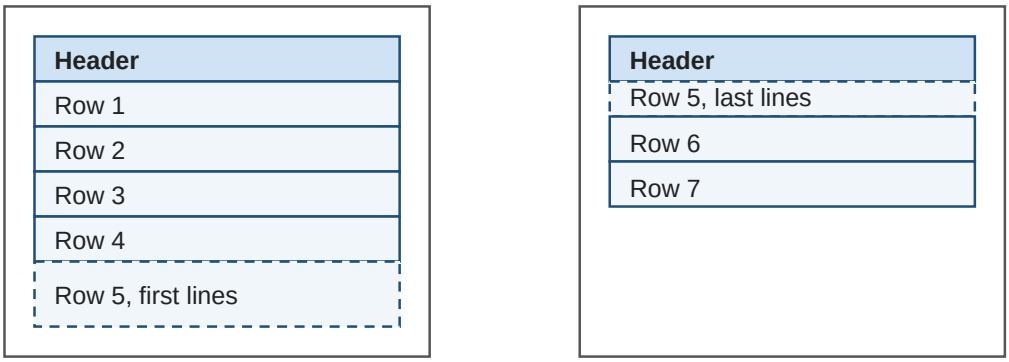
   The rows of a header group or a footer group are repeated on every page that the table uses, when the group has `break-inside: avoid`. A group that is taller than a quarter of the page is not repeated, so that the pages keep room for the rows of the body:

   ```
   header {
       display: table-header-group;
       break-inside: avoid;
   }

   footer {
       display: table-footer-group;
       break-inside: avoid;
   }
   ```

   The footer repeats at the end of each page, directly after the last row of the page. Without `break-inside: avoid`, the header is only at the top of the table, and the footer is only at its end.

A table on two pages



The header group repeats on each page. A tall row breaks, and its cells continue.

5. Force or avoid breaks

   The properties `break-before` and `break-after` work on rows and on row groups. With the value page the table continues on a new page, and with `avoid` CSSMedia tries not to break there:

   ```
   body[name="appendix"] {
       break-before: page;
   }
   ```

   A table is not held back by `orphans` and `widows`: a single row can be alone on a page. If you do not want that, use `break-before: avoid` and `break-after: avoid` on the rows that must stay together. The break moves to an earlier place between rows.

Tables reference

This reference lists the display values that make a table and the properties that style it. The sizes of boxes, the borders and the backgrounds are in the boxes reference, and the breaks of pages are in the page setup reference.

Display values of tables

display	The element is	Specification
`table`	A table, as a block.	CSS Tables 3 §2
`inline-table`	A table that sits in a line of text.	CSS Tables 3 §2
`table-caption`	The caption of the table, above it or below it.	CSS Tables 3 §2
`table-header-group`	Rows that are always at the start of the table, and that can repeat on every page.	CSS Tables 3 §2

display	The element is	Specification
`table-row-group`	A group of rows.	CSS Tables 3 §2
`table-footer-group`	Rows that are always at the end of the table, and that can repeat on every page.	CSS Tables 3 §2
`table-row`	A row of cells.	CSS Tables 3 §2
`table-cell`	A cell, with the content of the table.	CSS Tables 3 §2
`table-column`	A column. It has no content, and it gives a width, a border or a background to the cells of a column.	CSS Tables 3 §2
`table-column-group`	A group of columns.	CSS Tables 3 §2

Table properties

Name	Description	Values	Default	Specification
`table-layout`	Chooses how the widths of the columns are found: from the content of all the cells, or from the first row and the column elements.	`auto` or `fixed`; `fixed` needs a width on the table	`auto`	CSS Tables 3 §3.5.1
`border-collapse`	Chooses between a border for each cell and one border shared by neighbors.	`separate` or `collapse`	`separate`	CSS Tables 3 §3.5.2
`border-spacing`	Sets the gap between the borders of neighbor cells when the borders are separate.	One length for both directions, or two lengths for the horizontal and the vertical gap	`0`	CSS Tables 3 §3.5.2.1
`caption-side`	Puts the caption above or below the table.	`top` or `bottom`	`top`	CSS Tables 3 §3.5.3
`empty-cells`	Chooses whether the borders and the background of a cell without content are shown, when the borders are separate.	`show` or `hide`	`show`	CSS Tables 3 §5.2

Name	Description	Values	Default	Specification
`vertical-align`	On a cell, sets where its content goes when the row is taller than the content.	`baseline`, `top`, `middle` or `bottom`	`baseline`	CSS Inline 3

Multi-column layout

Flexbox and grid

Shadows, opacity, transforms and masks

Define pages

Use an @page rule to set the size, margins and other properties of the pages in the PDF.

Procedure

1. Write an @page rule

   An @page rule has an optional list of page selectors, separated by commas, and a block of declarations:

   ```
   @page {
       size: A4;
   }

   @page :first, :left {
       margin-top: 25mm;
   }
   ```

   The first rule has no selector and applies to every page. The second applies to pages that match either selector. A selector is an optional page type name followed by page pseudo-classes, with no white space between them.

   A block can contain page properties, the `size` descriptor, and page-margin rules.

2. Set the page size with `size`

   The `size` descriptor sets the size of the page. Its initial value is `auto`, which gives an A4 page.

   ```
   @page {
       size: A4;
   }

   @page landscape {
       size: A4 landscape;
   }

   @page square {
       size: 150mm;
   }

   @page custom {
       size: 210mm 297mm;
   }
   ```

   - A page size name, such as A4, selects a standard sheet.
   - `portrait` puts the shorter sides horizontal and `landscape` puts the longer sides horizontal. Use one of them with a name, or alone to orient the default A4 sheet.
   - One length sets the width and the height, which gives a square page.
   - Two lengths set the width and then the height. A length cannot be negative.

   These are the page size names:

Name	Millimeters	Inches
A3	297 x 420	11.69 x 16.54
A4	210 x 297	8.27 x 11.69
A5	148 x 210	5.83 x 8.27
B4	250 x 353	9.84 x 13.90
B5	176 x 250	6.93 x 9.84
`JIS-B4`	257 x 364	10.12 x 14.33
`JIS-B5`	182 x 257	7.17 x 10.12
`letter`	215.9 x 279.4	8.5 x 11
`legal`	215.9 x 355.6	8.5 x 14
`ledger`	279.4 x 431.8	11 x 17

The sizes are width by height, in portrait orientation.

7. Set the page margins

   Page margins are the space between the edge of the page and the page area, where the content of the document is placed. If you do not set a margin, it is `1in`.

   The `margin` property sets all four margins. It takes one to four values, which are lengths, percentages or `auto`:

   - One value sets all four sides.
   - Two values set the top and bottom, then the right and left.
   - Three values set the top, then the left and right, then the bottom.
   - Four values set the top, right, bottom and left, in that order.

   A percentage is a fraction of the page width for the left and right margins, and of the page height for the top and bottom margins.

```
@page {
    size: A4;
    margin: 20mm 25mm;
}
```

   Use `margin-top`, `margin-right`, `margin-bottom` and `margin-left` to set one side. For example, use wider inner margins in a two-sided document:

```
@page :left {
    margin-left: 30mm;
}

@page :right {
    margin-right: 30mm;
}
```

12. Select pages

A page selector chooses the pages that a rule applies to, by page type or by position.

Page types

A page type name matches the pages that get that name from the page property. For example, `@page wide` matches the pages of type `wide`. Names are case-sensitive.

Page pseudo-classes

- `:first` matches the first page.
- `:left` and `:right` match left and right pages. In a document that is read left to right, the first page is a right page.
- `:blank` matches the empty pages that forced page breaks produce.
- `:nth(An+B)` matches pages by their position in the document. `:nth(2n)` selects pages 2, 4, 6 and so on, and `:nth(3n+1)` selects pages 1, 4, 7 and so on.

Add `of` and a page type to count only the pages of one page group, as in `:nth(1 of chapter)`. An element that has a named page and a forced page break starts a new page group.

```
@page :first {
    margin-top: 30mm;
}

@page :nth(2n) {
    margin-top: 25mm;
}
```

17. Assign a page type with page

    The page property gives the pages of an element a page type. Its value is `auto` or a name that you also use in an @page rule.

    ```
    @page wide {
        size: A4 landscape;
    }

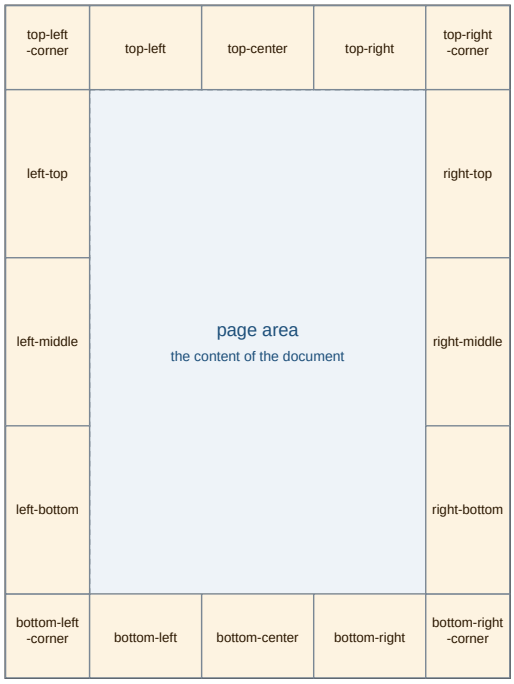
    wide-table {
        page: wide;
    }
    ```

    With `auto`, an element takes the page type of its nearest ancestor that has a name. A box with a different page type than the page before it starts on a new page.

18. Fill the page margins

    A page-margin rule is an at-rule inside an @page rule, such as `@top-center` or `@bottom-right`. Each one defines a box in the margin. The box appears only if its `content` property has a value other than none or `normal`. Use `counter(page)` to show the page number.

The page-margin boxes



For example, put a running title in the top-center box and the page number in the bottom-center box:

```
@page {
    @top-center {
        content: "Document title";
    }

    @bottom-center {
        content: counter(page);
    }
}
```

The page reference lists the 16 page-margin rules and where each box is. See Named strings to show the text of an element in a margin box.

Page setup reference

This reference lists the rules, descriptors and properties that CSSMedia uses to define pages.

Page rule, size and page type

Name	Description	Values	Default	Specification
@page	Defines page-box declarations and may contain page-margin rules.	Optional comma-separated page selectors		CSS Paged Media 3, @page

Name	Description	Values	Default	Specification
`size`	Sets page-box size and orientation. `auto` uses A4.	`auto`; one or two lengths; or A3, A4, A5, B4, B5, `JIS-B4`, `JIS-B5`, `ledger`, `legal`, or `letter`, optionally combined with `portrait` or `landscape`	A4	CSS Paged Media 3, `size`
`page`	Assigns a named page type to a box.	`auto` or a custom identifier	`auto`	CSS Paged Media 3, `page`

Page margins

Name	Description	Values	Default	Specification
`margin`	Sets the space outside the four sides of the box. One value applies to all sides. Two values set top and bottom, then left and right. Three set top, then left and right, then bottom. Four set top, right, bottom and left.	One to four lengths or percentages, or `auto`	`0`	CSS Box Model 4 §3.2
`margin-top`	Sets the space above the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1
`margin-right`	Sets the space at the right of the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1
`margin-bottom`	Sets the space below the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1
`margin-left`	Sets the space at the left of the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1

Page-margin rules

Name	Description	Values	Default	Specification
`@top-left-corner`	Defines the fixed-size box at the intersection of the top and left page margins.			CSS Paged Media 3, `@top-left-corner`
`@top-left`	Defines the box between the top-left corner and top-center boxes.			CSS Paged Media 3, `@top-left`

Name	Description	Values	Default	Specification
`@top-center`	Defines the centered box in the top page margin.			CSS Paged Media 3, `@top-center`
`@top-right`	Defines the box between the top-center and top-right corner boxes.			CSS Paged Media 3, `@top-right`
`@top-right-corner`	Defines the fixed-size box at the intersection of the top and right page margins.			CSS Paged Media 3, `@top-right-corner`
`@right-top`	Defines the box between the top-right corner and right-middle boxes.			CSS Paged Media 3, `@right-top`
`@right-middle`	Defines the centered box in the right page margin.			CSS Paged Media 3, `@right-middle`
`@right-bottom`	Defines the box between the right-middle and bottom-right corner boxes.			CSS Paged Media 3, `@right-bottom`
`@bottom-right-corner`	Defines the fixed-size box at the intersection of the bottom and right page margins.			CSS Paged Media 3, `@bottom-right-corner`
`@bottom-right`	Defines the box between the bottom-center and bottom-right corner boxes.			CSS Paged Media 3, `@bottom-right`
`@bottom-center`	Defines the centered box in the bottom page margin.			CSS Paged Media 3, `@bottom-center`
`@bottom-left`	Defines the box between the bottom-left corner and bottom-center boxes.			CSS Paged Media 3, `@bottom-left`
`@bottom-left-corner`	Defines the fixed-size box at the intersection of the bottom and left page margins.			CSS Paged Media 3, `@bottom-left-corner`
`@left-bottom`	Defines the box between the left-middle and bottom-left corner boxes.			CSS Paged Media 3, `@left-bottom`
`@left-middle`	Defines the centered box in the left page margin.			CSS Paged Media 3, `@left-middle`
`@left-top`	Defines the box between the top-left corner and left-middle boxes.			CSS Paged Media 3, `@left-top`

Breaks

Name	Description	Values	Default	Specification
`break-before`	Forces or avoids a break before a block, a row or a row group. `page`, `always` and `all` start a new page. `left` and `verso` start the next left page, and `right` and `recto` the next right page, with a blank page when needed. `avoid` and `avoid-page` move the break to an earlier place. `column` breaks only in columns.	`auto`, `avoid`, `avoid-page`, `page`, `always`, `all`, `left`, `right`, `recto`, `verso`, `column` or `avoid-column`	`auto`	CSS Fragmentation 4 §3.1
`break-after`	Forces or avoids a break after a block, a row or a row group, with the same values as `break-before`.	As `break-before`	`auto`	CSS Fragmentation 4 §3.1
`break-inside`	Avoids a break inside a box, so that it stays on one page when it fits on one. A table header or footer group with `avoid` repeats on each page of the table.	`auto`, `avoid`, `avoid-page` or `avoid-column`	`auto`	CSS Fragmentation 4 §3.2
`orphans`	Sets how many lines of a paragraph must stay at the bottom of a page. If fewer fit, the whole paragraph goes to the next page.	An integer of 1 or more	2	CSS Fragmentation 4 §3.3
`widows`	Sets how many lines of a paragraph must go to the top of the next page. If fewer would, lines from the previous page go with them.	An integer of 1 or more	2	CSS Fragmentation 4 §3.3

Page orientation, bleed and marks

Named strings

A named string saves text from an element so that you can use it elsewhere, such as in a running header or footer. `string-set` saves the text and `string()` retrieves it.

About this task

The element names in the examples, such as `title` and `h1`, are placeholders. Use the names that occur in your XML.

Procedure

1. Save a string with `string-set`

   The `string-set` property takes none, which is the initial value, or a comma-separated list of assignments. Each assignment is a name that you choose, followed by what to save.

   ```
   h1 {
       string-set: chapter-title content(text);
   }
   ```

   Each `h1` element saves its text in `chapter-title`. When a later `h1` is found, it replaces the saved value. One declaration can save several strings:

   ```
   h1 {
       string-set: chapter-title content(text), chapter-label "Chapter "
   counter(chapter);
   }
   ```

   What to save is a list of one or more of these items, joined in order:

   - A quoted string, such as `"Chapter "`.
   - `content()` gives text of the element. `content(text)`, which is the default, gives the text of the element and its descendants, with white space collapsed. `content(before)` and `content(after)` give the text generated by `::before` and `::after`. `content(first-letter)` gives the first letter.
   - `attr(name)` gives the value of an attribute of the element. `attr(name, "fallback")` gives the fallback when the attribute is missing.
   - `counter(name)` gives the value of the innermost counter with that name. `counters(name, ".")` gives the values of all the counters with that name, from the outermost, joined with the string. Both accept a counter style as the last argument, such as `upper-roman`.

   This XML and stylesheet save the text, a prefix and an attribute of a title:

   ```
   <title code="A-4">Install <em>the package</em></title>
   ```

```
title {
    string-set: title-text content(text), title-prefix content(before), title-
code attr(code, "No code");
}

title::before {
    content: "Chapter ";
}

@page {
    @top-left {
        content: string(title-prefix) string(title-code);
    }
    @top-right {
        content: string(title-text);
    }
}
```

The left margin shows "Chapter A-4" and the right margin shows "Install the package". For a title without a `code` attribute, the left margin shows "Chapter No code".

A string is saved where the box of its element starts, or where it would start if the element has `display: none`. That is why a page can show different values of the same string, depending on which elements it contains.

6. Retrieve a string with `string()`

   Use `string(name)` in the `content` property of a page-margin box. The name is the one you wrote in `string-set`. An optional second argument chooses which value to use on the page.

   ```
   @page {
       @top-center {
           content: string(chapter-title, first);
       }
   }
   ```

   - `first` is the first value saved on the page. If the page does not save one, it is the value in effect at the end of the previous page. This is the default.
   - `start` is the value in effect at the top of the page. It is empty if the string has not been saved yet.
   - `last` is the value in effect at the end of the page.
   - `first-except` works like `first`, but is empty on a page that saves the string. It is useful to hide a running header on the page where a chapter starts.

Named strings reference

This reference lists the property and the functions that CSSMedia uses for named strings.

Name	Description	Values	Default	Specification
`string-set`	Assigns named values from an element for retrieval in page content.	none or comma-separated pairs of a name and one or more strings		CSS Content 3 §2.7.1
`content()`	Gives text of an element. Use it in the value of `string-set`.	`text`, `before`, `after` or `first-letter`	`text`	CSS GCPM 3 §1.1.1.1
`string()`	Gives the value of a named string saved with `string-set`. Use it in the `content` property of a page-margin box.	A name, and an optional mode: `first`, `start`, `last` or `first-except`	`first`	CSS GCPM 3 §1.1.2

Links

A link points from an element to another part of the document or to an external address. XML has no standard names for links, so you use the CSSMedia property `-cssmedia-link` to tell CSSMedia which elements are links and where they go. The link reference lists the property, the function and the pseudo-classes used for links.

About this task

The element and attribute names in the examples, such as `xref` and `target`, are placeholders. Use the names that occur in your XML.

Procedure

1. Set a link destination

   The `-cssmedia-link` property sets the destination of an element. It is a CSSMedia extension, so its name has the prefix `-cssmedia-`. Its value is none, which is the initial value and means that the element is not a link, or one or more of these:

   - A string, such as `"chapter-2.xml#installation"` or `"#details"`.
   - A `url()` value.

   CSSMedia joins the values in order, without spaces, to make the destination. The property is not inherited, so set it on each element that is a link. Use none to remove a link.

   This XML has an `xref` element with a `target` attribute:

   ```
   <xref target="chapter-2.xml#installation">Installation instructions</xref>
   ```

   Use `attr()` to take the destination from the attribute:

   ```
   xref {
       -cssmedia-link: attr(target);
   }
   ```

   The text of the element is the text of the link, and the attribute is its destination. If the attribute is missing, `attr()` gives an empty string, unless you give a fallback.

   To link to a place in the same document, add # before the ID:

   ```
   <xref target="details">More details</xref>
   ```

   ```
   xref {
       -cssmedia-link: "#" attr(target);
   }
   ```

   The link goes to `#details`, the element of the document with the ID `details`.

   To link to an external address, write the address as a string:

   ```
   xref[role="external"] {
       -cssmedia-link: "https://example.org/manual/installation.html";
   }
   ```

A relative string, such as `"chapter-2.xml#installation"`, is resolved against the address of the XML document. A `url()` value is resolved against the address of the stylesheet. Use a string when the address is relative to the XML document.

4. Style the links

   Setting `-cssmedia-link` does not change how a link looks. Use `:any-link` to select the elements that have a destination, and set the properties that you want:

   ```
   :any-link {
       color: navy;
   }
   ```

   `:link` selects the same elements, because a PDF has no visited links. `:visited` never matches.

5. Select links by destination

   `:local-link` selects the links that point to the same document. CSSMedia ignores the fragment when it compares the addresses, so a link to `#details` is a local link.

   ```
   xref {
       -cssmedia-link: attr(target);
   }

   xref:local-link {
       color: navy;
   }
   ```

   Add a number to compare part of the address: `:local-link(0)` selects the links with the same origin, which is the scheme, the host and the port. `:local-link(1)` also compares the first segment of the path, `:local-link(2)` the first two, and so on. Combine it with `:not()` to select the links to other origins:

   ```
   xref:not(:local-link(0)) {
       color: maroon;
   }
   ```

6. Show a counter of the destination

   `-cssmedia-link-counter()` gives the value of a counter at the destination of the link of an element. Use it in the `content` property, for example to show the number of the footnote that a link points to:

   ```
   <footnoteref linkend="note-1"/>
   ```

   ```
   footnoteref {
       -cssmedia-link: "#" attr(linkend);
       content: -cssmedia-link-counter(footnote);
   }
   ```

   The element links to the element with the ID `note-1` and shows the value of the `footnote` counter of that element. An optional second argument sets the counter style, as in `-cssmedia-link-counter(footnote, lower-alpha)`.

Link reference

This reference lists the property, the function and the pseudo-classes that CSSMedia uses for links.

Property and function

Name	Description	Values	Default	Specification
`-cssmedia-link`	Makes an XML element a link to another part of the document or to an external address. The values are joined in order to make the destination.	none, or one or more strings or `url()` values	none	CSSMedia extension
`-cssmedia-link-counter()`	Gives the value of a counter at the destination of the link of the element, such as the number of a footnote that the link points to. CSSMedia extension.	The name of a counter, and an optional counter style.		CSSMedia extension

Pseudo-classes

Name	Description	Values	In a PDF	Specification
`:link`, `:any-link`	Elements that are hyperlinks, because `-cssmedia-link` gives them a destination.		Matches	Selectors 4 §8.1, §8.2
`:local-link`	Hyperlinks that point to the same document. The optional number limits how many path segments are compared.	An optional non-negative integer in parentheses.	Matches	Selectors 5 §2.1
`:visited`, `:target`	Links that were visited, and the element that a link points to. A PDF has no browsing history and no current target.		Never matches	Selectors 4 §8.2, §8.3

Footnotes

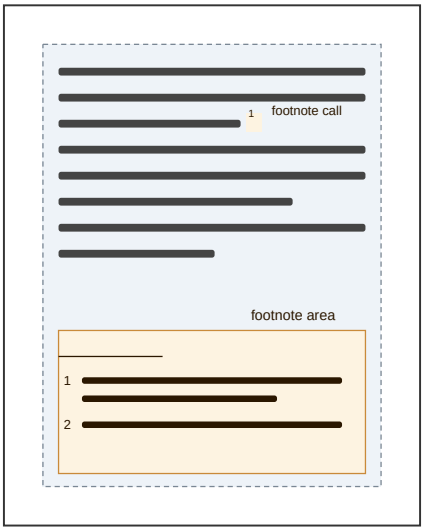
A footnote is an element that CSSMedia moves to the bottom of the page where the text refers to it. In the text, a number takes its place. The footnote reference lists the properties, rules and pseudo-elements used here.

About this task

These terms are used in this chapter:

- The **footnote body** is the content of the footnote element.
- The **footnote call** is the number that replaces the footnote in the text.
- The **footnote marker** is the number at the start of the footnote body.
- The **footnote area** is the box at the bottom of the page that holds the footnote bodies.

A page with footnotes



The element names in the examples, such as `fn`, are placeholders. Use the names that occur in your XML.

Procedure

1. Mark an element as a footnote

   Set `float: footnote` on the element that holds the text of the footnote:

   ```
   <p>Swordfish stores translations in memories<fn>A memory is a database of
   translated segments.</fn>.</p>
   ```

   ```
   fn {
       float: footnote;
       font-size: 8pt;
   }
   ```

   CSSMedia removes `fn` from the text, puts a number in its place, and shows the content of `fn` in the footnote area of the same page, in document order. If the footnote does not fit on the page, the rest continues on the next pages.

The rule of the footnote element styles the footnote body, so set the font size and the line height there.

2. Number the footnotes

   Footnotes are numbered with the `footnote` counter. A counter is a number that CSS keeps for you and that you can show in the content of a document. The `footnote` counter already exists: CSSMedia creates it at the start of the document with the value 0 and adds 1 each time it finds a footnote. The first footnote is 1, the second is 2, and so on through the whole document.

   The call and the marker of a footnote show the same value, because both use `counter(footnote)`.

   You change the numbering with the properties `counter-reset`, `counter-set` and `counter-increment`:

   - `counter-reset: footnote` sets the counter to 0, so the next footnote is 1. `counter-reset: footnote 9` sets it to 9, so the next footnote is 10.
   - `counter-set: footnote 4` also sets the value of the counter.
   - `counter-increment: footnote 2` adds 2 to the counter, and `counter-increment: footnote -1` subtracts 1.

   Where you write the declaration decides where the numbering changes. In an element rule, it changes when the element starts. This example starts the numbers at 1 in each chapter:

```
chapter {
    counter-reset: footnote;
}
```

   In an @page rule, it changes on every page. This example starts the numbers at 1 on each page:

```
@page {
    counter-reset: footnote;
}
```

   With the rule in @page, the number of a footnote depends on the page of its call, and a footnote that continues on the next page keeps its number.

   The optional second argument of `counter()` is a counter style, which sets how the number is written. For example, `counter(footnote, lower-alpha)` shows a, b, c, and `counter(footnote, upper-roman)` shows I, II, III. A style can also list the symbols to use:

```
fn::footnote-call {
    content: counter(footnote, symbols("*" "†" "‡"));
}

fn::footnote-marker {
    content: counter(footnote, symbols("*" "†" "‡")) " ";
}
```

   The first three footnotes show *, † and ‡. Write the same style in the call and in the marker.

6. Style the call and the marker

The pseudo-element `::footnote-call` is the number in the text. The pseudo-element `::footnote-marker` is the number at the start of the footnote. By default, the call is the value of the `footnote` counter as a superscript, and the marker is the value followed by a period and a space.

```
::footnote-call {
    font-size: 70%;
    vertical-align: super;
    line-height: 0;
}

::footnote-marker {
    font-weight: bold;
}
```

Use the `content` property to change what they show. For example, show the number in brackets:

```
fn::footnote-call {
    content: "[" counter(footnote) "]";
}

fn::footnote-marker {
    content: "[" counter(footnote) "] ";
}
```

7. Use your own callouts

   To show a symbol or a label from the XML instead of a number, give the call and the marker that text, and stop the footnote from using a number. Every footnote adds 1 to the counter, and `counter-increment: footnote -1` takes it back, so the other footnotes keep their numbers.

```
<p>The price<fn callout="*">Prices do not include taxes.</fn> is final.</p>
```

```
fn[callout] {
    counter-increment: footnote -1;
}

fn[callout]::footnote-call {
    content: attr(callout);
}

fn[callout]::footnote-marker {
    content: attr(callout) " ";
}
```

8. Style the footnote area

   Use an `@footnote` rule inside an `@page` rule to style the footnote area. The area sits at the bottom of the page area. A border and a margin separate it from the text.

```
@page {
    @footnote {
        border-top: 0.5pt solid black;
        margin-top: 6pt;
        padding-top: 3pt;
    }
}
```

Set `max-height` to limit the height of the area, so that a page does not fill up with footnotes. The limit does not apply to a page that has only footnotes. Because `@footnote` belongs to an @page rule, you can set a different limit for a named page:

```
@page limited {
    @footnote {
        max-height: 25mm;
    }
}
```

9. Choose how footnotes are displayed

   The `footnote-display` property sets how a footnote body appears in the footnote area:

   - `block` puts each footnote in its own block. This is the initial value.
   - `inline` runs the footnotes together in one paragraph.
   - `compact` puts footnotes in the same line when they fit, and in separate blocks when they do not.

   With `inline` and `compact`, add a margin after each footnote to separate it from the next one:

```
fn {
    float: footnote;
    footnote-display: compact;
    margin-inline-end: 1em;
}
```

13. Choose what happens when a footnote does not fit

    The `footnote-policy` property sets what CSSMedia does when the body of a footnote does not fit on the page of its call:

    - `auto` starts the footnote on the page of its call and continues it on the next pages. This is the initial value.
    - `line` moves the line that has the call to the next page, together with the footnote. Widows and orphans are respected, so more than one line can move.
    - `block` moves the whole paragraph that has the call to the next page, together with the footnote.

```
fn {
    float: footnote;
    footnote-policy: line;
}
```

Footnote reference

This reference lists the properties, the rule, the counter and the pseudo-elements that CSSMedia uses for footnotes.

Properties, rule and counter

Name	Description	Values	Default	Specification
`float`	Takes a box out of the normal flow and places it at the side, at the top or at the bottom of its container. The value `footnote` makes the element a footnote.	`none`, `left`, `right`, `top`, `bottom`, `block-start`, `block-end`, `inline-start`, `inline-end`, `snap-block`, `snap-inline` or `footnote`	`none`	CSS Page Floats 3 §3.2
`footnote-display`	Sets how a footnote body appears in the footnote area.	`block`, `inline`, or `compact`	`block`	CSS GCPM 3 §2.3
`footnote-policy`	Sets what happens when a footnote does not fit on the page of its call. `auto` continues it on the next pages, `line` moves the line with the call to the next page, and `block` moves the paragraph.	`auto`, `line`, or `block`	`auto`	CSS GCPM 3 §2.8
`footnote` counter	The counter that numbers the footnotes. Every footnote adds 1 to it. Change it with `counter-reset`, `counter-set` and `counter-increment`, in an element rule to number by section or in an @page rule to number by page.	Show it with `counter(footnote)`, with an optional counter style.	0 at the start of the document.	CSS GCPM 3 §2.5

Name	Description	Values	Default	Specification
`@footnote`	Styles the footnote area at the bottom of the page. It goes inside an `@page` rule.	A block of declarations, such as borders, margins, padding and `max-height`.		CSS GCPM 3 §2.4

Pseudo-elements

Name	Description	Values	In a PDF	Specification
`::footnote-call`	The reference that replaces a footnote in the text. By default it is the value of the `footnote` counter, as a superscript.		Matches	CSS GCPM 3 §2.6
`::footnote-marker`	The marker at the start of a footnote body. By default it is the value of the `footnote` counter, followed by a period and a space.		Matches	CSS GCPM 3 §2.7

Table of contents

A table of contents lists the headings of a document with the page where each one starts. CSSMedia builds it from the elements that you mark as headings, so the table is always up to date. The table of contents reference lists the properties, functions and at-rules that you use.

About this task

The element names in the examples, such as `contents`, `chapter` and `title`, are placeholders. Use the names that occur in your XML.

The table of contents and the bookmarks of the PDF come from the same marking. An element with a `bookmark-level` is a bookmark, and it is an entry in the table of contents. CSSMedia extensions have the prefix `-cssmedia-`, and CSSMedia ignores the same names without it.

Procedure

1. Mark the headings

   Set `bookmark-level` on the elements that you want in the table. Level 1 is the top level. Elements without a level, which is the initial value none, are not listed:

   ```
   chapter > title {
       bookmark-level: 1;
   }

   section > title {
       bookmark-level: 2;
   }
   ```

   The entry of an element has the text of the element. Use `bookmark-label` to change the text of the bookmark and of the entry. Its value is a list of strings, `attr()` values and `content()` values. The initial value is `content(text)`.

   A bookmark shows its child bookmarks when the PDF opens. Set `bookmark-state: closed` on a block-level element to hide them.

   > **Note**
   >
   > CSSMedia creates the bookmarks of the PDF automatically, one for each element that has a `bookmark-level`. PDF viewers, such as Adobe Acrobat, display them in a side panel. The bookmarks need only the `bookmark-level` declarations in your CSS. If no element has a `bookmark-level`, the PDF has no bookmarks. A table of contents is optional, and its options, such as `-cssmedia-toc-depth` and `-cssmedia-toc-label`, do not change the bookmarks.

2. Place the table of contents

   Use `-cssmedia-toc()` as the value of `content`. The table appears where the content is generated. A pseudo-element of an empty element is a good place. The table can come before the headings that it lists:

```
contents::before {
    content: -cssmedia-toc();
    display: block;
}
```

Each entry is a block with the text of the heading, a leader and the page number of the heading. The entry is also a link to the heading. The table is empty, and CSSMedia leaves it out, when no element has a `bookmark-level`.

3. Add a title

   The table has no title unless you set `-cssmedia-toc-title` where `content` is `-cssmedia-toc()`. Its value is a list of strings, like the value of `content`:

```
contents::before {
    content: -cssmedia-toc();
    -cssmedia-toc-title: "Contents";
    display: block;
}
```

4. Limit the depth

   The table lists every level. Set `-cssmedia-toc-depth` to the deepest level that you want:

```
contents::before {
    content: -cssmedia-toc();
    -cssmedia-toc-title: "Contents";
    -cssmedia-toc-depth: 2;
    display: block;
}
```

   The value `all` lists every level. The depth does not change the bookmarks of the PDF.

5. Style the table

   The title, the entries and the dividers have no element in your XML, so three at-rules style them. Each at-rule holds ordinary declarations, and its boxes inherit from the element that holds the table:

   - `@-cssmedia-toc-title` styles the title.
   - `@-cssmedia-toc-entry` styles the entries.
   - `@-cssmedia-toc-divider` styles the dividers, which the next step explains.

   Write a level after the name of `@-cssmedia-toc-entry` or `@-cssmedia-toc-divider` to style only that level. A rule for a level comes after the rules without level, so it wins. This example indents the entries of level 2:

```
@-cssmedia-toc-title {
    font-size: 20pt;
    margin-bottom: 10pt;
}

@-cssmedia-toc-entry 2 {
    margin-left: 24pt;
}
```

The `-cssmedia-toc-leader` property sets the leader between the text and the page number. Set it in `@-cssmedia-toc-entry`. The values are `dotted`, which is the initial value, `solid`, space and a string. A string is repeated to fill the space:

```
@-cssmedia-toc-entry {
    -cssmedia-toc-leader: solid;
}
```

9. Change the text of an entry

   By default, an entry has the same text as the bookmark of its element. Set `-cssmedia-toc-label` on the heading element to give the entry another text. The value is a list, like the value of `bookmark-label`. This rule adds a word to the entries of the appendixes. The bookmarks do not change:

   ```
   appendix > title {
       -cssmedia-toc-label: "Appendix " content();
   }
   ```

   The initial value is `auto`, which uses `bookmark-label`.

10. Group the entries with dividers

    A divider is a line of text before an entry. Use it to name a group of entries. Set `-cssmedia-toc-divider` on the heading element where the group starts:

    ```
    chapter:first-of-type > title {
        -cssmedia-toc-divider: "Chapters";
    }

    appendix:first-of-type > title {
        -cssmedia-toc-divider: "Appendixes";
    }

    @-cssmedia-toc-divider {
        font-weight: bold;
        margin-top: 8pt;
    }
    ```

    The initial value is none, which adds no divider.

11. Add the title to the bookmarks

    To make the title of the table a bookmark, set `bookmark-level` in `@-cssmedia-toc-title`:

    ```
    @-cssmedia-toc-title {
        bookmark-level: 1;
    }
    ```

Next Steps

The same pattern gives page numbers in other places. `target-counter()` gives the value of a counter at the destination of a link, and `leader()` fills the space before it. The Links chapter explains how to set destinations with `-cssmedia-link`.

Table of contents reference

This reference lists the properties, functions and at-rules that CSSMedia uses for bookmarks and tables of contents.

Properties and functions

Name	Description	Values	Default	Specification
`bookmark-level`	Makes an element a bookmark of the PDF and an entry in a table of contents, and sets its level. Level 1 is the top level.	none or an integer of 1 or greater	none	CSS Content 3 §3.1
`bookmark-label`	Sets the text of the bookmark of an element. It is also the default text of the entry of the element in a table of contents.	A list of generated content items	`content(text)`	CSS Content 3 §3.2
`bookmark-state`	Sets whether the bookmark of a block-level element shows its child bookmarks or hides them when the PDF opens.	`open` or `closed`	`open`	CSS Content 3 §3.3
`-cssmedia-toc()`	Generates a table of contents with an entry for each element that has a `bookmark-level`. Use it as the value of `content`. CSSMedia extension.	No arguments.		CSSMedia extension
`-cssmedia-toc-title`	Sets the title of the table of contents. Set it where `content` is `-cssmedia-toc()`. The title is also a bookmark if `@-cssmedia-toc-title` sets a `bookmark-level`. CSSMedia extension.	none or a list of generated content items	none	CSSMedia extension

Name	Description	Values	Default	Specification
`-cssmedia-toc-depth`	Sets the deepest level of `bookmark-level` that gets an entry in the table of contents. Set it where `content` is `-cssmedia-toc()`. CSSMedia extension.	`all` or an integer of 1 or greater	`all`	CSSMedia extension
`-cssmedia-toc-label`	Sets the text of the entry of an element in the table of contents. Set it on the element that has the `bookmark-level`. CSSMedia extension.	`auto` or a list of generated content items	`auto`, which uses the value of `bookmark-label`	CSSMedia extension
`-cssmedia-toc-divider`	Adds a line of text before the entry of an element in the table of contents, to group entries. Set it on the element that has the `bookmark-level`. CSSMedia extension.	none or a list of generated content items	none	CSSMedia extension
`-cssmedia-toc-leader`	Sets the leader between the text of an entry and its page number. Set it in `@-cssmedia-toc-entry`. CSSMedia extension.	`dotted`, `solid`, space or a string	`dotted`	CSSMedia extension
`leader()`	Fills the space between the content before it and the content after it on a line with a repeated pattern. Use it in `content`.	`dotted`, `solid`, space or a string		CSS Content 3 §2.5.1

Name	Description	Values	Default	Specification
`target-counter()`	Gives the value of a counter at the destination of a link, such as the page number of a heading. Use it in `content`.	A link, as `attr()`, a string or `url()`; a counter name; and an optional counter style		CSS Content 3 §2.6.1

At-rules

Name	Description	Values	Default	Specification
`@-cssmedia-toc-title`	Styles the title of the table of contents. CSSMedia extension.	A block of declarations.		CSSMedia extension
`@-cssmedia-toc-entry`	Styles the entries of the table of contents. Write a level after the name to style only the entries of that level. CSSMedia extension.	An optional integer of 1 or greater, and a block of declarations.		CSSMedia extension
`@-cssmedia-toc-divider`	Styles the dividers of the table of contents. Write a level after the name to style only the dividers of the entries of that level. CSSMedia extension.	An optional integer of 1 or greater, and a block of declarations.		CSSMedia extension

Images

An element of your XML can show an image from a file. This chapter explains how to turn an element into an image, how to size it and how to fit it in its box. The images reference at the end lists the properties.

XML has no element for images, so the stylesheet decides which elements are images. An element whose `content` is an image is a replaced element: CSSMedia shows the image in its place and ignores what the element contains. Most vocabularies keep the name of the file in an attribute, and the stylesheet reads it from there.

CSSMedia reads these image formats, and it knows the format from the content of the file, not from its name:

Format	What CSSMedia reads
PNG	All color types and bit depths, interlaced files, transparency and the color profile of the file.
JPEG	Baseline, extended and progressive files of 8 bits, in gray, RGB or CMYK, with the color profile of the file.
WebP	Lossy and lossless files, with transparency and the color profile of the file.
SVG	Drawings, which CSSMedia draws as vector graphics. SVG explains how to write them.

The same files work as the images of a `background-image`, which Backgrounds explains. An image is a picture in the PDF: it does not move, and its pixels are not text.

The element names in the examples of this chapter, such as `figure` and `src`, are placeholders. Use the names that occur in your XML.

The topics of this chapter follow the steps of the work:

- Place an image
- Fit an image in its box

Place an image

Use the `content` property to make an element show an image, then give the image the size that you need.

Procedure

1. Make an element an image

   Set `content` to a `url()` that names the file. The element shows the image, and what it contains is ignored:

   ```
   logo {
       content: url("images/logo.png");
   }
   ```

2. Read the file name from your XML

When each element has its own file, keep the name in an attribute and read it with `attr()`. Add the type `url` so that CSSMedia reads the value as a file name:

```
figure {
    display: block;
    content: attr(src url);
}
```

This rule shows `<figure src="images/pump.jpg"/>` as the picture `images/pump.jpg`.

3. Say where the file is

   A file name can be written in these ways:

   - A relative name in `attr()` is found from the location of the XML file, or from the `xml:base` of the element, if there is one.
   - A relative name in a `url()` of the stylesheet is found from the location of the stylesheet.
   - An absolute name can be a `file:` URL.
   - An `http:` or `https:` URL is downloaded when CSSMedia makes the PDF.
   - A `data:` URL holds the file itself, for example `` followed by the data.

   If CSSMedia cannot read a file, or cannot decode it, the box stays empty and the messages at the end of the run name the file.

9. Put an image in a line of text

   An element is inline by default, so an image is a character of its line. Its bottom edge sits on the baseline of the text. Use `vertical-align` to move it. Set `display: block` to put the image on a line of its own.

   A pseudo-element can hold an image too, which is useful for icons:

```
warning::before {
    content: url("images/warning.png");
    vertical-align: middle;
}
```

10. Set the size

    Without a width and a height, an image has its natural size, which is the size that the file has. One pixel of the image is one CSS pixel, or 0.75pt, so a photo of 1200 x 800 pixels is 900 x 600pt. Set `width`, `height`, or both. If you set only one, CSSMedia calculates the other to keep the proportions of the image.

    A common rule keeps wide pictures in the page:

```
figure {
    max-width: 100%;
}
```

    Files from cameras and scanners have many pixels for their size on paper. Use `image-resolution` to say how many pixels make a CSS pixel. The value is a resolution, such as `2dppx` or `192dpi`, and the initial value is `1dppx`. The keyword `from-image` uses the resolution that the file states, when it states one:

```
figure {
    image-resolution: from-image;
}
```

The properties of Box sizes apply to images. An image without a natural size, such as a drawing with no width and height, is 300 x 150 pixels.

Fit an image in its box

When the box of an image has another proportion than the image, you choose how the image fills it. You can also turn an image and choose how its pixels are drawn.

Procedure

1. Choose how the image fills the box

   The `object-fit` property applies when you set both the width and the height of the image, and they do not have the proportion of the image. These are its values:

   - `fill` is the initial value. The image is stretched to the box, and its proportions change.
   - `contain` scales the image as large as it can be and still fit in the box. Part of the box can stay empty.
   - `cover` scales the image as small as it can be and still cover the box. Part of the image is cut.
   - none keeps the natural size of the image.
   - `scale-down` is the smaller result of none and `contain`, so an image is never enlarged.

   The values of object-fit



The image is 60 x 30. The box is 90 x 90. The dashed line is the box.

```
photo {
    width: 120pt;
    height: 120pt;
    object-fit: cover;
}
```

7. Position the image in the box

   When the image does not fill the box, `object-position` decides where it goes. It takes the values of `background-position`: keywords, lengths and percentages. The initial value is `50% 50%`, the center of the box.

```
photo {
    object-fit: cover;
    object-position: top;
}
```

This photo fills its box, and the part that is cut comes from the bottom of the picture.

8. Turn the image

   Cameras store a photo in the same way for every position and write in the file how to turn it. The `image-orientation` property decides what CSSMedia does with that. The initial value, `from-image`, turns the image as the file says. A quarter turn swaps the width and the height of the natural size. Other values:

   - none shows the image as it is stored.
   - An angle, such as `90deg`, turns the image by that angle, rounded to a multiple of 90 degrees. It replaces what the file says.
   - `flip` mirrors the image, after the turn. It can follow an angle.

```
scan {
    image-orientation: 90deg flip;
}
```

12. Choose how the pixels are drawn

    When an image is shown larger than its pixels, `image-rendering` decides how the PDF viewer fills the space between them:

    - `auto` is the initial value. `smooth` and `high-quality` do the same. They ask the viewer to smooth the image.
    - `crisp-edges` asks the viewer not to blend the colors of the pixels.
    - `pixelated` enlarges the image by whole multiples of its pixels, so that each pixel becomes a sharp square. Use it for small images such as bar codes.

```
barcode {
    image-rendering: pixelated;
}
```

> **Note**
>
> A PDF does not move, so the property `image-animation` has no effect.

Images reference

This reference lists the properties that CSSMedia uses to show images and to size and fit them. Backgrounds and the box sizes are in the boxes reference.

Making an element an image

Name	Description	Values	Default	Specification
`content`	Replaces an element's contents or supplies generated content for pseudo-elements and page margin boxes.	`normal`, none, an image that replaces the contents, a list of generated items, or `element()`; a list can include alternative text after `/`	`normal`	CSS Content 3 §1

Image properties

Name	Description	Values	Default	Specification
`image-resolution`	Sets how many pixels of a raster image make one CSS pixel, which sets its natural size. `from-image` uses the resolution that the file states.	`from-image`, a resolution such as `2dppx` or `192dpi`, or both	`1dppx`	CSS Images 4 §6.1
`object-fit`	Chooses how an image fills its box: stretched, scaled to fit, scaled to cover, or at its natural size.	`fill`, none, `contain`, `cover` or `scale-down`; `scale-down` can follow `contain` or `cover`	`fill`	CSS Images 4 §5.1
`object-position`	Sets where an image goes in its box when it does not fill it.	One to four values: keywords, lengths and percentages, as in `background-position`	`50% 50%`	CSS Images 3 §4.6
`image-orientation`	Chooses whether an image is turned as its file says, and can turn or mirror it.	`from-image`, none, or an angle and `flip`	`from-image`	CSS Images 3 §5.1
`image-rendering`	Chooses how the pixels of an image are drawn when it is shown larger than its size: smoothed, with sharp edges, or enlarged by whole multiples of its pixels.	`auto`, `smooth`, `high-quality`, `pixelated` or `crisp-edges`	`auto`	CSS Images 3 §5.2

Name	Description	Values	Default	Specification
`image-animation`	Chooses whether an animated image plays. A PDF does not move, so it has no effect.	`normal`, `paused`, `stopped` or `running`	`normal`	CSS Image Animation 1

SVG

With SVG you draw shapes, curves, gradients and text. CSSMedia draws them as vector graphics in the PDF, so they stay sharp at any size. This chapter explains how to put a drawing in your document, how to style it and which parts of SVG you can use. The SVG reference at the end lists the properties and elements.

A drawing is a tree of elements, written in XML. It can be part of your own XML, or it can be a file that an element shows as an image. In both cases CSSMedia reads the drawing, applies the style sheets that reach it and writes the result as lines, curves and text in the PDF. Nothing becomes a bitmap, and the text of a drawing stays text.

SVG elements belong to a namespace, so every `svg` element declares it:

```
<svg xmlns="http://www.w3.org/2000/svg" width="120" height="60">
    <rect width="120" height="60" fill="#cfe3f5"/>
</svg>
```

A drawing has its own coordinate system. Its origin is the top left corner, the x axis goes to the right and the y axis goes down. One unit of this system, a *user unit*, is one CSS pixel. The `viewBox` attribute changes the scale, as Put a drawing in your document shows.

The element names are fixed by SVG. These are the elements that CSSMedia draws or uses:

Element	What it does
`svg`	Holds a drawing. Inside another drawing, it makes a nested drawing with its own size and scale.
`rect`, `circle`, `ellipse`, `line`, `polyline`, `polygon`, `path`	Draw shapes.
`g`, `defs`, `use`, `symbol`, `switch`	Group shapes, define shapes that are drawn later, and repeat them.
`linearGradient`, `radialGradient`, `stop`, `pattern`	Define the paints that fill and outline shapes.
`marker`	Defines a mark that is drawn at the vertices of a shape.
`clipPath`, `mask`	Define the area where another element shows.
`text`, `tspan`, `textPath`	Write text, in a line, along a path or inside a shape.
`image`	Shows a PNG, JPEG or WebP image, or another SVG file.
`a`	Makes the elements inside it a link.
`style`	Holds a style sheet for the drawing.

The elements `title`, `desc` and `metadata` describe a drawing and are not drawn.

The elements of SVG that are for screens, such as `animate` and `script`, have no effect in a PDF.

A drawing uses the same CSS as the rest of your document. Properties such as `fill` and `stroke` are CSS properties, and the style sheet of your document reaches the elements of a drawing that is part of your XML. The topics of this chapter follow the work of making a drawing:

- Put a drawing in your document
- Style a drawing
- Draw shapes and reuse them
- Paint shapes
- Clip, mask and blend
- Write text in a drawing

Put a drawing in your document

You write a drawing in your XML, or you show a drawing from a file. Then you set its size and its scale, and you place it in the text.

Procedure

1. Write the drawing in your XML

   Add an `svg` element in the SVG namespace where the drawing must appear. CSSMedia draws it as a replaced element on the line of the text that surrounds it.

   ```
   <para>The mark <svg xmlns="http://www.w3.org/2000/svg" width="24" height="24">
       <circle cx="12" cy="12" r="10" fill="#cfe3f5" stroke="#1d4f7a" stroke-
   width="2"/>
   </svg> stays on its line.</para>
   ```

   You can declare the namespace with a prefix instead, as in `<s:svg xmlns:s="http://www.w3.org/2000/svg">`. An `svg` element in another namespace is an ordinary element of your XML, and CSSMedia does not draw it.

2. Set the size

   The `width` and `height` attributes of the `svg` element give the size of the drawing. A number is a length in pixels, and you can use units, such as `5cm` or `1in`. A rule of your style sheet with `width` or `height` wins over the attributes:

   ```
   svg {
       width: 4cm;
       height: 2cm;
   }
   ```

   A drawing without a size is 300 pixels wide and 150 pixels high. When it has a `viewBox` and you set only the width or only the height, CSSMedia takes the other size from the proportion of the `viewBox`.

3. Scale the drawing with a viewBox

   The `viewBox` attribute holds four numbers: the x and y of the top left corner of the area you draw in, and its width and height. CSSMedia scales that area to fill the size of the drawing, so you can draw with the numbers that suit the picture, whatever size it has on the page.

When the area and the drawing have different proportions, `preserveAspectRatio` decides what happens. It has an alignment, from `xMinYMin` to `xMaxYMax`, and one of these words:

- `meet` is the default. The whole area is visible, and part of the drawing stays empty.
- `slice` fills the drawing, and the parts of the area outside it are cut.

The value none stretches the area to the size of the drawing and ignores the proportions.

The values of preserveAspectRatio



preserveAspectRatio with a square viewBox in a wide box

```
<svg xmlns="http://www.w3.org/2000/svg" width="100" height="60"
     viewBox="0 0 50 50" preserveAspectRatio="xMidYMid slice">
    <circle cx="25" cy="25" r="25" fill="#cfe3f5" stroke="#1d4f7a"/>
</svg>
```

6. Place the drawing

   An `svg` element is inline, like a word. Set `display: block` to put the drawing on a line of its own. A drawing has margins, padding and borders like any other box:

   ```
   svg {
       display: block;
       margin: 6pt auto;
       border: 0.5pt solid gray;
   }
   ```

7. Show a drawing from a file

   Give an element the file as its `content`, as Place an image explains for every image. The size of the file is the one in its `svg` element, and you can set another size with `width` and `height`:

   ```
   chart {
       display: block;
       content: attr(src url);
       width: 8cm;
   }
   ```

   A file is a document of its own. It has its own styles: `style` elements, `style` attributes and style sheets linked with an `xml-stylesheet` processing instruction. The style sheet of your document does not reach it.

   If CSSMedia cannot read a file, or the file is not an SVG document, it does not draw it and names the file in the messages at the end of the run. An image that contains itself is reported in the same way.

Inside a drawing, the `image` element shows a file with the `href` attribute, and it can be a PNG, JPEG or WebP image, or another SVG file. Its `x`, `y`, `width` and `height` give the area it fills, and `preserveAspectRatio` fits the image in that area as it does for a `viewBox`.

Style a drawing

You can style the elements of a drawing with attributes, with rules in the drawing and, when the drawing is part of your XML, with the style sheet of your document.

Procedure

1. Use presentation attributes for simple drawings

   Most properties of a drawing can be written as attributes of the element, such as `fill`, `stroke`, `stroke-width` and `font-size`. These are presentation attributes. A presentation attribute is the weakest style an element can have: any rule overrides it.

   ```
   <rect width="60" height="40" fill="#cfe3f5" stroke="#1d4f7a"/>
   ```

2. Use rules for the styles you repeat

   A drawing follows the rules of CSS: selectors, specificity, inheritance and the order of the cascade. These sources of style can reach an element:

   - The presentation attributes of the element. They are the weakest.
   - The rules of the style sheet of your document, when the drawing is part of your XML.
   - The rules in a `style` element inside the drawing. They come after the rules of your document.
   - The `style` attribute of the element. It wins over the rules.

   The rules follow the usual order: the more specific selector wins, and the later rule wins when the specificity is the same. A declaration marked `!important` wins over the rest.

   ```
   <svg xmlns="http://www.w3.org/2000/svg" width="100" height="60">
       <style>
           .warning { fill: #f2d4d4; stroke: #8a2a2a; }
       </style>
       <rect class="warning" width="100" height="60"/>
   </svg>
   ```

   A type selector without a namespace matches the elements of any namespace. When your XML has elements named like those of SVG, such as `text`, use a namespace in the selector:

   ```
   @namespace svg url("http://www.w3.org/2000/svg");

   svg|text {
       font-size: 10pt;
   }
   ```

7. Let the elements inherit

   The properties of paint and text are inherited. Set `fill` or `stroke` on a `g` element and every shape inside it uses them. A drawing in your XML also inherits the properties of the

elements around it, such as `font-family` and `color`. The keyword `currentColor` is the value of `color`, so a drawing can follow the color of the text:

```
<svg xmlns="http://www.w3.org/2000/svg" width="16" height="16">
    <circle cx="8" cy="8" r="7" fill="currentColor"/>
</svg>
```

Custom properties work too: define `--accent` on the `svg` element and write `fill: var(--accent)` in a rule.

8. Set the geometry with CSS

   The geometry of the shapes is also a set of CSS properties: `x`, `y`, `width` and `height` for rectangles, `cx`, `cy` and `r` for circles, `rx` and `ry` for ellipses and rounded corners, and `d` for the data of a path. A rule that sets them has the same effect as the attributes, and it can use any length unit and `calc()`:

   ```
   circle.dot {
       cx: 50px;
       cy: 50px;
       r: 2em;
   }
   ```

   Percentages refer to the size of the drawing, or to the area of its `viewBox` when it has one.

9. Choose the fonts of the text

   The text of a drawing in your XML uses the fonts of your document, including the fonts that its `@font-face` rules define. A `style` element in the drawing can define more fonts for that drawing. A drawing in a file does not see the fonts of your document: it uses the standard fonts, the fonts you give to CSSMedia with the option `-f`, and its own `@font-face` rules.

10. Make parts of the drawing links

    Put the elements in an `a` element and write the destination in its `href` attribute. The PDF gets a link over the area of the elements, or over the characters of the text:

    ```
    <a href="https://www.example.com/">
        <rect width="60" height="20" fill="#cfe3f5"/>
    </a>
    ```

Draw shapes and reuse them

You draw with seven shape elements. You can group shapes, move and turn them, and define a shape once to draw it many times.

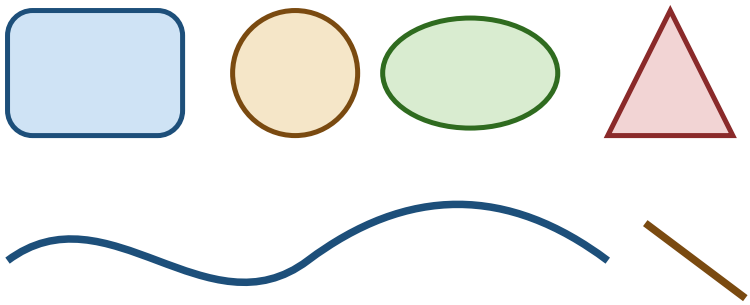
Procedure

1. Draw the basic shapes

   Each shape element has attributes for its position and size. All the numbers are in user units, which are CSS pixels unless you change the scale with a `viewBox`.

Element	Attributes
`rect`	`x` and `y` for the top left corner, `width` and `height`, and `rx` and `ry` for rounded corners. When you write only one radius, the other one has the same value.
`circle`	`cx` and `cy` for the center, and `r` for the radius.
`ellipse`	`cx` and `cy` for the center, and `rx` and `ry` for the two radii.
`line`	`x1` and `y1` for the start, and `x2` and `y2` for the end. A line has no inside, so it needs a `stroke` to be visible.
`polyline` and `polygon`	`points`, a list of x and y pairs. A `polygon` closes the shape with a line from the last point to the first one.
`path`	`d`, the path data.

Shapes and paths



```
<svg xmlns="http://www.w3.org/2000/svg" width="320" height="140" viewBox="0 0
320 140">
    <rect x="10" y="10" width="70" height="50" rx="10" fill="#cfe3f5"
stroke="#1d4f7a" stroke-width="2"/>
    <circle cx="125" cy="35" r="25" fill="#f5e6c8" stroke="#7a4a10" stroke-
width="2"/>
    <ellipse cx="195" cy="35" rx="35" ry="22" fill="#d9ecd0" stroke="#2f6b1f"
stroke-width="2"/>
    <polygon points="250,60 275,10 300,60" fill="#f2d4d4" stroke="#8a2a2a"
stroke-width="2"/>
    <path d="M10 110 C 50 80, 90 140, 130 110 S 210 80, 250 110" fill="none"
stroke="#1d4f7a" stroke-width="3"/>
    <line x1="265" y1="95" x2="305" y2="125" stroke="#7a4a10" stroke-
width="3"/>
</svg>
```

2. Write the data of a path

   The `d` attribute is a list of commands, each one with its numbers. A command in capital letters uses absolute coordinates. The same command in lowercase letters uses coordinates relative to the current point:

Command	Numbers	What it does
`M`, `m`	x y	Starts a new part of the path at a point, without drawing.
`L`, `l`	x y	Draws a straight line to a point.
`H`, h, V, v	x, or y	Draws a horizontal or a vertical line.
`C`, c	x1 y1 x2 y2 x y	Draws a cubic Bézier curve with two control points.
`S`, s	x2 y2 x y	Draws a cubic curve that continues the previous curve smoothly.
`Q`, q	x1 y1 x y	Draws a quadratic Bézier curve with one control point.
`T`, t	x y	Draws a quadratic curve that continues the previous curve smoothly.
A, a	rx ry rotation large-arc sweep x y	Draws an arc of an ellipse. The two flags, 0 or 1, choose which of the four possible arcs is drawn.
Z, z	none	Closes the current part with a line to its start.

If the data has an error, CSSMedia draws the path up to the command before the error and ignores the rest.

3. Group shapes and move them

   A `g` element groups shapes. The properties you set on the group, such as `fill`, are inherited by the shapes inside it. The `transform` attribute moves, turns or scales the group and everything in it. It is a list of these functions:

   - `translate(x y)` moves the group.
   - `scale(x y)` enlarges or reduces it. With one number, the scale is the same in both directions.
   - `rotate(angle)` turns it by an angle in degrees, around the origin of the drawing. `rotate(angle x y)` turns it around the point x y.
   - `skewX(angle)` and `skewY(angle)` slant it.
   - `matrix(a b c d e f)` gives the six numbers of the transform directly.

```
<g transform="translate(40 20) rotate(15)">
    <rect width="60" height="30" fill="#cfe3f5"/>
</g>
```

   The CSS property `transform` does the same with CSS syntax: `transform: translate(40px, 20px) rotate(15deg)`. You can also use the properties `rotate`, `scale` and `translate` and the 3D functions such as `rotateY()` and `perspective()`. The origin of a transform in a drawing is the origin of the drawing. Set `transform-origin` to turn an element around another point:

```
.turned {
    transform: rotate(30deg);
    transform-origin: 40px 30px;
}
```

9. Define a shape once and draw it many times

   Elements inside `defs` are not drawn. Give them an `id`, then draw them with a `use` element. Its `href` names the shape, and its `x` and `y` move the copy. The copy takes its style from the `use` element where the shape does not set its own, so each copy can have another color:

   ```
   <defs>
       <rect id="cell" width="20" height="20"/>
   </defs>
   <use href="#cell" x="0" y="0" fill="#cfe3f5"/>
   <use href="#cell" x="30" y="0" fill="#f5e6c8"/>
   ```

   A `symbol` is a shape that has its own `viewBox`. It is never drawn by itself, and a `use` element that gives it a `width` and a `height` draws it scaled to that size. A `use` element can name an element of another drawing of your document too.

10. Draw the first alternative that applies

    A `switch` element draws only its first child that has neither the `systemLanguage` nor the `requiredExtensions` attribute. CSSMedia supports no extension and has no language preference, so the children with these attributes never apply. Put the fallback last, without them.

11. Nest drawings and hide elements

    An `svg` element inside a drawing makes a nested drawing. Its `x`, `y`, `width` and `height` place it, and its `viewBox` scales its content. What it draws outside its box is cut, unless you set `overflow: visible`.

    With `display: none` an element and its children are not drawn. With `visibility: hidden` an element is not painted, but its children can still be visible when they set `visibility: visible`.

Paint shapes

A shape has a fill, which paints its inside, and a stroke, which paints its outline. You choose their colors, their opacity, the look of the outline, and gradients, patterns and markers.

Procedure

1. Choose the fill and the stroke

   The `fill` property paints the inside of a shape. Its initial value is black. The `stroke` property paints the outline, and its initial value is none, so a shape has no outline until you give it a paint. A paint is a color, none, `currentColor` or a reference to a gradient or a pattern, written as `url(#id)`. After a reference you can write a color that is used when the reference names nothing.

   When a shape crosses itself, or has holes, `fill-rule` decides which areas are inside. With `nonzero`, the initial value, an inner outline that goes in the same direction as the outer one

is filled too, and one that goes in the opposite direction is a hole. With evenodd every second area is empty, whatever the directions are.

The properties `fill-opacity` and `stroke-opacity` make each paint transparent, with a number from 0 to 1. The property `opacity` makes the whole element transparent, fill, stroke and children together, so the overlaps do not show.

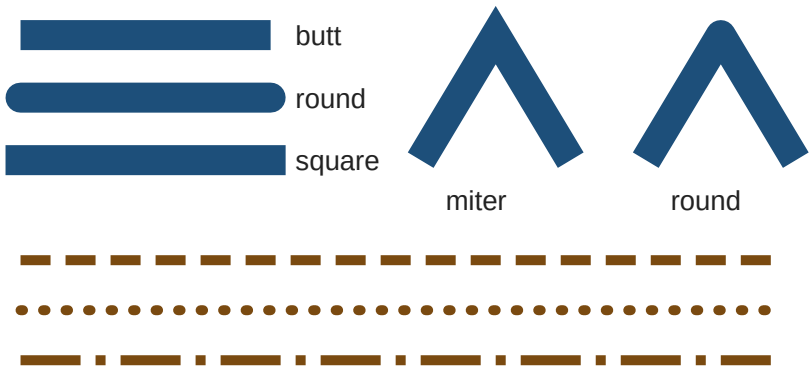
```
.alert {
    fill: #f2d4d4;
    stroke: #8a2a2a;
    stroke-width: 2;
    fill-opacity: 0.8;
}
```

2. Shape the outline

   These properties change how the outline looks:

   - `stroke-width` is the thickness. The initial value is 1.
   - `stroke-linecap` is the end of an open line: `butt` ends it flat at the point, `round` adds a half circle and `square` adds half the width.
   - `stroke-linejoin` is the corner: `miter`, `round` or `bevel`. A sharp miter turns into a bevel when it is longer than `stroke-miterlimit` times the width, which is 4 initially.
   - `stroke-dasharray` makes a dashed line. It is a list of lengths for dashes and gaps, in turn. A list with an odd number of lengths is repeated, so `10 5 3` means `10 5 3 10 5 3`. A negative length makes the list invalid.
   - `stroke-dashoffset` moves the start of the pattern along the line.

   Caps, joins and dashes



   The `paint-order` property decides what is painted first. The value `stroke` paints the outline under the fill, so a thick outline does not cover the inside of the shape. The value `fill stroke markers` is the initial one.

   The `pathLength` attribute of a shape tells CSSMedia to measure the shape as if it had that length, so lengths in `stroke-dasharray` scale with it. This line is 80 units long and counts as 100, so each dash of 10 is 8 units long:

```
<line x1="10" y1="20" x2="90" y2="20" stroke="black" pathLength="100" stroke-
dasharray="10 10"/>
```

The property `vector-effect: non-scaling-stroke` keeps the thickness of the outline when a transform scales the shape.

8. Fill with a gradient

   Define a gradient in `defs`, give it an `id` and use the reference as a paint. A `linearGradient` changes the color along a line, and a `radialGradient` changes it from a center to a circle. The `stop` elements inside give the colors:

   - `offset` is the position of the stop, from 0 to 1 or in percent.
   - `stop-color` is its color and `stop-opacity` its opacity.

   A linear gradient goes from `x1 y1` to `x2 y2`, which are 0% 0% and 100% 0% initially, so it goes from left to right. A radial gradient has the center `cx cy`, the radius `r` and the focus `fx fy`, the point where the first stop is painted. These attributes are fractions of the box of the shape. Set `gradientUnits="userSpaceOnUse"` to give them in user units, and use `gradientTransform` to turn or scale the gradient.

   The attribute `spreadMethod` decides what happens outside the line of the gradient: `pad` extends the last colors, `repeat` starts again and `reflect` goes back. The property `color-interpolation` set to `linearRGB` mixes the colors in a linear color space, which gives brighter middle colors.

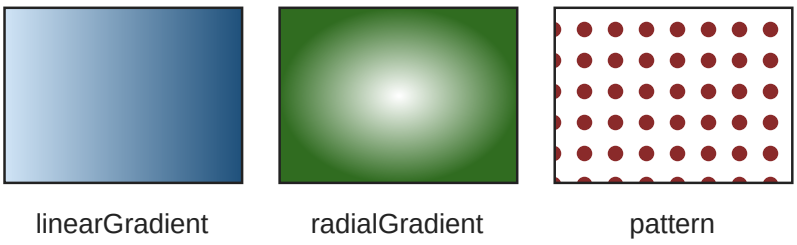
   A gradient can take its stops and attributes from another gradient, named with `href`. A gradient with one stop paints that color, and a gradient without stops paints nothing.

```
<defs>
    <linearGradient id="fade">
        <stop offset="0" stop-color="#cfe3f5"/>
        <stop offset="1" stop-color="#1d4f7a"/>
    </linearGradient>
</defs>
<rect width="95" height="70" fill="url(#fade)"/>
```

11. Fill with a pattern

    A `pattern` repeats a small drawing, the tile, to fill a shape. The tile is at `x y` and has the size `width` by `height`. These numbers are fractions of the box of the shape, unless you set `patternUnits="userSpaceOnUse"`. The elements inside the pattern are the drawing of the tile. A `viewBox` scales it to the tile, and `patternTransform` turns or scales the whole pattern.

    A gradient, a radial gradient and a pattern



```
<defs>
    <pattern id="dots" width="12" height="12" patternUnits="userSpaceOnUse">
        <circle cx="6" cy="6" r="3" fill="#8a2a2a"/>
    </pattern>
</defs>
<rect width="95" height="70" fill="url(#dots)"/>
```

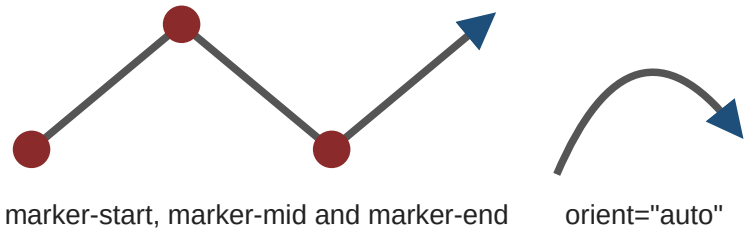
12. Mark the vertices

    A `marker` is a small drawing that CSSMedia repeats on the vertices of a line, a polyline, a polygon or a path. Define it in `defs` and name it in these properties of the shape: `marker-start` for the first vertex, `marker-mid` for the ones between and `marker-end` for the last. The property `marker` sets the three at once.

    These attributes of the `marker` element control the mark:

    - `markerWidth` and `markerHeight` give its size, 3 by 3 initially. A `viewBox` scales the drawing to that size.
    - `refX` and `refY` give the point of the mark that goes on the vertex.
    - `orient` is the direction. With `auto` the mark follows the direction of the line, and with an angle it is fixed.
    - `markerUnits` is `strokeWidth` initially, so the mark is scaled by the thickness of the line. With `userSpaceOnUse` it has the size of its numbers.

    Markers




```
<defs>
    <marker id="arrow" viewBox="0 0 10 10" markerWidth="5" markerHeight="5"
refX="5" refY="5" orient="auto">
        <path d="M0 0 L10 5 L0 10 Z" fill="#1d4f7a"/>
    </marker>
</defs>
<path d="M230 80 Q 260 10 300 60" fill="none" stroke="#555555" stroke-
width="3" marker-end="url(#arrow)"/>
```

Clip, mask and blend

You can show only a part of an element, fade it with a mask, and choose how it mixes with what is behind it.

Procedure

1. Clip an element

A clip cuts away the parts of an element outside an area. Define the area with a `clipPath` element, and name it in the `clip-path` property of the element. The shapes inside the `clipPath` make the area: the parts of the element that are under them stay visible. The `clip-rule` property of a shape works as `fill-rule` does for the area it makes.

By default the numbers inside a `clipPath` are user units. With `clipPathUnits="objectBoundingBox"` they are fractions of the box of the clipped element, so the same clip fits elements of any size. A `clip-path` that names something that is not a `clipPath` clips nothing.

The `clip-path` property accepts basic shapes too: `circle()`, `ellipse()`, `inset()`, `polygon()` and `path()`. A word before or after the shape chooses the box the shape is measured in: `fill-box` is the box of the shape without its stroke, `stroke-box` includes the stroke, and `view-box` is the whole drawing.

```
<defs>
    <clipPath id="disc">
        <circle cx="55" cy="45" r="32"/>
    </clipPath>
</defs>
<rect x="10" y="10" width="90" height="70" fill="#cfe3f5" clip-
path="url(#disc)"/>
```

```
.badge {
    clip-path: circle(40%);
}
```

2. Fade an element with a mask

   A mask makes parts of an element transparent. Define it with a `mask` element, and name it in the `mask` property or attribute of the element. The children of the `mask` are drawn, and where they are light the element shows, and where they are dark it is transparent. The mask uses the luminance of the colors, with their opacity. With `mask-type: alpha` on the `mask` element, the mask uses only the opacity of what is drawn, and the colors do not matter.

   A mask has a region, given by `x`, `y`, `width` and `height`, outside which the element is not shown. It is the box of the masked element made 10% larger on every side, as the initial values `-10%` and `120%` say. The attribute `maskUnits="userSpaceOnUse"` gives them in user units. The attribute `maskContentUnits="objectBoundingBox"` makes the numbers of the children fractions of the box of the element. A mask that names nothing makes the element invisible.

```
<defs>
    <linearGradient id="ramp">
        <stop offset="0" stop-color="#ffffff"/>
        <stop offset="1" stop-color="#000000"/>
    </linearGradient>
    <mask id="fade-out">
        <rect x="120" y="10" width="95" height="70" fill="url(#ramp)"/>
    </mask>
</defs>
<rect x="120" y="10" width="95" height="70" fill="#1d4f7a" mask="url(#fade-
out)"/>
```

3. Blend an element with what is behind it

   The `mix-blend-mode` property chooses how the colors of an element mix with the colors under it. The initial value is `normal`. The values such as `multiply`, `screen`, `overlay`, `darken` and `difference` are the blend modes of the PDF format. The property `isolation: isolate` on a group stops its children from blending with what is outside the group.

   A clip, a mask and a blend mode




```
<circle cx="260" cy="45" r="28" fill="#6fc3e8"/>
<circle cx="285" cy="45" r="28" fill="#f2d649" style="mix-blend-mode:
multiply"/>
```

   A group with an `opacity` below 1, a clip or a mask is isolated too.

Write text in a drawing

The text of a drawing is text in the PDF, with the fonts you choose. You can place it character by character, set it along a path, wrap it to a width or pour it into a shape.

Procedure

1. Write a line of text

   A `text` element writes a line. Its `x` and `y` attributes give the point where the line starts. The `y` is the baseline, the line the letters stand on, so letters go above it. The properties `font-family`, `font-size`, `font-weight` and `font-style`, and the other font properties of Font selection and sizing, work as they do in the rest of the document. The text is painted with `fill` and `stroke`, like a shape.

   The property `text-anchor` chooses which part of the line is at the point: `start`, the initial value, `middle` or end. Use `middle` to center a label on a point.

Spaces and line feeds in the text collapse, as they do in the rest of your document. Set `xml:space="preserve"`, or `white-space: pre`, to keep them.

```
<text x="10" y="30" font-size="20" fill="#222222">Plain <tspan fill="#8a2a2a"
font-weight="bold">text</tspan> with a <tspan dy="-6" font-size="14"
fill="#1d4f7a">raised</tspan><tspan dy="6"> word</tspan></text>
```

2. Change and move parts of a line

   A `tspan` element is a part of the text with its own style or position. It can have all the attributes of `text`. These attributes place the characters:

   - `x` and `y` set the position of characters. A list of numbers, such as `x="10 30 50"`, sets one position for each of the first characters. The characters after the last number follow the previous one.
   - `dx` and `dy` move a character and the ones after it, by the lengths they give. They can be lists too.
   - `rotate` turns each character, by the angles of its list in degrees. The last angle is used for the rest of the characters.

   The property `baseline-shift` raises or lowers a part of the line, with the values `super` and `sub` or a length. The property `dominant-baseline` chooses the baseline of the text that is placed at the `y`, such as `middle` or `hanging`.

   The properties `letter-spacing`, `word-spacing` and `text-decoration` work as in the rest of your document. The attribute `textLength` gives the length that the text must have. The attribute `lengthAdjust` chooses how CSSMedia reaches it: `spacing`, the initial value, changes the space between characters, and `spacingAndGlyphs` stretches the characters too.

6. Write along a path

   Put a `textPath` element inside a `text` element, and name a shape in its `href`. The text follows the outline of the shape. The attribute `startOffset` moves the start along the path, with a length or a percentage of the length of the path. With `side="right"` the text goes on the right side of the path, in the opposite direction.

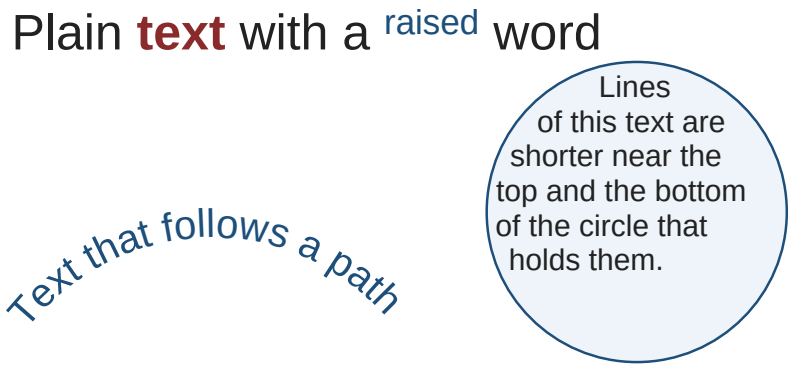
```
<defs>
    <path id="curve" d="M10 150 C 50 90, 130 90, 170 150"/>
</defs>
<text font-size="16" fill="#1d4f7a"><textPath href="#curve"
startOffset="10">Text that follows a path</textPath></text>
```

7. Wrap text to a width or into a shape

   With the property `inline-size` on a `text` element, the text wraps to lines of that length. The line breaks follow the rules of the languages of the text. The property `text-anchor` aligns the lines in the length.

   With the property `shape-inside` the text fills a shape. Each line is as long as the shape is wide at that height, and the text goes around the holes. The value is a basic shape, such as `circle()`, `ellipse()`, `polygon()` or `inset()`, a reference to a shape of the drawing, such as `url(#id)`, or the URL of an image, whose opaque pixels are the area. In a shape, `text-align` aligns the lines, and `text-align: justify` makes them fill the width.

A line, text on a path and text in a shape




```
<circle cx="260" cy="95" r="60" fill="#eef4fa" stroke="#1d4f7a"/>
<text x="205" y="60" font-size="12" fill="#222222" style="shape-inside:
circle(58px at 260px 95px)">Lines of this text are shorter near the top and
the bottom of the circle that holds them.</text>
```

8. Write vertically or from right to left

   The property `writing-mode` set to `vertical-rl` or `vertical-lr` writes the text in vertical lines, as in the languages of East Asia, and `text-orientation` chooses how its characters are turned. The property `direction: rtl` and the Unicode bidirectional algorithm write text from right to left where the language needs it.

SVG reference

This reference lists the elements of SVG that CSSMedia draws and the properties that style them.

SVG elements

Element	Description	Attributes
`svg`	A drawing, or a nested drawing with its own size and scale.	`xmlns`, `x`, `y`, `width`, `height`, `viewBox`, `preserveAspectRatio`
`g`	Groups elements, so that they share styles and a transform.	`transform`
`defs`	Holds elements that are defined to be used later and are not drawn.	None
`use`	Draws a copy of another element.	`href`, `x`, `y`, `width`, `height`

Element	Description	Attributes
`symbol`	A shape with its own `viewBox`, drawn by a `use` element.	`viewBox`, `preserveAspectRatio`, `refX`, `refY`
`switch`	Draws its first child that has no conditional attributes.	`systemLanguage`, `requiredExtensions` on the children
`rect`	A rectangle, with optional rounded corners.	`x`, `y`, `width`, `height`, `rx`, `ry`
`circle`	A circle.	`cx`, `cy`, `r`
`ellipse`	An ellipse.	`cx`, `cy`, `rx`, `ry`
`line`	A straight line.	`x1`, `y1`, `x2`, `y2`
`polyline`	An open shape made of straight lines.	`points`
`polygon`	A closed shape made of straight lines.	`points`
`path`	A shape made of lines, curves and arcs.	`d`, `pathLength`
`linearGradient`	A paint that changes color along a line.	`x1`, `y1`, `x2`, `y2`, `gradientUnits`, `gradientTransform`, `spreadMethod`, `href`
`radialGradient`	A paint that changes color from a center to a circle.	`cx`, `cy`, `r`, `fx`, `fy`, `gradientUnits`, `gradientTransform`, `spreadMethod`, `href`
`stop`	A color of a gradient, at a position.	`offset`
`pattern`	A paint that repeats a small drawing.	`x`, `y`, `width`, `height`, `patternUnits`, `patternContentUnits`, `patternTransform`, `viewBox`, `preserveAspectRatio`, `href`
`marker`	A mark that is drawn on the vertices of a shape.	`markerWidth`, `markerHeight`, `refX`, `refY`, `orient`, `markerUnits`, `viewBox`, `preserveAspectRatio`
`clipPath`	An area that clips other elements.	`clipPathUnits`
`mask`	A drawing whose light or opaque parts show the masked element.	`x`, `y`, `width`, `height`, `maskUnits`, `maskContentUnits`

Element	Description	Attributes
`text`	Text, in a line, wrapped to a length or inside a shape.	`x`, `y`, `dx`, `dy`, `rotate`, `textLength`, `lengthAdjust`
`tspan`	A part of a text with its own style or position.	`x`, `y`, `dx`, `dy`, `rotate`, `textLength`, `lengthAdjust`
`textPath`	Text that follows the outline of a shape.	`href`, `startOffset`, `side`
`image`	A PNG, JPEG, WebP or SVG file.	`href`, `x`, `y`, `width`, `height`, `preserveAspectRatio`
`a`	A link around the elements or the text it holds.	`href`
`style`	A style sheet for the drawing.	`type`, `media`

Geometry properties

Name	Description	Values	Default	Specification
`x`, `y`	Set the position of a `rect`, `image`, `svg`, `symbol` or `use` element.	A length or a percentage	`0`	SVG 2
`cx`, `cy`	Set the center of a `circle` or an `ellipse`.	A length or a percentage	`0`	SVG 2
`r`	Sets the radius of a `circle`.	A length or a percentage	`0`	SVG 2
`rx`, `ry`	Set the radii of an `ellipse`, or the rounded corners of a `rect`. With `auto` the radius takes the value of the other one.	A length, a percentage or `auto`	`auto`	SVG 2
`d`	Sets the data of a `path`.	none or a string of path data	none	SVG 2
`path-length`	Sets the length that a shape counts as having, for the lengths of its dashes.	none or a number	none	SVG 2

Paint properties

Name	Description	Values	Default	Specification
`fill`	Sets the paint of the inside of a shape or of the characters of a text.	A color, none, `currentColor` or `url(#id)`, optionally followed by a color that is used when the reference names nothing	`black`	SVG 2
`fill-opacity`	Sets the opacity of the fill.	A number from 0 to 1, or a percentage	`1`	SVG 2
`fill-rule`	Chooses which areas of a shape are inside it.	`nonzero` or `evenodd`	`nonzero`	SVG 2
`stroke`	Sets the paint of the outline of a shape or of a text.	As `fill`	`none`	SVG 2
`stroke-width`	Sets the thickness of the outline.	A length, a percentage or a number	`1`	SVG 2
`stroke-opacity`	Sets the opacity of the outline.	A number from 0 to 1, or a percentage	`1`	SVG 2
`stroke-linecap`	Sets the shape of the ends of an open line.	`butt`, `round` or `square`	`butt`	SVG 2
`stroke-linejoin`	Sets the shape of the corners of the outline.	`miter`, `round` or `bevel`	`miter`	SVG 2
`stroke-miterlimit`	Sets the length at which a sharp corner turns into a bevel, as a multiple of the thickness.	A number of 1 or more	`4`	SVG 2
`stroke-dasharray`	Makes the outline dashed, with a list of lengths for dashes and gaps.	none or a list of lengths, percentages or numbers	`none`	SVG 2
`stroke-dashoffset`	Moves the start of the dash pattern along the outline.	A length, a percentage or a number	`0`	SVG 2

Name	Description	Values	Default	Specification
`paint-order`	Chooses the order in which the fill, the stroke and the markers are painted.	`normal`, or `fill`, `stroke` and `markers` in the order you want	`normal`	SVG 2
`vector-effect`	With `non-scaling-stroke` the outline keeps its thickness when a transform scales the shape.	none or `non-scaling-stroke`	none	SVG 2
`color-interpolation`	Chooses the color space in which the colors of a gradient mix.	`auto`, `sRGB` or `linearRGB`	`sRGB`	SVG 2
`stop-color`	Sets the color of a `stop` of a gradient.	A color	`black`	SVG 2
`stop-opacity`	Sets the opacity of a `stop` of a gradient.	A number from 0 to 1, or a percentage	`1`	SVG 2
`marker`	Sets `marker-start`, `marker-mid` and `marker-end` at once.	none or `url(#id)`	none	SVG 2
`marker-start`, `marker-mid`, `marker-end`	Name the marker that is drawn on the first vertex, on the vertices between, and on the last vertex of a shape.	none or `url(#id)`	none	SVG 2

Clipping, masking and transform properties

Name	Description	Values	Default	Specification
`clip-path`	Shows only the part of an element inside an area.	none, `url(#id)` of a `clipPath`, or a basic shape, with `fill-box`, `stroke-box` or `view-box`	none	CSS Masking 1 §5.1

Name	Description	Values	Default	Specification
`clip-rule`	Chooses which areas of a shape inside a `clipPath` are inside it.	`nonzero` or `evenodd`	`nonzero`	CSS Masking 1 §6.2
`mask`	Makes parts of an element transparent with a `mask` element.	none or `url(#id)` of a `mask` element	none	CSS Masking 1 §7.9
`mask-type`	Chooses whether a `mask` element uses the luminance or the opacity of what it holds.	`luminance` or `alpha`	`luminance`	CSS Masking 1 §9.2
`opacity`	Makes an element and its children transparent as a group.	A number from 0 to 1, or a percentage	`1`	CSS Color 4 §3.3
`transform`	Moves, turns, scales or skews an element and its children.	none or a list of transform functions, 2D or 3D	none	CSS Transforms 1 §4
`transform-origin`	Sets the point that `transform` turns and scales around. In a drawing it is the origin of the drawing.	One to three lengths, percentages or keywords	`0 0`	CSS Transforms 1 §5
`translate`, `rotate`, `scale`	Move, turn or scale an element, each in a property of its own.	none or the arguments of the function of the same name	none	CSS Transforms 2 §5

Blending of whole boxes

Name	Description	Values	Default	Specification
`mix-blend-mode`	Sets how a box blends with what is behind it.	The values of `background-blend-mode`	`normal`	Compositing 1 §3.4.1
`isolation`	Chooses whether a box makes a group of its own for blending.	`auto` or `isolate`	`auto`	Compositing 1 §3.4.2

Text properties of a drawing

Name	Description	Values	Default	Specification
`text-anchor`	Chooses which part of a line of text is at its x position.	`start`, `middle` or `end`	`start`	SVG 2
`dominant-baseline`	Chooses the baseline of the text that is placed at its y position.	`auto`, `alphabetic`, `middle`, `central`, `hanging`, `ideographic`, `mathematical`, `text-top` or `text-bottom`	`auto`	SVG 2
`baseline-shift`	Raises or lowers a part of a line of text.	`super`, `sub`, a length or a percentage	`0`	SVG 2
`inline-size`	On a `text` element, sets the length of its lines, and the text wraps to it.	`auto`, a length or a percentage	`auto`	SVG 2
`shape-inside`	On a `text` element, pours the text into a shape.	`auto`, a basic shape, `url(#id)` of a shape, or the URL of an image	`auto`	CSS Shapes 2

MathML

Complete reference

Look up every selector, property, value and at-rule by subject.

This chapter is a complete reference for CSSMedia. Each table links to the CSS specification that defines its entries. The chapters of this guide end with a shorter reference that lists the entries they explain. The complete references are grouped by subject:

- Selector reference
- Cascade reference
- Margins, padding and borders
- Box sizes
- Backgrounds
- Display and positioning
- Tables
- Text and fonts
- Languages and hyphenation
- Page reference
- Links and generated content reference
- Named strings
- Links
- Footnotes
- Table of contents
- Images
- SVG

Selector reference

These tables list every selector that CSSMedia accepts. The `In a PDF` column says whether the selector matches elements while CSSMedia generates a PDF.

Basic selectors

Basic selectors match an element by its name, ID or attributes. See Basic selectors and namespaces.

Name	Description	Values	In a PDF	Specification
E	Type selector. Matches elements named E.	An element name.	Matches	Selectors 4 §5.1
*	Universal selector. Matches any element.		Matches	Selectors 4 §5.2

Name	Description	Values	In a PDF	Specification
`#id`	ID selector. Matches the element whose `id` or `xml:id` attribute has this value.	An identifier.	Matches	Selectors 4 §6.7
`.name`	Class selector. Matches inside SVG and MathML content, which define a `class` attribute.	An identifier.	Matches in SVG and MathML only	Selectors 4 §6.6
`[attr]`	Matches elements that have the attribute.	An attribute name, with an optional namespace prefix.	Matches	Selectors 4 §6
`[attr=value]`	Matches when the attribute value is exactly `value`.	A string or an identifier. Add `i` to ignore ASCII case or `s` to require it.	Matches	Selectors 4 §6.1
`[attr~=value]`	Matches when the value is one of the white-space separated words of the attribute.	A word without white space.	Matches	Selectors 4 §6.1
`[attr|=value]`	Matches when the attribute is `value` or starts with `value-`.	A string or an identifier.	Matches	Selectors 4 §6.1
`[attr^=value]`	Matches when the attribute starts with `value`.	A non-empty string.	Matches	Selectors 4 §6.2
`[attr$=value]`	Matches when the attribute ends with `value`.	A non-empty string.	Matches	Selectors 4 §6.2
`[attr*=value]`	Matches when the attribute contains `value`.	A non-empty string.	Matches	Selectors 4 §6.2

Name	Description	Values	In a PDF	Specification
`ns|E`, `*|E`, `|E`	Limits a type selector to a declared namespace, to any namespace, or to no namespace. The same prefixes work in attribute selectors.	A prefix declared with `@namespace`.	Matches	Selectors 4 §5.3, CSS Namespaces 3 §3
`&`	Nesting selector. Stands for the selector of the parent rule.		Matches	CSS Nesting 1

Combinators

A combinator relates two selectors. See Combinators.

Name	Description	Values	In a PDF	Specification
`A B`	Descendant combinator. Matches `B` inside `A` at any depth.	Two selectors separated by white space.	Matches	Selectors 4 §14.1
`A > B`	Child combinator. Matches `B` when its parent is `A`.	Two selectors.	Matches	Selectors 4 §14.2
`A + B`	Next-sibling combinator. Matches `B` when it directly follows `A`.	Two selectors.	Matches	Selectors 4 §14.3
`A ~ B`	Subsequent-sibling combinator. Matches `B` when `A` is an earlier sibling.	Two selectors.	Matches	Selectors 4 §14.4
`A || B`	Column combinator. Matches the cells `B` of the column that `A` describes.	Two selectors.	Matches	Selectors 5 §8.1
`A /attr/ B`	Reference combinator. Matches `B` when the attribute `attr` of `A` refers to its ID.	Two selectors and an attribute name between slashes.	Matches	Selectors 5 §9.1

Pseudo-classes

A pseudo-class tests a property of an element other than its name and attributes. See Pseudo-classes.

Name	Description	Values	In a PDF	Specification
`:root`	The root element of the document.		Matches	Selectors 4 §13.1
`:empty`	An element with no child elements and no text other than white space.		Matches	Selectors 4 §13.2
`:scope`	The scoping element. Without another scope, the root element.		Matches	Selectors 4 §8.4
`:first-child`, `:last-child`, `:only-child`	The first, last or only element among its siblings.		Matches	Selectors 4 §13.3
`:first-of-type`, `:last-of-type`, `:only-of-type`	The first, last or only element of its type among its siblings.		Matches	Selectors 4 §13.4
`:nth-child()`, `:nth-last-child()`	Elements at a position among their siblings, counted from the start or from the end.	odd, even or An+B, optionally followed by `of` and a selector list.	Matches	Selectors 4 §13.3
`:nth-of-type()`, `:nth-last-of-type()`	Elements at a position among the siblings of the same type.	odd, even or An+B.	Matches	Selectors 4 §13.4
`:nth-col()`, `:nth-last-col()`	Table cells at a position among the columns of their table.	odd, even or An+B.	Matches	Selectors 5 §8.2, §8.3
`:is()`, `:matches()`	An element that matches any selector of the list. `:matches()` is the old name of `:is()`.	A selector list. Invalid selectors are ignored.	Matches	Selectors 4 §4.2
`:where()`	Works as `:is()` and adds no specificity.	A selector list. Invalid selectors are ignored.	Matches	Selectors 4 §4.4

Name	Description	Values	In a PDF	Specification
`:not()`	An element that matches none of the selectors of the list.	A selector list. An invalid selector makes the whole selector invalid.	Matches	Selectors 4 §4.3
`:has()`	An element that has a related element matching the selector.	A relative selector list, such as `> figure`. It cannot contain another `:has()` or a pseudo-element.	Matches	Selectors 4 §4.5
`:lang()`	Elements in a language. The language comes from `xml:lang` or the `-cssmedia-lang` property.	One or more language ranges, as identifiers or strings.	Matches	Selectors 4 §7.2
`:dir()`	Elements by text direction, read from the text and the language, not from the `direction` property.	`ltr` or `rtl`.	Matches	Selectors 4 §7.1
`:link`, `:any-link`	Elements that are hyperlinks, because `-cssmedia-link` gives them a destination.		Matches	Selectors 4 §8.1, §8.2
`:local-link`	Hyperlinks that point to the same document. The optional number limits how many path segments are compared.	An optional non-negative integer in parentheses.	Matches	Selectors 5 §2.1

Name	Description	Values	In a PDF	Specification
`:read-only`, `:defined`	Every element is read-only and defined, because an XML element has no editable or undefined state.		Always matches	Selectors 4 §12.1.2, §5.4
`:hover`, `:active`, `:focus`, `:focus-visible`, `:focus-within`	User action states, such as pointing at an element or giving it keyboard focus.		Never matches	Selectors 4 §9
`:interest-source`, `:interest-target`	Source and target of a user's declared interest.		Never matches	Selectors 5 §3.1
`:visited`, `:target`	Links that were visited, and the element that a link points to. A PDF has no browsing history and no current target.		Never matches	Selectors 4 §8.2, §8.3
`:playing`, `:paused`, `:seeking`, `:buffering`, `:stalled`, `:muted`, `:volume-locked`	Media playback, loading and sound states.		Never matches	Selectors 4 §10
`:current`, `:current()`, `:past`, `:future`	Elements in relation to a timeline, such as speech or video playback.	A selector list for `:current()`.	Never matches	Selectors 5 §5
`:open`, `:popover-open`, `:modal`, `:fullscreen`, `:picture-in-picture`	Display states of interactive elements.		Never matches	Selectors 4 §11

Name	Description	Values	In a PDF	Specification
`:enabled`, `:disabled`, `:read-write`, `:placeholder-shown`, `:autofill`, `:default`, `:checked`, `:unchecked`, `:indeterminate`, `:blank`	Form control states. An XML element is never a form control.		Never matches	Selectors 4 §12, Selectors 5 §4.1
`:valid`, `:invalid`, `:in-range`, `:out-of-range`, `:required`, `:optional`, `:user-valid`, `:user-invalid`	Form validation states.		Never matches	Selectors 4 §12.3
`:state()`	A custom state that a document language exposes. XML has none.	An identifier.	Never matches	Selectors 5 §6
`:heading`, `:heading()`	Headings, as a document language defines them. CSSMedia does not give XML elements a heading role.	A list of levels for `:heading()`.	Never matches	Selectors 5 §7

Pseudo-elements

A pseudo-element selects a part of an element or generated content. See Pseudo-elements.

Name	Description	Values	In a PDF	Specification
`::before`, `::after`	Generated content before or after the content of an element. The `content` property gives it.		Matches	CSS Pseudo-Elements 4 §4.1
`::marker`	The marker of a list item. Also `::before::marker` and `::after::marker`.		Matches	CSS Pseudo-Elements 4 §4.2

Name	Description	Values	In a PDF	Specification
`::first-line`	The first formatted line of a block.		Matches	CSS Pseudo-Elements 4 §2.1
`::first-letter`	The first letter of a block, with the punctuation around it. `::first-letter::prefix` and `::first-letter::suffix` select that punctuation.		Matches	CSS Pseudo-Elements 4 §2.2
`::footnote-call`	The reference that replaces a footnote in the text. By default it is the value of the `footnote` counter, as a superscript.		Matches	CSS GCPM 3 §2.6
`::footnote-marker`	The marker at the start of a footnote body. By default it is the value of the `footnote` counter, followed by a period and a space.		Matches	CSS GCPM 3 §2.7
`::selection`, `::search-text`, `::target-text`, `::spelling-error`, `::grammar-error`, `::highlight()`	Highlighted ranges created by selection, search, navigation, spelling and grammar checking, or scripts.	A custom identifier for `::highlight()`.	No range exists	CSS Pseudo-Elements 4 §3.1
`::placeholder`, `::file-selector-button`, `::details-content`	Parts of form controls and disclosure widgets.		No such part exists	CSS Pseudo-Elements 4 §4.3, §5.1, §5.2

Name	Description	Values	In a PDF	Specification
`::backdrop`	The backdrop of an element shown in the top layer.		No such part exists	CSS Position 4 §3.2

Cascade reference

The `Default` column lists values used by CSSMedia's current PDF-oriented rule evaluation. Media-query width, height and ratio values use the A4 default page size.

Property and at-rule reference

Name	Description	Values	Default	Specification
`all`	Resets all properties except `direction` and `unicode-bidi`.	`initial` | `inherit` | `unset` | `revert` | `revert-layer`		CSS Cascade 5 §3.2
`@charset`	Declares the stylesheet's fallback character encoding.	`@charset "<label>";`		CSS Syntax 3 §8.3
`@import`	Loads another stylesheet, optionally conditioned on a layer, support test or media query.	`@import <url-or-string> [layer | layer(<layer-name>)]? [supports(<condition>)]? <media-query-list>?;`		CSS Cascade 5 §2
`@namespace`	Declares a namespace prefix for selectors.	`@namespace <namespace-prefix>? <string-or-url>;`		CSS Namespaces 3 §3
`@layer`	Declares layer order or groups rules in a cascade layer.	`@layer <layer-name>? { <stylesheet> }` or `@layer <layer-name-list>;`		CSS Cascade 5 §6.4

Name	Description	Values	Default	Specification
`@property`	Registers a custom property with syntax, inheritance and optional initial value.	`@property <custom-property-name> { <declaration-list> }`		CSS Properties and Values API 1 §3
`syntax`	Defines the grammar of a registered custom property.	`<string>`		CSS Properties and Values API 1 §3.1
`inherits`	Sets whether a registered custom property inherits by default.	`true` | `false`		CSS Properties and Values API 1 §3.2
`initial-value`	Sets the registered custom property's initial value.	`<declaration-value>`	Guaranteed-invalid for `syntax: "*"`; otherwise required	CSS Properties and Values API 1 §3.3
`@media`	Includes rules when a media query matches the PDF environment.	`@media <media-query-list> { <rule-list> }`		CSS Conditional 3 §5
`@supports`	Includes rules when CSSMedia accepts a declaration, selector or supported feature test.	`@supports <supports-condition> { <rule-list> }`		CSS Conditional 3 §6
`@supports-condition`	Defines a named group of support tests.	`@supports-condition <supports-condition-name> { <block-contents> }`		CSS Conditional 5 §8

Name	Description	Values	Default	Specification
`@container`	Includes rules when a selected ancestor container passes the query.	`@container <container-condition># { <block-contents> }`		CSS Conditional 5 §5.4
`@scope`	Limits the selectors in a rule list to a scope root and optional scope limit.	`@scope [(<scope-start>)]? [to (<scope-end>)]? { <rule-list> }`		CSS Cascade 6 §2.5
`@custom-media`	Gives a name to a media query list or boolean value.	`@custom-media <extension-name> [<media-query-list> | true | false];`		Media Queries 5 §10
`@when`	Includes rules when a generalized boolean condition is true.	`@when <boolean-condition> { <rule-list> }`		CSS Conditional 5 §3
`@else`	Supplies the next branch of a conditional chain.	`@else <boolean-condition>? { <rule-list> }`		CSS Conditional 5 §4
`@function`	Defines a custom CSS function.	`@function <name>(<parameters>) [returns <type>]? { <declaration-rule-list> }`		CSS Functions and Mixins 1 §2.1
`result`	Supplies a custom function's return value.	`<declaration-value>?`		CSS Functions and Mixins 1 §2.2

- ? means optional, | means choose one, # means one or more comma-separated items, || means one or more options in any order, and brackets group parts of the syntax.
- `<label>` is an encoding name.
- `<url-or-string>` and `<string-or-url>` each mean a URL or quoted string.
- `<namespace-prefix>` is the prefix bound to a namespace.

- `<layer-name>` names a cascade layer. `<layer-name-list>` is a comma-separated list of layer names.
- `<stylesheet>` is a CSS stylesheet.
- `<custom-property-name>` is a custom property name beginning with `--`.
- `<declaration-list>` is a list of declarations.
- `<block-contents>` is content inside a rule's braces, including declarations and nested rules.
- `<declaration-value>` is the value part of a declaration: one or more CSS tokens, such as `navy`, `calc(1em + 2px)` or `var(--size)`. It cannot contain a top-level semicolon or `!`, since those delimit declarations and `!important`. Semicolons and exclamation marks inside functions are not top-level. See CSS Syntax 3.
- `<string>` is a quoted CSS string.
- `<media-query-list>` is one or more comma-separated media queries.
- `<rule-list>` is a sequence of rules.
- `<condition>` and `<supports-condition>` are support conditions made from feature tests and boolean operators.
- `<supports-condition-name>` names a reusable support condition.
- `<container-condition>` optionally names a query container and gives its query.
- `<boolean-condition>` combines `media()` and `supports()` tests.
- `<scope-start>` and `<scope-end>` are selector lists that set the scope roots and limits of `@scope`.
- `<extension-name>` names a custom media query.
- `<name>` in the function syntax is a function name beginning with `--`.
- `<parameters>` is a comma-separated list of function parameters.
- `<type>` is the optional syntax expected for the function result.
- `<declaration-rule-list>` contains declarations and nested rules.
- `<number>` in the function example is a CSS number, which can be an integer or include a fractional part.

The `@charset` rule must be the first rule in a stylesheet to affect byte decoding. Use `@namespace` before ordinary rules, and place `@import` rules before namespace declarations and ordinary rules.

Function names start with `--`. Parameters can have a syntax type and a default value. Use the `result` descriptor for the returned value:

```
@function --double(--value <number>) returns <number> {
    result: calc(var(--value) * 2);
}
```

`@supports` declaration tests use parentheses. `selector()` tests one complex selector. `font-tech()`, `font-format()` and `at-rule()` test supported features. A named condition is tested by placing its name inside parentheses:

```
@supports (color: navy) and selector(chapter > title) {
    chapter > title { color: navy; }
}

@supports-condition --has-navy {
    color: navy;
}

@supports (--has-navy) {
    chapter { color: navy; }
}
```

`@when` conditions combine `media()` and `supports()` tests with `and`, `or` and `not`. Parentheses group mixed operators:

```
@when media(width >= 6in) and supports(color: navy) {
    chapter { color: navy; }
}
```

Use `@else` immediately after an `@when`, `@media`, `@supports` or `@container` branch. An unrelated rule ends the chain.

Media features

Media feature values describe the page or PDF environment used for conditional rule evaluation. They do not describe a screen.

Name	Description	Values	Default	Specification
`any-hover`	Whether any pointing device can hover.	`none` | `hover`	`none`	Media Queries 5
`any-pointer`	The accuracy of any available pointing device.	`none` | `coarse` | `fine`	`none`	Media Queries 5
`aspect-ratio`	The ratio of page width to page height.	`<ratio>`	A4 page ratio	Media Queries 5
`color`	Bits per color component in the output.	`<integer>`	`8`	Media Queries 5
`color-gamut`	The approximate output color gamut.	`srgb` | `p3` | `rec2020`	`srgb`	Media Queries 5
`color-index`	Number of entries in the output color lookup table.	`<integer>`	`0`	Media Queries 5

Name	Description	Values	Default	Specification
`device-aspect-ratio`	The ratio of output width to output height.	`<ratio>`	A4 page ratio	Media Queries 5
`device-height`	Output height.	`<length>`	A4 page height	Media Queries 5
`device-width`	Output width.	`<length>`	A4 page width	Media Queries 5
`display-mode`	The presentation mode.	`fullscreen` | `standalone` | `minimal-ui` | `browser` | `picture-in-picture`	Not set	Media Queries 5
`dynamic-range`	The output's dynamic range.	`standard` | `high`	`standard`	Media Queries 5
`environment-blending`	Whether output blends with the surrounding environment.	`opaque` | `additive` | `subtractive`	`opaque`	Media Queries 5
`forced-colors`	Whether the output forces a limited color palette.	`none` | `active`	`none`	Media Queries 5
`grid`	Whether the output is grid-based.	`<mq-boolean>` (`0` or `1`)	`0`	Media Queries 5
`height`	Page height.	`<length>`	A4 page height	Media Queries 5
`horizontal-viewport-segments`	Number of horizontal viewport segments.	`<integer>`	`1`	Media Queries 5
`hover`	Whether the primary pointing device can hover.	`none` | `hover`	`none`	Media Queries 5
`inverted-colors`	Whether the output inverts colors.	`none` | `inverted`	`none`	Media Queries 5

Name	Description	Values	Default	Specification
`monochrome`	Number of bits per pixel in a monochrome output.	`<integer>`	`0`	Media Queries 5
`nav-controls`	Availability of navigation controls.	`none` | `back`	`none`	Media Queries 5
`orientation`	Whether the page is portrait or landscape.	`portrait` | `landscape`	`portrait`	Media Queries 5
`overflow-block`	How the output handles block-axis overflow.	`none` | `scroll` | `paged`	`paged`	Media Queries 5
`overflow-inline`	How the output handles inline-axis overflow.	`none` | `scroll`	`none`	Media Queries 5
`pointer`	The accuracy of the primary pointing device.	`none` | `coarse` | `fine`	`none`	Media Queries 5
`prefers-color-scheme`	The preferred color scheme.	`light` | `dark`	`light`	Media Queries 5
`prefers-contrast`	The preferred contrast level.	`no-preference` | `less` | `more` | `custom`	`no-preference`	Media Queries 5
`prefers-reduced-data`	Whether the user prefers reduced data use.	`no-preference` | `reduce`	`no-preference`	Media Queries 5
`prefers-reduced-motion`	Whether the user prefers reduced motion.	`no-preference` | `reduce`	`no-preference`	Media Queries 5
`prefers-reduced-transparency`	Whether the user prefers reduced transparency.	`no-preference` | `reduce`	`no-preference`	Media Queries 5
`resolution`	Output pixel density.	`<resolution>` | `infinite`	`infinite`	Media Queries 5
`scan`	The output's scanning process.	`interlace` | `progressive`	Not set	Media Queries 5

Name	Description	Values	Default	Specification
`scripting`	Whether scripting is available.	`none` | `initial-only` | `enabled`	`none`	Media Queries 5
`update`	How frequently the output can update.	`none` | `slow` | `fast`	`none`	Media Queries 5
`vertical-viewport-segments`	Number of vertical viewport segments.	`<integer>`	`1`	Media Queries 5
`video-color-gamut`	The color gamut of the output video.	`srgb` | `p3` | `rec2020`	`srgb`	Media Queries 5
`video-dynamic-range`	The dynamic range of the output video.	`standard` | `high`	`standard`	Media Queries 5
`width`	Page width.	`<length>`	A4 page width	Media Queries 5

- `<ratio>` is a ratio of two numbers.
- `<integer>` is a whole number.
- `<length>` is a CSS length, such as `12pt` or `2em`.
- `<mq-boolean>` is an integer with the value `0` or `1`.
- `<resolution>` is a pixel density, such as `300dpi`.

Container size features

A size query in `@container` tests the size of the nearest ancestor that has `container-type`. An ancestor with `size` answers all these features, and an ancestor with `inline-size` answers only the features of the inline axis.

Name	Description	Values	Default	Specification
`width`	The width of the content box of the container.	A length, alone or in a comparison or a range	The size of the container	CSS Conditional 5 §6.1
`height`	The height of the content box of the container.	A length, alone or in a comparison or a range	The size of the container	CSS Conditional 5 §6.1

Name	Description	Values	Default	Specification
`inline-size`	The size of the content box of the container along the lines of text.	A length, alone or in a comparison or a range	The size of the container	CSS Conditional 5 §6.1
`block-size`	The size of the content box of the container across the lines of text.	A length, alone or in a comparison or a range	The size of the container	CSS Conditional 5 §6.1
`aspect-ratio`	The ratio of the width to the height of the container.	A ratio, alone or in a comparison or a range	The ratio of the container	CSS Conditional 5 §6.1
`orientation`	Whether the container is wider than high.	`landscape` | `portrait`	The orientation of the container	CSS Conditional 5 §6.1

Container style queries

A container style query uses `style()` to test a computed style or custom property on an ancestor. See Conditional rules.

Margins, padding and borders

This reference lists the properties that CSSMedia uses for the margins, padding, borders and rounded corners of a box.

Margins

Name	Description	Values	Default	Specification
`margin`	Sets the space outside the four sides of the box. One value applies to all sides. Two values set top and bottom, then left and right. Three set top, then left and right, then bottom. Four set top, right, bottom and left.	One to four lengths or percentages, or `auto`	`0`	CSS Box Model 4 §3.2
`margin-top`	Sets the space above the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1
`margin-right`	Sets the space at the right of the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1
`margin-bottom`	Sets the space below the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1

Name	Description	Values	Default	Specification
`margin-left`	Sets the space at the left of the box.	A length, a percentage or `auto`	`0`	CSS Box Model 4 §3.1
`margin-block`	Sets the margins at the start and the end of the block axis.	One or two lengths, percentages or `auto`	`0`	CSS Logical Properties 1 §4.2
`margin-block-start`	Sets the margin at the start of the block axis.	A length, a percentage or `auto`	`0`	CSS Logical Properties 1 §4.2
`margin-block-end`	Sets the margin at the end of the block axis.	A length, a percentage or `auto`	`0`	CSS Logical Properties 1 §4.2
`margin-inline`	Sets the margins at the start and the end of the inline axis.	One or two lengths, percentages or `auto`	`0`	CSS Logical Properties 1 §4.2
`margin-inline-start`	Sets the margin at the start of the inline axis.	A length, a percentage or `auto`	`0`	CSS Logical Properties 1 §4.2
`margin-inline-end`	Sets the margin at the end of the inline axis.	A length, a percentage or `auto`	`0`	CSS Logical Properties 1 §4.2
`margin-break`	Chooses what happens to the margins of a box where a page or column break touches them.	`auto`, `keep` or `discard`	`auto`	CSS Fragmentation 4 §5.2
`margin-trim`	Removes the margins of the children that touch the edges of the container.	`none`, `block`, `inline`, or one or more of `block-start`, `block-end`, `inline-start` and `inline-end`	`none`	CSS Box Model 4 §3.3

Padding

Name	Description	Values	Default	Specification
`padding`	Sets the space between the content and the border on the four sides. The values are read like those of `margin`.	One to four non-negative lengths or percentages	`0`	CSS Box Model 4 §4.2

Name	Description	Values	Default	Specification
`padding-top`	Sets the space between the content and the top border.	A non-negative length or percentage	`0`	CSS Box Model 4 §4.1
`padding-right`	Sets the space between the content and the right border.	A non-negative length or percentage	`0`	CSS Box Model 4 §4.1
`padding-bottom`	Sets the space between the content and the bottom border.	A non-negative length or percentage	`0`	CSS Box Model 4 §4.1
`padding-left`	Sets the space between the content and the left border.	A non-negative length or percentage	`0`	CSS Box Model 4 §4.1
`padding-block`	Sets the padding at the start and the end of the block axis.	One or two non-negative lengths or percentages	`0`	CSS Logical Properties 1 §4.4
`padding-block-start`	Sets the padding at the start of the block axis.	A non-negative length or percentage	`0`	CSS Logical Properties 1 §4.4
`padding-block-end`	Sets the padding at the end of the block axis.	A non-negative length or percentage	`0`	CSS Logical Properties 1 §4.4
`padding-inline`	Sets the padding at the start and the end of the inline axis.	One or two non-negative lengths or percentages	`0`	CSS Logical Properties 1 §4.4
`padding-inline-start`	Sets the padding at the start of the inline axis.	A non-negative length or percentage	`0`	CSS Logical Properties 1 §4.4
`padding-inline-end`	Sets the padding at the end of the inline axis.	A non-negative length or percentage	`0`	CSS Logical Properties 1 §4.4

Borders

Name	Description	Values	Default	Specification
`border`	Sets the width, style and color of the four borders. It also resets `border-image`.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4

Name	Description	Values	Default	Specification
`border-width`	Sets the width of the four borders. The values are read like those of `margin`.	One to four widths	`medium`	CSS Borders 4 §2.3
`border-style`	Sets the line style of the four borders. The values are read like those of `margin`.	One to four styles	`none`	CSS Borders 4 §2.2
`border-color`	Sets the color of the four borders. The values are read like those of `margin`.	One to four colors	`currentcolor`	CSS Borders 4 §2.1
`border-top`	Sets the width, style and color of the top border.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-right`	Sets the width, style and color of the right border.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-bottom`	Sets the width, style and color of the bottom border.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-left`	Sets the width, style and color of the left border.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-block`	Sets the width, style and color of the borders at the start and the end of the block axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-block-start`	Sets the width, style and color of the border at the start of the block axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-block-end`	Sets the width, style and color of the border at the end of the block axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-inline`	Sets the width, style and color of the borders at the start and the end of the inline axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4

Name	Description	Values	Default	Specification
`border-inline-start`	Sets the width, style and color of the border at the start of the inline axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-inline-end`	Sets the width, style and color of the border at the end of the inline axis.	A width, a style and a color, in any order	See the individual properties	CSS Borders 4 §2.4
`border-top-width`	Sets the width of the top border.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-right-width`	Sets the width of the right border.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-bottom-width`	Sets the width of the bottom border.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-left-width`	Sets the width of the left border.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-block-width`	Sets the width of the borders at the start and the end of the block axis.	One or two widths	`medium`	CSS Borders 4 §2.3
`border-block-start-width`	Sets the width of the border at the start of the block axis.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-block-end-width`	Sets the width of the border at the end of the block axis.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-inline-width`	Sets the width of the borders at the start and the end of the inline axis.	One or two widths	`medium`	CSS Borders 4 §2.3
`border-inline-start-width`	Sets the width of the border at the start of the inline axis.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3

Name	Description	Values	Default	Specification
`border-inline-end-width`	Sets the width of the border at the end of the inline axis.	`thin`, `medium`, `thick` or a non-negative length	`medium`	CSS Borders 4 §2.3
`border-top-style`	Sets the line style of the top border.	`none`, `hidden`, `dotted`, `dashed`, `solid`, `double`, `groove`, `ridge`, `inset` or `outset`	none	CSS Borders 4 §2.2
`border-right-style`	Sets the line style of the right border.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-bottom-style`	Sets the line style of the bottom border.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-left-style`	Sets the line style of the left border.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-block-style`	Sets the line style of the borders at the start and the end of the block axis.	One or two styles	none	CSS Borders 4 §2.2
`border-block-start-style`	Sets the line style of the border at the start of the block axis.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-block-end-style`	Sets the line style of the border at the end of the block axis.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-inline-style`	Sets the line style of the borders at the start and the end of the inline axis.	One or two styles	none	CSS Borders 4 §2.2
`border-inline-start-style`	Sets the line style of the border at the start of the inline axis.	The values of `border-top-style`	none	CSS Borders 4 §2.2
`border-inline-end-style`	Sets the line style of the border at the end of the inline axis.	The values of `border-top-style`	none	CSS Borders 4 §2.2

Name	Description	Values	Default	Specification
`border-top-color`	Sets the color of the top border.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-right-color`	Sets the color of the right border.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-bottom-color`	Sets the color of the bottom border.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-left-color`	Sets the color of the left border.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-block-color`	Sets the color of the borders at the start and the end of the block axis.	One or two colors	`currentcolor`	CSS Borders 4 §2.1
`border-block-start-color`	Sets the color of the border at the start of the block axis.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-block-end-color`	Sets the color of the border at the end of the block axis.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-inline-color`	Sets the color of the borders at the start and the end of the inline axis.	One or two colors	`currentcolor`	CSS Borders 4 §2.1
`border-inline-start-color`	Sets the color of the border at the start of the inline axis.	A color	`currentcolor`	CSS Borders 4 §2.1
`border-inline-end-color`	Sets the color of the border at the end of the inline axis.	A color	`currentcolor`	CSS Borders 4 §2.1

Rounded corners

Name	Description	Values	Default	Specification
`border-radius`	Sets the radius of the four corners. Values after a slash set the vertical radii.	One to four non-negative lengths or percentages, optionally followed by `/` and one to four more	`0`	CSS Borders 4 §3.5
`border-top-left-radius`	Sets the radius of the top left corner. One value is a circle and two values are an ellipse.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-top-right-radius`	Sets the radius of the top right corner.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-bottom-right-radius`	Sets the radius of the bottom right corner.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-bottom-left-radius`	Sets the radius of the bottom left corner.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-start-start-radius`	Sets the radius of the corner at the start of the block axis and the start of the inline axis.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-start-end-radius`	Sets the radius of the corner at the start of the block axis and the end of the inline axis.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-end-start-radius`	Sets the radius of the corner at the end of the block axis and the start of the inline axis.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5
`border-end-end-radius`	Sets the radius of the corner at the end of the block axis and the end of the inline axis.	One or two non-negative lengths or percentages	`0`	CSS Borders 4 §3.5

Name	Description	Values	Default	Specification
`border-top-radius`	Sets the radius of the two top corners.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-right-radius`	Sets the radius of the two right corners.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-bottom-radius`	Sets the radius of the two bottom corners.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-left-radius`	Sets the radius of the two left corners.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-block-start-radius`	Sets the radius of the two corners at the start of the block axis.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-block-end-radius`	Sets the radius of the two corners at the end of the block axis.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-inline-start-radius`	Sets the radius of the two corners at the start of the inline axis.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`border-inline-end-radius`	Sets the radius of the two corners at the end of the inline axis.	One or two non-negative lengths or percentages, optionally with a slash	`0`	CSS Borders 4 §3.5
`corner`	Sets the radius and the shape of the four corners.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-top-left`	Sets the radius and the shape of the top left corner.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-top-right`	Sets the radius and the shape of the top right corner.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9

Name	Description	Values	Default	Specification
`corner-bottom-right`	Sets the radius and the shape of the bottom right corner.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-bottom-left`	Sets the radius and the shape of the bottom left corner.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-start-start`	Sets the radius and the shape of the corner at the start of the block axis and the start of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-start-end`	Sets the radius and the shape of the corner at the start of the block axis and the end of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-end-start`	Sets the radius and the shape of the corner at the end of the block axis and the start of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-end-end`	Sets the radius and the shape of the corner at the end of the block axis and the end of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-top`	Sets the radius and the shape of the two top corners.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-right`	Sets the radius and the shape of the two right corners.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-bottom`	Sets the radius and the shape of the two bottom corners.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-left`	Sets the radius and the shape of the two left corners.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-block-start`	Sets the radius and the shape of the two corners at the start of the block axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9

Name	Description	Values	Default	Specification
`corner-block-end`	Sets the radius and the shape of the two corners at the end of the block axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-inline-start`	Sets the radius and the shape of the two corners at the start of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9
`corner-inline-end`	Sets the radius and the shape of the two corners at the end of the inline axis.	A radius and a shape	See the individual properties	CSS Borders 4 §3.9

Box sizes

This reference lists the properties that CSSMedia uses to size a box.

Preferred sizes

Name	Description	Values	Default	Specification
`box-sizing`	Chooses what `width` and `height` measure. `content-box` measures the content, and the padding and the border are added. `border-box` measures the box through the border.	`content-box` or `border-box`	`content-box`	CSS Sizing 3 §3.3
`width`	Sets the preferred width of the box.	`auto`, a non-negative length or percentage, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, `calc-size()` or `anchor-size()`	`auto`	CSS Sizing 3 §3.1
`height`	Sets the preferred height of the box.	The values of `width`	`auto`	CSS Sizing 3 §3.1
`inline-size`	Sets the preferred size along the inline axis, which is the width in horizontal text.	The values of `width`	`auto`	CSS Logical Properties 1 §4.1

Name	Description	Values	Default	Specification
`block-size`	Sets the preferred size along the block axis, which is the height in horizontal text.	The values of `width`	`auto`	CSS Logical Properties 1 §4.1
`size`	Sets `width` and `height`. One value applies to both.	One or two values of `width`	`auto`	CSS Sizing 4 §3.1

Minimum and maximum sizes

Name	Description	Values	Default	Specification
`min-width`	Sets the smallest width of the box.	The values of `width`	`auto`	CSS Sizing 3 §3.1
`min-height`	Sets the smallest height of the box.	The values of `width`	`auto`	CSS Sizing 3 §3.1
`min-inline-size`	Sets the smallest size along the inline axis.	The values of `width`	`auto`	CSS Logical Properties 1 §4.1
`min-block-size`	Sets the smallest size along the block axis.	The values of `width`	`auto`	CSS Logical Properties 1 §4.1
`min-size`	Sets `min-width` and `min-height`. One value applies to both.	One or two values of `min-width`	`auto`	CSS Sizing 4 §3.1
`max-width`	Sets the largest width of the box.	none, or the values of `width` except `auto`	none	CSS Sizing 3 §3.1
`max-height`	Sets the largest height of the box.	The values of `max-width`	none	CSS Sizing 3 §3.1
`max-inline-size`	Sets the largest size along the inline axis.	The values of `max-width`	none	CSS Logical Properties 1 §4.1
`max-block-size`	Sets the largest size along the block axis.	The values of `max-width`	none	CSS Logical Properties 1 §4.1
`max-size`	Sets `max-width` and `max-height`. One value applies to both.	One or two values of `max-width`	none	CSS Sizing 4 §3.1

Scaling

Name	Description	Values	Default	Specification
`zoom`	Scales all the lengths of an element, such as its font size, margins and width, by a factor. `2` and `200%` double them. The property is not inherited.	A number or a percentage of 0 or more	`1`	CSS Viewport 1 §4

Aspect ratio and intrinsic sizes

Name	Description	Values	Default	Specification
`aspect-ratio`	Gives the box a preferred ratio between its width and its height. The first number is the width and the second is the height. A single number is a ratio to 1.	`auto`, a non-negative number or two non-negative numbers separated by `/`, such as `16 / 9`. `auto` can come before or after the ratio.	`auto`	CSS Sizing 4 §4.1
`contain-intrinsic-width`	Sets the width that a box with size containment has instead of the size of its content.	An optional `auto`, then none or a non-negative length	none	CSS Sizing 4 §5.2
`contain-intrinsic-height`	Sets the height that a box with size containment has instead of the size of its content.	The values of `contain-intrinsic-width`	none	CSS Sizing 4 §5.2
`contain-intrinsic-inline-size`	Sets the inline size that a box with size containment has instead of the size of its content.	The values of `contain-intrinsic-width`	none	CSS Sizing 4 §5.2
`contain-intrinsic-block-size`	Sets the block size that a box with size containment has instead of the size of its content.	The values of `contain-intrinsic-width`	none	CSS Sizing 4 §5.2
`contain-intrinsic-size`	Sets `contain-intrinsic-width` and `contain-intrinsic-height`. One value applies to both.	One or two values of `contain-intrinsic-width`	none	CSS Sizing 4 §5.2

Name	Description	Values	Default	Specification
`frame-sizing`	Chooses the size of a replaced element that is used for its sizing.	`auto`, `content-width`, `content-height`, `content-block-size` or `content-inline-size`	`auto`	CSS Sizing 4 §5.3
`min-intrinsic-sizing`	Chooses when the minimum size that a box contributes to its container is compressed.	`legacy`, `zero-if-scroll`, `zero-if-extrinsic` or both of the last two	`legacy`	CSS Sizing 4 §5.5

Backgrounds

This reference lists the properties that CSSMedia uses to paint the background of a box.

Color, images and shorthands

Name	Description	Values	Default	Specification
`background`	Sets all the background properties in one declaration. Layers are separated by commas, and the color goes in the last layer. It resets the properties that you leave out.	One or more layers, each with an image, a position, a size after `/`, a repeat style, an attachment and one or two boxes; and a color in the last layer	See the individual properties	CSS Backgrounds 4 §2.12
`background-tbd`	Sets the background layers like `background`, but takes no color and does not reset it.	One or more layers	See the individual properties	CSS Backgrounds 4 §2.11
`background-color`	Sets the solid color painted below the images.	A color	`transparent`	CSS Backgrounds 4 §2.1

Name	Description	Values	Default	Specification
`background-image`	Sets the images of the layers. The first image is the top layer.	A comma-separated list of none or images: `url()`, gradients, `image()`, `image-set()` and `cross-fade()`	none	CSS Backgrounds 4 §2.2

Position, size and repetition

Name	Description	Values	Default	Specification
`background-position`	Sets where each image is placed in its positioning area.	One to four values: `left`, `center`, `right`, `top`, `bottom`, lengths and percentages. A side keyword can be followed by an offset.	`0% 0%`	CSS Backgrounds 4 §2.7
`background-position-x`	Sets the horizontal position of each image.	`center`, `left`, `right`, `x-start`, `x-end`, a length or percentage, or a side keyword with an offset	`0%`	CSS Backgrounds 4 §2.6
`background-position-y`	Sets the vertical position of each image.	`center`, `top`, `bottom`, `y-start`, `y-end`, a length or percentage, or a side keyword with an offset	`0%`	CSS Backgrounds 4 §2.6
`background-position-inline`	Sets the position of each image along the inline axis.	`center`, `start` or end, a length or percentage, or a keyword with an offset	`0%`	CSS Backgrounds 4 §2.6
`background-position-block`	Sets the position of each image along the block axis.	The values of `background-position-inline`	`0%`	CSS Backgrounds 4 §2.6

Name	Description	Values	Default	Specification
`background-size`	Sets the size of each image. `cover` makes the image cover the area, and `contain` makes the whole image fit in it.	`cover`, `contain`, or one or two values that are `auto` or non-negative lengths or percentages	`auto`	CSS Backgrounds 4 §2.10
`background-repeat`	Sets how each image repeats on the two axes. One value applies to both axes, and two values are the horizontal and the vertical ones.	`repeat-x`, `repeat-y`, `repeat-inline`, `repeat-block`, or one or two of `repeat`, `no-repeat`, `space` and `round`	`repeat`	CSS Backgrounds 4 §2.4
`background-repeat-x`	Sets how each image repeats on the horizontal axis.	`repeat`, `no-repeat`, `space` or `round`	`repeat`	CSS Backgrounds 4 §2.3
`background-repeat-y`	Sets how each image repeats on the vertical axis.	The values of `background-repeat-x`	`repeat`	CSS Backgrounds 4 §2.3
`background-repeat-inline`	Sets how each image repeats along the inline axis.	The values of `background-repeat-x`	`repeat`	CSS Backgrounds 4 §2.3
`background-repeat-block`	Sets how each image repeats along the block axis.	The values of `background-repeat-x`	`repeat`	CSS Backgrounds 4 §2.3
`background-origin`	Chooses the box that is the positioning area of the images.	`border-box`, `padding-box` or `content-box`	`padding-box`	CSS Backgrounds 4 §2.9
`background-clip`	Chooses the area that the background is painted in.	`border-box`, `padding-box`, `content-box`, `border-area`, `text`, or `border-area` and `text` together	`border-box`	CSS Backgrounds 4 §2.8

Name	Description	Values	Default	Specification
`background-attachment`	Chooses what the images are attached to. In a PDF, `fixed` attaches them to the page, so that they are drawn again on every page.	`scroll`, `local` or `fixed`	`scroll`	CSS Backgrounds 4 §2.5

Blending

Name	Description	Values	Default	Specification
`background-blend-mode`	Sets how each image layer blends with the layers below it and with the background color.	`normal`, `multiply`, `screen`, `overlay`, `darken`, `lighten`, `color-dodge`, `color-burn`, `hard-light`, `soft-light`, `difference`, `exclusion`, hue, `saturation`, `color` or `luminosity`; a comma-separated list	`normal`	Compositing 1 §3.4.3

Blending of whole boxes

Name	Description	Values	Default	Specification
`mix-blend-mode`	Sets how a box blends with what is behind it.	The values of `background-blend-mode`	`normal`	Compositing 1 §3.4.1
`isolation`	Chooses whether a box makes a group of its own for blending.	`auto` or `isolate`	`auto`	Compositing 1 §3.4.2

Display and positioning

The `display` property determines the kind of box an element generates. Positioning properties describe how a box is placed relative to its normal position or containing block.

Name	Description	Values	Default	Specification
`display`	Selects whether an element generates a box and describes its outer and inner display types.	`none`, `contents`, or combinations of outer types (`block`, `inline`, `run-in`), inner types (`flow`, `flow-root`,		CSS Display 4 §2

Name	Description	Values	Default	Specification
		`table`, `flex`, `grid`, `ruby`), and `list-item`; also internal table and ruby types, legacy combinations such as `inline-block`, `inline-flex`, and `inline-grid`, plus `math`, `grid-lanes`, and `inline-grid-lanes`		
`visibility`	Controls whether a box is visible while preserving its place in layout.	`visible`, `hidden`, `force-hidden`, or `collapse`		CSS Display 4 §5
`order`	Changes the order of flex and grid items for layout and painting.	An integer		CSS Display 4 §3
`reading-flow`	Chooses the reading order of children in a block, flex, or grid container.	`normal`, `source-order`, `flex-visual`, `flex-flow`, `grid-rows`, `grid-columns`, or `grid-order`		CSS Display 4 §4
`reading-order`	Assigns an integer ordering value to a direct child of a reading-flow container.	An integer		CSS Display 4 §4.1
`position`	Chooses the positioning scheme for an element.	`static`, `relative`, `absolute`, `sticky`, `fixed`, or `running()`		CSS Position 3 §2; CSS GCPM 3 §1.2.1
`top`, `right`, `bottom`, `left`	Set an inset from the corresponding side of the containing block for a positioned element.	`auto`, a length or percentage, `anchor()`, or `anchor-size()`		CSS Position 3 §3.1
`inset`	Sets the four physical insets.	One to four values accepted by `top`, `right`, `bottom`, and `left`		CSS Position 3 §3.2

Name	Description	Values	Default	Specification
`inset-block`, `inset-inline`	Set the start and end insets along the block or inline axis.	One or two values accepted by the corresponding logical inset properties		CSS Position 3 §3.2
`inset-block-start`, `inset-block-end`, `inset-inline-start`, `inset-inline-end`	Set one inset on the block or inline axis.	`auto`, a length or percentage, `anchor()`, or `anchor-size()`		CSS Position 3 §3.1
`z-index`	Sets a positioned box's stack level.	`auto` or an integer		CSS 2 §9.9
`float`	Takes a box out of the normal flow and places it at the side, at the top or at the bottom of its container. The value `footnote` makes the element a footnote.	`none`, `left`, `right`, `top`, `bottom`, `block-start`, `block-end`, `inline-start`, `inline-end`, `snap-block`, `snap-inline` or `footnote`	none	CSS Page Floats 3 §3.2

- A length is a size with a unit, such as `12pt` or `1em`. A percentage is relative to the containing block, as defined for the relevant inset.
- `anchor()` uses an anchor's side or center as an inset coordinate. `anchor-size()` uses a selected anchor's size. See CSS Anchor Positioning 1 and the `anchor-size()` function.
- `running()` names an element to move into a page margin box. See CSS GCPM 3.
- An integer is a whole number without a fractional part. Negative values, zero, and positive values are allowed for `order` and `z-index`; CSS Display defines how ordering is applied.

Tables

This reference lists the display values that make a table and the properties that style it.

Display values of tables

`display`	The element is	Specification
`table`	A table, as a block.	CSS Tables 3 §2
`inline-table`	A table that sits in a line of text.	CSS Tables 3 §2

display	The element is	Specification
`table-caption`	The caption of the table, above it or below it.	CSS Tables 3 §2
`table-header-group`	Rows that are always at the start of the table, and that can repeat on every page.	CSS Tables 3 §2
`table-row-group`	A group of rows.	CSS Tables 3 §2
`table-footer-group`	Rows that are always at the end of the table, and that can repeat on every page.	CSS Tables 3 §2
`table-row`	A row of cells.	CSS Tables 3 §2
`table-cell`	A cell, with the content of the table.	CSS Tables 3 §2
`table-column`	A column. It has no content, and it gives a width, a border or a background to the cells of a column.	CSS Tables 3 §2
`table-column-group`	A group of columns.	CSS Tables 3 §2

Table properties

Name	Description	Values	Default	Specification
`table-layout`	Chooses how the widths of the columns are found: from the content of all the cells, or from the first row and the column elements.	`auto` or `fixed`; `fixed` needs a width on the table	`auto`	CSS Tables 3 §3.5.1
`border-collapse`	Chooses between a border for each cell and one border shared by neighbors.	`separate` or `collapse`	`separate`	CSS Tables 3 §3.5.2
`border-spacing`	Sets the gap between the borders of neighbor cells when the borders are separate.	One length for both directions, or two lengths for the horizontal and the vertical gap	`0`	CSS Tables 3 §3.5.2.1
`caption-side`	Puts the caption above or below the table.	`top` or `bottom`	`top`	CSS Tables 3 §3.5.3

Name	Description	Values	Default	Specification
`empty-cells`	Chooses whether the borders and the background of a cell without content are shown, when the borders are separate.	`show` or `hide`	`show`	CSS Tables 3 §5.2
`vertical-align`	On a cell, sets where its content goes when the row is taller than the content.	`baseline`, `top`, `middle` or `bottom`	`baseline`	CSS Inline 3

Text and fonts

This reference lists the properties, at-rules and descriptors that CSSMedia uses to select, load and shape fonts.

Font selection and sizing

Name	Description	Values	Default	Specification
`font`	Sets the font style, variant, weight, width, size, line height and family in one declaration.	A system font keyword (`caption`, `icon`, `menu`, `message-box`, `small-caption` or `status-bar`), or optional style, `normal` or `small-caps`, weight and width, then the size, an optional `/` and line height, and the family	See the individual properties	CSS Fonts 4 §2.7
`font-family`	Lists the font families to try, in order. A family name with spaces needs quotes.	Family names, and `serif`, `sans-serif`, `monospace`, `system-ui`, `cursive`, `fantasy`, `math`, `ui-serif`, `ui-sans-serif`, `ui-monospace`, `ui-rounded` or `generic()` keywords	`sans-serif`, which is Helvetica	CSS Fonts 4 §2.1

Name	Description	Values	Default	Specification
`font-size`	Sets the size of the text.	A keyword from `xx-small` to `xxx-large`, `larger`, `smaller`, `math`, or a non-negative length or percentage	`medium`	CSS Fonts 4 §2.5
`font-size-adjust`	Scales the text so that a chosen metric of the font, such as the height of lowercase letters, has the same relative size in every font.	none, or an optional metric (`ex-height`, `cap-height`, `ch-width`, `ic-width` or `ic-height`) followed by `from-font` or a non-negative number	`none`	CSS Fonts 5 §3.4
`font-weight`	Selects the weight of the face.	`normal`, `bold`, `bolder`, `lighter` or a number from 1 to 1000	`normal`	CSS Fonts 4 §2.2
`font-width`	Selects the width of the face.	`normal`, a non-negative percentage, or `ultra-condensed`, `extra-condensed`, `condensed`, `semi-condensed`, `semi-expanded`, `expanded`, `extra-expanded` or `ultra-expanded`	`normal`	CSS Fonts 4 §2.3
`font-stretch`	Alias of `font-width`.	The values of `font-width`	`normal`	CSS Fonts 4 §2.3.1
`font-style`	Selects an upright, italic or oblique face.	`normal`, `italic`, `left`, `right`, or `oblique` with an optional angle from -90deg to 90deg	`normal`	CSS Fonts 4 §2.4
`font-synthesis`	Allows or forbids the faces that CSSMedia creates when the family has none: bold, slanted, small capitals, subscripts and superscripts.	none, or any of `weight`, `style`, `small-caps` and `position`	`weight style small-caps position`	CSS Fonts 4 §2.8.5

Name	Description	Values	Default	Specification
`font-synthesis-weight`	Allows or forbids synthesized bold.	`auto` or `none`	`auto`	CSS Fonts 4 §2.8.1
`font-synthesis-style`	Allows or forbids synthesized italic and oblique faces.	`auto`, `none` or `oblique-only`	`auto`	CSS Fonts 4 §2.8.2
`font-synthesis-small-caps`	Allows or forbids synthesized small capitals.	`auto` or `none`	`auto`	CSS Fonts 4 §2.8.3
`font-synthesis-position`	Allows or forbids synthesized subscripts and superscripts.	`auto` or `none`	`auto`	CSS Fonts 4 §2.8.4
`line-height`	Sets the height of a line of text.	`normal`, a non-negative number, or a non-negative length or percentage	`normal`	CSS Inline Layout 3 §5.1

Text color, alignment and lines

Name	Description	Values	Default	Specification
`color`	Sets the color of the text. Borders and other properties that use `currentcolor` take it too, and the property is inherited.	A color: a keyword, `#rgb` or `#rrggbb`, `rgb()`, `hsl()` or another color function, `currentcolor` or `transparent`	`CanvasText`, black	CSS Color 4 §3.2
`text-align`	Aligns the lines of a block. `justify` stretches every line but the last, and `justify-all` stretches the last one too. `start` and `end` follow the direction of the text.	`start`, `end`, `left`, `right`, `center`, `justify`, `justify-all` or `match-parent`	`start`	CSS Text 4 §7.1

Name	Description	Values	Default	Specification
`text-transform`	Changes the case of the text when it is drawn, without changing your XML. `full-width` replaces the letters and digits by their full-width forms, which CJK text uses.	none, or `capitalize`, `uppercase` or `lowercase`, with `full-width`	none	CSS Text 4 §2.1
`text-decoration`	Draws lines on the text. It is a shorthand for `text-decoration-line`, `text-decoration-style`, `text-decoration-color` and `text-decoration-thickness`. The lines run across the text of the children too.	A line: none or one or more of `underline`, `overline` and `line-through`; a style: `solid`, `double`, `dotted`, `dashed` or `wavy`; a color; a thickness	none	CSS Text Decoration 4 §2.6
`white-space`	Chooses whether spaces and line feeds collapse and whether lines wrap. It is a shorthand for `white-space-collapse` and `text-wrap-mode`. `pre` keeps spaces and line feeds and does not wrap, `pre-wrap` keeps them and wraps, `pre-line` keeps line feeds only, and `nowrap` collapses spaces and does not wrap. `break-spaces` keeps spaces and lets a line break after each.	`normal`, `pre`, `nowrap`, `pre-wrap`, `pre-line` or `break-spaces`	`normal`	CSS Text 4 §3
`overflow-wrap`	Lets a word that is longer than its line break at any letter, so that it does not overflow the box.	`normal`, `break-word` or `anywhere`	`normal`	CSS Text 4 §6.4

Font variants and features

Name	Description	Values	Default	Specification
`font-variant`	Shorthand for the `font-variant-*` properties.	`normal`, none, or values of the longhand properties	`normal`	CSS Fonts 4 §6.11
`font-variant-alternates`	Selects alternate glyphs that the font names, such as stylistic sets and swashes.	`normal`, or any of `stylistic()`, `historical-forms`, `styleset()`, `character-variant()`, `swash()`, `ornaments()` and `annotation()`	`normal`	CSS Fonts 4 §6.8
`font-variant-caps`	Selects small capitals and other capital forms.	`normal`, `small-caps`, `all-small-caps`, `petite-caps`, `all-petite-caps`, `unicase` or `titling-caps`	`normal`	CSS Fonts 4 §6.6
`font-variant-east-asian`	Selects glyph forms and widths for East Asian text.	`normal`, or any of `jis78`, `jis83`, `jis90`, `jis04`, `simplified`, `traditional`, `full-width`, `proportional-width` and `ruby`	`normal`	CSS Fonts 4 §6.10
`font-variant-emoji`	Chooses text or emoji presentation for characters that have both.	`normal`, `text`, `emoji` or `unicode`	`normal`	CSS Fonts 4 §9.3
`font-variant-ligatures`	Turns common, discretionary, historical and contextual ligatures on or off.	`normal`, none, or any of `common-ligatures`, `no-common-ligatures`, `discretionary-ligatures`, `no-discretionary-ligatures`, `historical-ligatures`, `no-historical-ligatures`, `contextual` and `no-contextual`	`normal`	CSS Fonts 4 §6.4

Name	Description	Values	Default	Specification
`font-variant-numeric`	Selects the shape and spacing of numbers, fractions and zeros.	`normal`, or any of `lining-nums`, `oldstyle-nums`, `proportional-nums`, `tabular-nums`, `diagonal-fractions`, `stacked-fractions`, `ordinal` and `slashed-zero`	`normal`	CSS Fonts 4 §6.7
`font-variant-position`	Selects subscript or superscript glyphs.	`normal`, `sub` or `super`	`normal`	CSS Fonts 4 §6.5
`font-kerning`	Turns the kerning of the font on or off.	`auto`, `normal` or `none`	`auto`	CSS Fonts 4 §6.3
`font-feature-settings`	Turns OpenType features on or off by their four-letter tags.	`normal`, or a comma-separated list of tags, each followed by an optional integer, on or `off`	`normal`	CSS Fonts 4 §6.12
`font-language-override`	Sets the OpenType language system of the text, in place of its language.	`normal` or a string	`normal`	CSS Fonts 4 §6.13

Variable and color fonts

Name	Description	Values	Default	Specification
`font-optical-sizing`	Sets the optical size axis of a variable font from the font size.	`auto` or none	`auto`	CSS Fonts 4 §8.1
`font-variation-settings`	Sets the axes of a variable font by their four-letter tags.	`normal`, or a comma-separated list of axis tags in quotes, each followed by a number	`normal`	CSS Fonts 4 §8.2
`font-palette`	Chooses the color palette of a color font.	`normal`, `light`, `dark`, the name of an `@font-palette-values` rule, or `palette-mix()`	`normal`	CSS Fonts 4 §9.1

The @font-face rule

Name	Description	Values	Default	Specification
`@font-face`	Makes a font face available to the stylesheet under a family name.	A block of the descriptors below		CSS Fonts 4 §4.1
`font-family` descriptor	Names the family of the face. This name hides an installed family with the same name.	A family name	Required	CSS Fonts 4 §4.2
`src` descriptor	Lists the sources of the face. CSSMedia uses the first one that it can load.	A comma-separated list of `url()` and `local()` sources, each with optional `format()` and `tech()` hints	Required	CSS Fonts 4 §4.3
`font-weight` descriptor	Gives the weight, or the range of weights, of the face.	`auto`, or one or two values, each `normal`, `bold` or a number from 1 to 1000	`auto`	CSS Fonts 4 §4.4
`font-width` descriptor	Gives the width, or the range of widths, of the face.	`auto`, or one or two values, each a width keyword or a non-negative percentage	`auto`	CSS Fonts 4 §4.4
`font-stretch` descriptor	Alias of the `font-width` descriptor.	The values of the `font-width` descriptor	`auto`	CSS Fonts 4 §4.4.1
`font-style` descriptor	Gives the style, or the range of oblique angles, of the face.	`auto`, `normal`, `italic`, `left`, `right`, or `oblique` with one or two angles from -90deg to 90deg	`auto`	CSS Fonts 4 §4.4
`font-size` descriptor	Limits the font sizes for which the face is used.	`auto` or one or two numbers	`auto`	CSS Fonts 5 §4.3

Name	Description	Values	Default	Specification
`font-named-instance` descriptor	Selects a named instance of a variable font.	`auto` or a string	`auto`	CSS Fonts 4 §4.7
`unicode-range` descriptor	Limits the characters for which the face is used.	A comma-separated list of code points, ranges and ranges with ? wildcards	`U+0-10FFFF`	CSS Fonts 4 §4.5
`font-feature-settings` descriptor	Sets the OpenType features that the face uses by default.	The values of the `font-feature-settings` property	`normal`	CSS Fonts 4 §4.6
`font-variation-settings` descriptor	Sets the variable font axes that the face uses by default.	The values of the `font-variation-settings` property	`normal`	CSS Fonts 4 §4.6
`font-language-override` descriptor	Sets the OpenType language system that the face uses by default.	`normal` or a string	`normal`	CSS Fonts 4 §4.10
`size-adjust` descriptor	Scales the glyphs and the metrics of the face.	A non-negative percentage	`100%`	CSS Fonts 5 §4.4
`ascent-override` descriptor	Replaces the ascent of the face.	One or two values, each `normal` or a non-negative percentage	`normal`	CSS Fonts 5 §4.5
`descent-override` descriptor	Replaces the descent of the face.	One or two values, each `normal` or a non-negative percentage	`normal`	CSS Fonts 5 §4.5
`line-gap-override` descriptor	Replaces the line gap of the face.	One or two values, each `normal` or a non-negative percentage	`normal`	CSS Fonts 5 §4.5
`superscript-position-override` descriptor	Replaces the position of the superscript glyphs.	One or two values, each `normal`, `from-font` or a percentage	`normal`	CSS Fonts 5 §4.6

Name	Description	Values	Default	Specification
`superscript-size-override` descriptor	Replaces the size of the superscript glyphs.	One or two values, each `normal`, `from-font` or a non-negative percentage	`normal`	CSS Fonts 5 §4.6
`subscript-position-override` descriptor	Replaces the position of the subscript glyphs.	One or two values, each `normal`, `from-font` or a percentage	`normal`	CSS Fonts 5 §4.6
`subscript-size-override` descriptor	Replaces the size of the subscript glyphs.	One or two values, each `normal`, `from-font` or a non-negative percentage	`normal`	CSS Fonts 5 §4.6

The @font-feature-values rule

Name	Description	Values	Default	Specification
`@font-feature-values`	Gives names to the alternate glyphs of one or more font families, for use in `font-variant-alternates`.	One or more family names, and a block with the rules below		CSS Fonts 4 §6.9
`@stylistic`	Names stylistic alternates (`salt`).	Names, each with one non-negative integer		CSS Fonts 4 §6.9.1
`@styleset`	Names stylistic sets (`ss01` to `ss99`).	Names, each with one or more integers from 1 to 99. 0 and larger values select no feature.		CSS Fonts 4 §6.9.2
`@character-variant`	Names character variants (`cv01` to `cv99`).	Names, each with one or two integers. The first, from 1 to 99, selects the feature, and the second sets its value. 0 and larger values select no feature.		CSS Fonts 4 §6.9.2
`@swash`	Names swashes (`swsh`, `cswh`).	Names, each with one non-negative integer		CSS Fonts 4 §6.9.1

Name	Description	Values	Default	Specification
`@ornaments`	Names ornaments (`ornm`).	Names, each with one non-negative integer		CSS Fonts 4 §6.9.1
`@annotation`	Names annotation forms (`nalt`).	Names, each with one non-negative integer		CSS Fonts 4 §6.9.1
`@historical-forms`	Names historical forms (`hist`).	Names, each with one or more non-negative integers		CSS Fonts 4 §6.9.1

The @font-palette-values rule

Name	Description	Values	Default	Specification
`@font-palette-values`	Defines a named color palette for a color font. Use its name in `font-palette`.	A name that starts with `--`, and a block of the descriptors below		CSS Fonts 4 §9.2
`font-family` descriptor	Names the families that the palette is for.	One or more family names	Required	CSS Fonts 4 §9.2.1
`base-palette` descriptor	Chooses the palette of the font that the new palette starts from.	`light`, `dark` or a non-negative integer	The first palette of the font	CSS Fonts 4 §9.2.2
`override-colors` descriptor	Replaces colors of the base palette.	A comma-separated list of pairs of a palette index and a color	No replacements	CSS Fonts 4 §9.2.3

Languages and hyphenation

This reference lists the property, the selector and the properties that CSSMedia uses for the language of the text, hyphenation and quotation marks.

Properties

Name	Description	Values	Default	Specification
`-cssmedia-lang`	Sets the language of the elements from a value of your XML, such as an attribute. CSSMedia extension.	none or a string with a language tag. `attr()` reads the tag from an attribute.	none	CSSMedia extension

Name	Description	Values	Default	Specification
`hyphens`	Chooses how words are broken at the end of a line. `manual` breaks only at the soft hyphens that the text has, `auto` also uses the hyphenation patterns of the language, and none never breaks inside a word.	none, `manual` or `auto`	`manual`	CSS Text 4 §6.3.1
`hyphenate-character`	Sets the string that CSSMedia writes at the end of a line that it breaks inside a word.	`auto` or a string. `auto` is U+2010 when the font has it, and U+002D otherwise.	`auto`	CSS Text 4 §6.3.2
`hyphenate-limit-chars`	Sets the shortest word that CSSMedia hyphenates, and the least number of letters before and after the hyphen.	One to three values, each `auto` or an integer. The values are the word length, the letters before and the letters after. A missing third value is the second, and a missing second value is `auto`.	`auto`	CSS Text 4 §6.3.4
`hyphenate-limit-lines`	Sets the largest number of consecutive lines of an element that end with a hyphen.	`no-limit` or an integer	`no-limit`	CSS Text 4 §6.3.5
`hyphenate-limit-last`	Stops CSSMedia from ending a line with a hyphen at the end of an element or before a page break. `always` applies to the last full line of the element and to the last line before a page break. `column` and page apply to the last line before a page break, and `spread` to the last line before a spread break.	none, `always`, `column`, page or `spread`	none	CSS Text 4 §6.3.5

Name	Description	Values	Default	Specification
`hyphenate-limit-zone`	Sets how much empty space a line can have before CSSMedia hyphenates a word to fill it. A percentage is relative to the width of the line.	A length or a percentage	`0`	CSS Text 4 §6.3.3
`quotes`	Selects the quotation marks used by generated `open-quote` and `close-quote` content. With `auto`, the marks depend on the language of the text.	`auto`, `none`, `match-parent`, or one or more pairs of quoted strings	`auto`	CSS Content 3 §2.4.1

Selector

Name	Description	Values	In a PDF	Specification
`:lang()`	Elements in a language. The language comes from `xml:lang` or the `-cssmedia-lang` property.	One or more language ranges, as identifiers or strings.	Matches	Selectors 4 §7.2

Page reference

Name	Description	Values	Default	Specification
`page`	Assigns a named page type to a box.	`auto` or a custom identifier	`auto`	CSS Paged Media 3, `page`
`@page`	Defines page-box declarations and may contain page-margin rules.	Optional comma-separated page selectors		CSS Paged Media 3, `@page`
`size`	Sets page-box size and orientation. `auto` uses A4.	`auto`; one or two lengths; or A3, A4, A5, B4, B5, `JIS-B4`, `JIS-B5`, `ledger`, `legal`, or `letter`, optionally combined with `portrait` or `landscape`	A4	CSS Paged Media 3, `size`

Name	Description	Values	Default	Specification
`bleed`	Sets the bleed extension beyond the page box. `auto` uses `6pt` with crop marks and `0` otherwise.	`auto` or a length	`0` (`auto`)	CSS Paged Media 3, `bleed`
`marks`	Requests crop marks, registration marks, or both outside the page box.	`none`, `crop`, `cross`, or both `crop` and `cross`	`none`	CSS Paged Media 3, `marks`
`page-orientation`	Rotates the laid-out page in the output.	`upright`, `rotate-left`, or `rotate-right`	`upright`	CSS Paged Media 3, `page-orientation`
`@top-left-corner`	Defines the fixed-size box at the intersection of the top and left page margins.			CSS Paged Media 3, `@top-left-corner`
`@top-left`	Defines the box between the top-left corner and top-center boxes.			CSS Paged Media 3, `@top-left`
`@top-center`	Defines the centered box in the top page margin.			CSS Paged Media 3, `@top-center`
`@top-right`	Defines the box between the top-center and top-right corner boxes.			CSS Paged Media 3, `@top-right`
`@top-right-corner`	Defines the fixed-size box at the intersection of the top and right page margins.			CSS Paged Media 3, `@top-right-corner`
`@right-top`	Defines the box between the top-right corner and right-middle boxes.			CSS Paged Media 3, `@right-top`

Name	Description	Values	Default	Specification
`@right-middle`	Defines the centered box in the right page margin.			CSS Paged Media 3, `@right-middle`
`@right-bottom`	Defines the box between the right-middle and bottom-right corner boxes.			CSS Paged Media 3, `@right-bottom`
`@bottom-right-corner`	Defines the fixed-size box at the intersection of the bottom and right page margins.			CSS Paged Media 3, `@bottom-right-corner`
`@bottom-right`	Defines the box between the bottom-center and bottom-right corner boxes.			CSS Paged Media 3, `@bottom-right`
`@bottom-center`	Defines the centered box in the bottom page margin.			CSS Paged Media 3, `@bottom-center`
`@bottom-left`	Defines the box between the bottom-left corner and bottom-center boxes.			CSS Paged Media 3, `@bottom-left`
`@bottom-left-corner`	Defines the fixed-size box at the intersection of the bottom and left page margins.			CSS Paged Media 3, `@bottom-left-corner`
`@left-bottom`	Defines the box between the left-middle and bottom-left corner boxes.			CSS Paged Media 3, `@left-bottom`
`@left-middle`	Defines the centered box in the left page margin.			CSS Paged Media 3, `@left-middle`

Name	Description	Values	Default	Specification
`@left-top`	Defines the box between the top-left corner and left-middle boxes.			CSS Paged Media 3, `@left-top`
`@footnote`	Styles the footnote area at the bottom of the page. It goes inside an `@page` rule.	A block of declarations, such as borders, margins, padding and `max-height`.		CSS GCPM 3 §2.4

Breaks

Name	Description	Values	Default	Specification
`break-before`	Forces or avoids a break before a block, a row or a row group. `page`, `always` and `all` start a new page. `left` and `verso` start the next left page, and `right` and `recto` the next right page, with a blank page when needed. `avoid` and `avoid-page` move the break to an earlier place. `column` breaks only in columns.	`auto`, `avoid`, `avoid-page`, `page`, `always`, `all`, `left`, `right`, `recto`, `verso`, `column` or `avoid-column`	`auto`	CSS Fragmentation 4 §3.1
`break-after`	Forces or avoids a break after a block, a row or a row group, with the same values as `break-before`.	As `break-before`	`auto`	CSS Fragmentation 4 §3.1
`break-inside`	Avoids a break inside a box, so that it stays on one page when it fits on one. A table header or footer group with `avoid` repeats on each page of the table.	`auto`, `avoid`, `avoid-page` or `avoid-column`	`auto`	CSS Fragmentation 4 §3.2
`orphans`	Sets how many lines of a paragraph must stay at the bottom of a page. If fewer fit, the whole paragraph goes to the next page.	An integer of 1 or more	2	CSS Fragmentation 4 §3.3
`widows`	Sets how many lines of a paragraph must go to the top of the next page. If fewer would, lines from the previous page go with them.	An integer of 1 or more	2	CSS Fragmentation 4 §3.3

Declarations in page-margin rules

Each page-margin at-rule contains declarations written as `property: value;`. Use properties that apply to page-margin boxes, including `content`, font and text properties, color, backgrounds, borders, margins, padding, dimensions and `vertical-align`. See the CSS Paged Media page-margin property list.

For example, set a running title in the top-center box and a page number in the bottom-center box:

```
@page {
    @top-center {
        content: "Annual report";
        font-weight: bold;
    }

    @bottom-center {
        content: "Page " counter(page);
        text-align: center;
    }
}
```

Links and generated content reference

The values list the syntax defined by the cited specifications.

Name	Description	Values	Default	Specification
`content`	Replaces an element's contents or supplies generated content for pseudo-elements and page margin boxes.	`normal`, none, an image that replaces the contents, a list of generated items, or `element()`; a list can include alternative text after `/`	`normal`	CSS Content 3 §1
`bookmark-label`	Sets the text of the bookmark of an element. It is also the default text of the entry of the element in a table of contents.	A list of generated content items	`content(text)`	CSS Content 3 §3.2

Name	Description	Values	Default	Specification
`bookmark-level`	Makes an element a bookmark of the PDF and an entry in a table of contents, and sets its level. Level 1 is the top level.	none or an integer of 1 or greater	`none`	CSS Content 3 §3.1
`bookmark-state`	Sets whether the bookmark of a block-level element shows its child bookmarks or hides them when the PDF opens.	`open` or `closed`	`open`	CSS Content 3 §3.3
`quotes`	Selects the quotation marks used by generated `open-quote` and `close-quote` content. With `auto`, the marks depend on the language of the text.	`auto`, `none`, `match-parent`, or one or more pairs of quoted strings	`auto`	CSS Content 3 §2.4.1
`string-set`	Assigns named values from an element for retrieval in page content.	none or comma-separated pairs of a name and one or more strings		CSS Content 3 §2.7.1
`string()`	Gives the value of a named string saved with `string-set`. Use it in the `content` property of a page-margin box.	A name, and an optional mode: `first`, `start`, `last` or `first-except`	`first`	CSS GCPM 3 §1.1.2
`content()`	Gives text of an element. Use it in the value of `string-set`.	`text`, `before`, `after` or `first-letter`	`text`	CSS GCPM 3 §1.1.1.1

Name	Description	Values	Default	Specification
`-cssmedia-link-counter()`	Gives the value of a counter at the destination of the link of the element, such as the number of a footnote that the link points to. CSSMedia extension.	The name of a counter, and an optional counter style.		CSSMedia extension
`-cssmedia-link`	Makes an XML element a link to another part of the document or to an external address. The values are joined in order to make the destination.	none, or one or more strings or `url()` values	none	CSSMedia extension
`link-parameters`	Supplies parameters for links to external resources.	none or a comma-separated list of `param()` values		CSS Link Parameters 1 §2.1
`leader()`	Fills the space between the content before it and the content after it on a line with a repeated pattern. Use it in `content`.	`dotted`, `solid`, `space` or a string		CSS Content 3 §2.5.1
`target-counter()`	Gives the value of a counter at the destination of a link, such as the page number of a heading. Use it in `content`.	A link, as `attr()`, a string or `url()`; a counter name; and an optional counter style		CSS Content 3 §2.6.1

Name	Description	Values	Default	Specification
`-cssmedia-toc()`	Generates a table of contents with an entry for each element that has a `bookmark-level`. Use it as the value of `content`. CSSMedia extension.	No arguments.		CSSMedia extension
`-cssmedia-toc-title`	Sets the title of the table of contents. Set it where `content` is `-cssmedia-toc()`. The title is also a bookmark if `@-cssmedia-toc-title` sets a `bookmark-level`. CSSMedia extension.	none or a list of generated content items	none	CSSMedia extension
`-cssmedia-toc-depth`	Sets the deepest level of `bookmark-level` that gets an entry in the table of contents. Set it where `content` is `-cssmedia-toc()`. CSSMedia extension.	`all` or an integer of 1 or greater	`all`	CSSMedia extension
`-cssmedia-toc-label`	Sets the text of the entry of an element in the table of contents. Set it on the element that has the `bookmark-level`. CSSMedia extension.	`auto` or a list of generated content items	`auto`, which uses the value of `bookmark-label`	CSSMedia extension

Name	Description	Values	Default	Specification
`-cssmedia-toc-divider`	Adds a line of text before the entry of an element in the table of contents, to group entries. Set it on the element that has the `bookmark-level`. CSSMedia extension.	none or a list of generated content items	none	CSSMedia extension
`-cssmedia-toc-leader`	Sets the leader between the text of an entry and its page number. Set it in `@-cssmedia-toc-entry`. CSSMedia extension.	`dotted`, `solid`, `space` or a string	`dotted`	CSSMedia extension
`@-cssmedia-toc-title`	Styles the title of the table of contents. CSSMedia extension.	A block of declarations.		CSSMedia extension
`@-cssmedia-toc-entry`	Styles the entries of the table of contents. Write a level after the name to style only the entries of that level. CSSMedia extension.	An optional integer of 1 or greater, and a block of declarations.		CSSMedia extension
`@-cssmedia-toc-divider`	Styles the dividers of the table of contents. Write a level after the name to style only the dividers of the entries of that level. CSSMedia extension.	An optional integer of 1 or greater, and a block of declarations.		CSSMedia extension

- A <`content-list`> is one or more generated content items, such as strings, quote keywords, counters, images, attribute values, or `content()` values. See CSS Content 3 and Named strings.

- A <content-replacement> is one image that replaces the element's normal contents. See CSS Content 3.
- The image form of element() uses an element ID to produce an image; the running-element form selects a named element for use in page margin boxes. See CSS Images 4 and CSS GCPM 3.
- A positive <integer> for bookmark-level is a whole number of at least 1. The top level is 1.
- A <string> is text between quotes. url() and src() provide URL values.
- A <custom-ident> is an author-chosen identifier, not a CSS-wide keyword or none.
- param() takes a dashed parameter name beginning with -- and an optional <declaration-value>. If omitted, the parameter value is empty. A <declaration-value> is the sequence of CSS component values after a declaration's colon, excluding top-level semicolons and unmatched closing braces. See CSS Link Parameters 1 and CSS Syntax 3.

For examples of destinations, named strings, and generated content, see Links and Named strings.

Counters, list markers, and footnotes

Name	Description	Values	Default	Specification
counter-reset	Creates a new counter instance and initializes it.	none or one or more counter names, each optionally followed by an integer; the default value for each name is 0		CSS Lists 3 §4.1
counter-increment	Increases each named counter, creating it if needed.	none or one or more counter names, each optionally followed by an integer; the default increment is 1		CSS Lists 3 §4.2
counter-set	Sets the value of existing counters, creating a counter if it does not exist.	none or one or more counter names, each optionally followed by an integer; the default value is 0		CSS Lists 3 §4.3
list-style	Sets the marker's position, image, and type.	Any order of inside or outside, an image or none, and a counter style, string, or none; any component can be omitted		CSS Lists 3 §3.6
list-style-image	Uses an image as the list marker.	An image or none		CSS Lists 3 §3.1

Name	Description	Values	Default	Specification
`list-style-position`	Places the marker inside the list item's contents or outside its principal box.	`inside` or `outside`		CSS Lists 3 §3.5
`list-style-type`	Chooses the marker's counter style or a literal marker string.	A counter-style name, a string, or none		CSS Lists 3 §3.4
`marker-side`	Chooses whether an outside marker's position follows the list item's directionality or its parent's.	`match-self` or `match-parent`		CSS Lists 3 §3.7
`@counter-style`	Defines a named counter style for markers and counter functions.	A name followed by counter-style descriptors		CSS Counter Styles 3 §3
`system` descriptor	Selects the algorithm that converts counter values to symbols.	`cyclic`, `numeric`, `alphabetic`, `symbolic`, `additive`, `fixed` with an optional starting integer, or `extends` with another counter-style name		CSS Counter Styles 3 §3.1
`symbols` descriptor	Lists the symbols used by the cyclic, fixed, symbolic, alphabetic, or numeric systems.	One or more symbols		CSS Counter Styles 3 §3.2
`additive-symbols` descriptor	Maps descending counter weights to symbols for the additive system.	Comma-separated pairs of a non-negative integer and a symbol, in descending weight order		CSS Counter Styles 3 §3.3
`negative` descriptor	Adds symbols before and optionally after a negative counter representation.	One or two symbols		CSS Counter Styles 3 §3.4
`prefix` descriptor	Adds a symbol before each marker.	One symbol		CSS Counter Styles 3 §3.7

Name	Description	Values	Default	Specification
`suffix` descriptor	Adds a symbol after each marker.	One symbol		CSS Counter Styles 3 §3.7
`range` descriptor	Limits the values that the counter style can represent before fallback is used.	`auto` or comma-separated pairs of integer bounds or `infinite`		CSS Counter Styles 3 §3.5
`pad` descriptor	Pads short representations to a minimum length with a chosen symbol.	A non-negative integer and a symbol		CSS Counter Styles 3 §3.6
`fallback` descriptor	Names the counter style used when this style cannot represent a value.	A counter-style name		CSS Counter Styles 3 §3.8
`footnote` counter	The counter that numbers the footnotes. Every footnote adds 1 to it. Change it with `counter-reset`, `counter-set` and `counter-increment`, in an element rule to number by section or in an @page rule to number by page.	Show it with `counter(footnote)`, with an optional counter style.	0 at the start of the document.	CSS GCPM 3 §2.5
`footnote-display`	Sets how a footnote body appears in the footnote area.	`block`, `inline`, or `compact`	`block`	CSS GCPM 3 §2.3
`footnote-policy`	Sets what happens when a footnote does not fit on the page of its call. `auto` continues it on the next pages, `line` moves the line with the call to the next page, and `block` moves the paragraph.	`auto`, `line`, or `block`	`auto`	CSS GCPM 3 §2.8

- A <counter-name> is an identifier naming a CSS counter. none and CSS-wide keywords are not counter names. See CSS Lists 3.
- An <integer> is a whole number without a fractional part. In the range descriptor, infinite means there is no bound in that direction. See CSS Values 4.
- An <image> is a URL, image function, or gradient that can provide image data. See CSS Images 3.
- A <counter-style> is a predefined counter style name, a name defined by @counter-style, or a symbols() function. See CSS Counter Styles 3.
- A <counter-style-name> is the identifier naming a predefined or author-defined counter style. See CSS Counter Styles 3.
- A <symbol> is a string, an identifier whose characters form the symbol, or an image. See CSS Counter Styles 3.

For example, this rule defines a two-symbol cycle. Replace item with an element name in your XML. The display declaration makes those elements list items, and their markers use the defined style.

```
@counter-style stars {
    system: cyclic;
    symbols: "★" "□";
    suffix: " ";
}

item {
    display: list-item;
    list-style-type: stars;
}
```

CSSMedia applies counter operations and prepares text marker content while building the box tree.

Named strings

This reference lists the property and the functions that CSSMedia uses for named strings.

Name	Description	Values	Default	Specification
string-set	Assigns named values from an element for retrieval in page content.	none or comma-separated pairs of a name and one or more strings		CSS Content 3 §2.7.1
content()	Gives text of an element. Use it in the value of string-set.	text, before, after or first-letter	text	CSS GCPM 3 §1.1.1.1

Name	Description	Values	Default	Specification
`string()`	Gives the value of a named string saved with `string-set`. Use it in the `content` property of a page-margin box.	A name, and an optional mode: `first`, `start`, `last` or `first-except`	`first`	CSS GCPM 3 §1.1.2

Links

This reference lists the property, the function and the pseudo-classes that CSSMedia uses for links.

Property and function

Name	Description	Values	Default	Specification
`-cssmedia-link`	Makes an XML element a link to another part of the document or to an external address. The values are joined in order to make the destination.	none, or one or more strings or `url()` values	none	CSSMedia extension
`-cssmedia-link-counter()`	Gives the value of a counter at the destination of the link of the element, such as the number of a footnote that the link points to. CSSMedia extension.	The name of a counter, and an optional counter style.		CSSMedia extension

Pseudo-classes

Name	Description	Values	In a PDF	Specification
`:link`, `:any-link`	Elements that are hyperlinks, because `-cssmedia-link` gives them a destination.		Matches	Selectors 4 §8.1, §8.2
`:local-link`	Hyperlinks that point to the same document. The optional number limits how many path segments are compared.	An optional non-negative integer in parentheses.	Matches	Selectors 5 §2.1
`:visited`, `:target`	Links that were visited, and the element that a link points to. A PDF has no browsing history and no current target.		Never matches	Selectors 4 §8.2, §8.3

Footnotes

This reference lists the properties, the rule, the counter and the pseudo-elements that CSSMedia uses for footnotes.

Properties, rule and counter

Name	Description	Values	Default	Specification
`float`	Takes a box out of the normal flow and places it at the side, at the top or at the bottom of its container. The value `footnote` makes the element a footnote.	`none`, `left`, `right`, `top`, `bottom`, `block-start`, `block-end`, `inline-start`, `inline-end`, `snap-block`, `snap-inline` or `footnote`	`none`	CSS Page Floats 3 §3.2
`footnote-display`	Sets how a footnote body appears in the footnote area.	`block`, `inline`, or `compact`	`block`	CSS GCPM 3 §2.3
`footnote-policy`	Sets what happens when a footnote does not fit on the page of its call. `auto` continues it on the next pages, `line` moves the line with the call to the next page, and `block` moves the paragraph.	`auto`, `line`, or `block`	`auto`	CSS GCPM 3 §2.8
`footnote` counter	The counter that numbers the footnotes. Every footnote adds 1 to it. Change it with `counter-reset`, `counter-set` and `counter-increment`, in an element rule to number by section or in an @page rule to number by page.	Show it with `counter(footnote)`, with an optional counter style.	0 at the start of the document.	CSS GCPM 3 §2.5

Name	Description	Values	Default	Specification
`@footnote`	Styles the footnote area at the bottom of the page. It goes inside an `@page` rule.	A block of declarations, such as borders, margins, padding and `max-height`.		CSS GCPM 3 §2.4

Pseudo-elements

Name	Description	Values	In a PDF	Specification
`::footnote-call`	The reference that replaces a footnote in the text. By default it is the value of the `footnote` counter, as a superscript.		Matches	CSS GCPM 3 §2.6
`::footnote-marker`	The marker at the start of a footnote body. By default it is the value of the `footnote` counter, followed by a period and a space.		Matches	CSS GCPM 3 §2.7

Table of contents

This reference lists the properties, functions and at-rules that CSSMedia uses for bookmarks and tables of contents.

Properties and functions

Name	Description	Values	Default	Specification
`bookmark-level`	Makes an element a bookmark of the PDF and an entry in a table of contents, and sets its level. Level 1 is the top level.	none or an integer of 1 or greater	none	CSS Content 3 §3.1
`bookmark-label`	Sets the text of the bookmark of an element. It is also the default text of the entry of the element in a table of contents.	A list of generated content items	`content(text)`	CSS Content 3 §3.2

Name	Description	Values	Default	Specification
`bookmark-state`	Sets whether the bookmark of a block-level element shows its child bookmarks or hides them when the PDF opens.	`open` or `closed`	`open`	CSS Content 3 §3.3
`-cssmedia-toc()`	Generates a table of contents with an entry for each element that has a `bookmark-level`. Use it as the value of `content`. CSSMedia extension.	No arguments.		CSSMedia extension
`-cssmedia-toc-title`	Sets the title of the table of contents. Set it where `content` is `-cssmedia-toc()`. The title is also a bookmark if `@-cssmedia-toc-title` sets a `bookmark-level`. CSSMedia extension.	none or a list of generated content items	none	CSSMedia extension
`-cssmedia-toc-depth`	Sets the deepest level of `bookmark-level` that gets an entry in the table of contents. Set it where `content` is `-cssmedia-toc()`. CSSMedia extension.	`all` or an integer of 1 or greater	`all`	CSSMedia extension
`-cssmedia-toc-label`	Sets the text of the entry of an element in the table of contents. Set it on the element that has the `bookmark-level`. CSSMedia extension.	`auto` or a list of generated content items	`auto`, which uses the value of `bookmark-label`	CSSMedia extension

Name	Description	Values	Default	Specification
`-cssmedia-toc-divider`	Adds a line of text before the entry of an element in the table of contents, to group entries. Set it on the element that has the `bookmark-level`. CSSMedia extension.	none or a list of generated content items	none	CSSMedia extension
`-cssmedia-toc-leader`	Sets the leader between the text of an entry and its page number. Set it in `@-cssmedia-toc-entry`. CSSMedia extension.	`dotted`, `solid`, space or a string	`dotted`	CSSMedia extension
`leader()`	Fills the space between the content before it and the content after it on a line with a repeated pattern. Use it in `content`.	`dotted`, `solid`, space or a string		CSS Content 3 §2.5.1
`target-counter()`	Gives the value of a counter at the destination of a link, such as the page number of a heading. Use it in `content`.	A link, as `attr()`, a string or `url()`; a counter name; and an optional counter style		CSS Content 3 §2.6.1

At-rules

Name	Description	Values	Default	Specification
`@-cssmedia-toc-title`	Styles the title of the table of contents. CSSMedia extension.	A block of declarations.		CSSMedia extension
`@-cssmedia-toc-entry`	Styles the entries of the table of contents. Write a level after the name to style only the entries of that level. CSSMedia extension.	An optional integer of 1 or greater, and a block of declarations.		CSSMedia extension

Name	Description	Values	Default	Specification
`@-cssmedia-toc-divider`	Styles the dividers of the table of contents. Write a level after the name to style only the dividers of the entries of that level. CSSMedia extension.	An optional integer of 1 or greater, and a block of declarations.		CSSMedia extension

Images

This reference lists the properties that CSSMedia uses to size, fit and draw images.

Image properties

Name	Description	Values	Default	Specification
`image-resolution`	Sets how many pixels of a raster image make one CSS pixel, which sets its natural size. `from-image` uses the resolution that the file states.	`from-image`, a resolution such as `2dppx` or `192dpi`, or both	`1dppx`	CSS Images 4 §6.1
`object-fit`	Chooses how an image fills its box: stretched, scaled to fit, scaled to cover, or at its natural size.	`fill`, `none`, `contain`, `cover` or `scale-down`; `scale-down` can follow `contain` or `cover`	`fill`	CSS Images 4 §5.1
`object-position`	Sets where an image goes in its box when it does not fill it.	One to four values: keywords, lengths and percentages, as in `background-position`	`50% 50%`	CSS Images 3 §4.6
`image-orientation`	Chooses whether an image is turned as its file says, and can turn or mirror it.	`from-image`, `none`, or an angle and `flip`	`from-image`	CSS Images 3 §5.1
`image-rendering`	Chooses how the pixels of an image are drawn when it is shown larger than its size: smoothed, with sharp edges, or enlarged by whole multiples of its pixels.	`auto`, `smooth`, `high-quality`, `pixelated` or `crisp-edges`	`auto`	CSS Images 3 §5.2

Name	Description	Values	Default	Specification
`image-animation`	Chooses whether an animated image plays. A PDF does not move, so it has no effect.	`normal`, `paused`, `stopped` or `running`	`normal`	CSS Image Animation 1

SVG

This reference lists the elements of SVG that CSSMedia draws and the properties that style them.

SVG elements

Element	Description	Attributes
`svg`	A drawing, or a nested drawing with its own size and scale.	`xmlns`, `x`, `y`, `width`, `height`, `viewBox`, `preserveAspectRatio`
`g`	Groups elements, so that they share styles and a transform.	`transform`
`defs`	Holds elements that are defined to be used later and are not drawn.	None
`use`	Draws a copy of another element.	`href`, `x`, `y`, `width`, `height`
`symbol`	A shape with its own `viewBox`, drawn by a `use` element.	`viewBox`, `preserveAspectRatio`, `refX`, `refY`
`switch`	Draws its first child that has no conditional attributes.	`systemLanguage`, `requiredExtensions` on the children
`rect`	A rectangle, with optional rounded corners.	`x`, `y`, `width`, `height`, `rx`, `ry`
`circle`	A circle.	`cx`, `cy`, `r`
`ellipse`	An ellipse.	`cx`, `cy`, `rx`, `ry`
`line`	A straight line.	`x1`, `y1`, `x2`, `y2`
`polyline`	An open shape made of straight lines.	`points`
`polygon`	A closed shape made of straight lines.	`points`

Element	Description	Attributes
`path`	A shape made of lines, curves and arcs.	`d`, `pathLength`
`linearGradient`	A paint that changes color along a line.	`x1`, `y1`, `x2`, `y2`, `gradientUnits`, `gradientTransform`, `spreadMethod`, `href`
`radialGradient`	A paint that changes color from a center to a circle.	`cx`, `cy`, `r`, `fx`, `fy`, `gradientUnits`, `gradientTransform`, `spreadMethod`, `href`
`stop`	A color of a gradient, at a position.	`offset`
`pattern`	A paint that repeats a small drawing.	`x`, `y`, `width`, `height`, `patternUnits`, `patternContentUnits`, `patternTransform`, `viewBox`, `preserveAspectRatio`, `href`
`marker`	A mark that is drawn on the vertices of a shape.	`markerWidth`, `markerHeight`, `refX`, `refY`, `orient`, `markerUnits`, `viewBox`, `preserveAspectRatio`
`clipPath`	An area that clips other elements.	`clipPathUnits`
`mask`	A drawing whose light or opaque parts show the masked element.	`x`, `y`, `width`, `height`, `maskUnits`, `maskContentUnits`
`text`	Text, in a line, wrapped to a length or inside a shape.	`x`, `y`, `dx`, `dy`, `rotate`, `textLength`, `lengthAdjust`
`tspan`	A part of a text with its own style or position.	`x`, `y`, `dx`, `dy`, `rotate`, `textLength`, `lengthAdjust`
`textPath`	Text that follows the outline of a shape.	`href`, `startOffset`, `side`
`image`	A PNG, JPEG, WebP or SVG file.	`href`, `x`, `y`, `width`, `height`, `preserveAspectRatio`
`a`	A link around the elements or the text it holds.	`href`
`style`	A style sheet for the drawing.	`type`, `media`

Geometry properties

Name	Description	Values	Default	Specification
x, y	Set the position of a `rect`, `image`, `svg`, `symbol` or `use` element.	A length or a percentage	`0`	SVG 2
cx, cy	Set the center of a `circle` or an `ellipse`.	A length or a percentage	`0`	SVG 2
r	Sets the radius of a `circle`.	A length or a percentage	`0`	SVG 2
rx, ry	Set the radii of an `ellipse`, or the rounded corners of a `rect`. With `auto` the radius takes the value of the other one.	A length, a percentage or `auto`	`auto`	SVG 2
d	Sets the data of a `path`.	none or a string of path data	none	SVG 2
`path-length`	Sets the length that a shape counts as having, for the lengths of its dashes.	none or a number	none	SVG 2

Paint properties

Name	Description	Values	Default	Specification
`fill`	Sets the paint of the inside of a shape or of the characters of a text.	A color, none, `currentColor` or `url(#id)`, optionally followed by a color that is used when the reference names nothing	`black`	SVG 2
`fill-opacity`	Sets the opacity of the fill.	A number from 0 to 1, or a percentage	`1`	SVG 2
`fill-rule`	Chooses which areas of a shape are inside it.	`nonzero` or evenodd	`nonzero`	SVG 2
`stroke`	Sets the paint of the outline of a shape or of a text.	As `fill`	none	SVG 2
`stroke-width`	Sets the thickness of the outline.	A length, a percentage or a number	`1`	SVG 2
`stroke-opacity`	Sets the opacity of the outline.	A number from 0 to 1, or a percentage	`1`	SVG 2

Name	Description	Values	Default	Specification
`stroke-linecap`	Sets the shape of the ends of an open line.	`butt`, `round` or `square`	`butt`	SVG 2
`stroke-linejoin`	Sets the shape of the corners of the outline.	`miter`, `round` or `bevel`	`miter`	SVG 2
`stroke-miterlimit`	Sets the length at which a sharp corner turns into a bevel, as a multiple of the thickness.	A number of 1 or more	`4`	SVG 2
`stroke-dasharray`	Makes the outline dashed, with a list of lengths for dashes and gaps.	none or a list of lengths, percentages or numbers	`none`	SVG 2
`stroke-dashoffset`	Moves the start of the dash pattern along the outline.	A length, a percentage or a number	`0`	SVG 2
`paint-order`	Chooses the order in which the fill, the stroke and the markers are painted.	`normal`, or `fill`, `stroke` and `markers` in the order you want	`normal`	SVG 2
`vector-effect`	With `non-scaling-stroke` the outline keeps its thickness when a transform scales the shape.	none or `non-scaling-stroke`	`none`	SVG 2
`color-interpolation`	Chooses the color space in which the colors of a gradient mix.	`auto`, `sRGB` or `linearRGB`	`sRGB`	SVG 2
`stop-color`	Sets the color of a `stop` of a gradient.	A color	`black`	SVG 2
`stop-opacity`	Sets the opacity of a `stop` of a gradient.	A number from 0 to 1, or a percentage	`1`	SVG 2

Name	Description	Values	Default	Specification
`marker`	Sets `marker-start`, `marker-mid` and `marker-end` at once.	none or `url(#id)`	none	SVG 2
`marker-start`, `marker-mid`, `marker-end`	Name the marker that is drawn on the first vertex, on the vertices between, and on the last vertex of a shape.	none or `url(#id)`	none	SVG 2

Clipping, masking and transform properties

Name	Description	Values	Default	Specification
`clip-path`	Shows only the part of an element inside an area.	none, `url(#id)` of a `clipPath`, or a basic shape, with `fill-box`, `stroke-box` or `view-box`	none	CSS Masking 1 §5.1
`clip-rule`	Chooses which areas of a shape inside a `clipPath` are inside it.	`nonzero` or `evenodd`	`nonzero`	CSS Masking 1 §6.2
`mask`	Makes parts of an element transparent with a `mask` element.	none or `url(#id)` of a `mask` element	none	CSS Masking 1 §7.9
`mask-type`	Chooses whether a `mask` element uses the luminance or the opacity of what it holds.	`luminance` or `alpha`	`luminance`	CSS Masking 1 §9.2
`opacity`	Makes an element and its children transparent as a group.	A number from 0 to 1, or a percentage	`1`	CSS Color 4 §3.3
`transform`	Moves, turns, scales or skews an element and its children.	none or a list of transform functions, 2D or 3D	none	CSS Transforms 1 §4

Name	Description	Values	Default	Specification
`transform-origin`	Sets the point that `transform` turns and scales around. In a drawing it is the origin of the drawing.	One to three lengths, percentages or keywords	`0 0`	CSS Transforms 1 §5
`translate`, `rotate`, `scale`	Move, turn or scale an element, each in a property of its own.	none or the arguments of the function of the same name	none	CSS Transforms 2 §5

Text properties of a drawing

Name	Description	Values	Default	Specification
`text-anchor`	Chooses which part of a line of text is at its x position.	`start`, `middle` or `end`	`start`	SVG 2
`dominant-baseline`	Chooses the baseline of the text that is placed at its y position.	`auto`, `alphabetic`, `middle`, `central`, `hanging`, `ideographic`, `mathematical`, `text-top` or `text-bottom`	`auto`	SVG 2
`baseline-shift`	Raises or lowers a part of a line of text.	`super`, `sub`, a length or a percentage	`0`	SVG 2
`inline-size`	On a `text` element, sets the length of its lines, and the text wraps to it.	`auto`, a length or a percentage	`auto`	SVG 2
`shape-inside`	On a `text` element, pours the text into a shape.	`auto`, a basic shape, `url(#id)` of a shape, or the URL of an image	`auto`	CSS Shapes 2

Licenses

CSSMedia

- Available from https://github.com/maxprograms-com/CSSMedia
- Published under Business Source License 1.1

Adobe Core 14 AFM files

- Available from https://download.macromedia.com/pub/developer/opentype/tech-notes/Core14_AFMs.zip
- Published under a permissive license, the text of which is the file `MustRead.html` inside the zip. The files may be used, copied and distributed for any purpose and without charge, with or without modification, provided that the copyright notices are retained, the AFM files are not distributed without `MustRead.html`, and modifications are noted

Adobe Glyph List and ITC Zapf Dingbats Glyph List

- Available from https://github.com/adobe-type-tools/agl-aglfn
- Published under BSD 3-Clause License

css-tree

- Available from https://github.com/csstree/csstree
- Published under MIT License

Noto Fonts

- Available from https://fonts.google.com/noto
- Published under SIL Open Font License 1.1

pdfnative

- Available from https://github.com/Nizoka/pdfnative
- Published under MIT License

TeX hyphenation patterns

- Available from https://github.com/hyphenation/tex-hyphen
- Published under LPPL, MIT, MPL or other permissive licenses

TypesBCP47

- Available from https://github.com/maxprograms-com/TypesBCP47
- Published under Eclipse Public License v1.0

TypesXML

- Available from https://github.com/maxprograms-com/TypesXML
- Published under Eclipse Public License v1.0

Unicode Character Database

- Available from https://www.unicode.org/Public/UCD/latest/ucd/
- Published under Unicode License v3

Unicode CLDR

- Available from https://github.com/unicode-org/cldr-json

- Published under Unicode License v3

Universal Shaping Engine data

- Available from https://github.com/microsoft/font-tools/tree/main/USE
- Published under MIT License

W3C CSS Counter Styles Level 3

- Available from https://www.w3.org/TR/css-counter-styles-3/
- Published under W3C Software and Document License

W3C webref

- Available from https://github.com/w3c/webref
- Published under MIT License

Glossary

Bookmark

A title in the navigation tree of a PDF. PDF viewers, such as Adobe Acrobat, show the bookmarks in a side panel. Selecting a title goes to its page. Bookmarks can have child bookmarks.

Box

The rectangle that layout reserves on a page for an XML element and its content. It has a content area, padding, a border and a margin. An element can make one box, more than one, or none, depending on its `display` value.

Containing block

The box that gives its size to the percentages of the boxes inside it. For a box in the normal flow, it is the content area of the nearest parent box that is a block.

CSS

Cascading Style Sheets (CSS) is the language, defined by the W3C, that describes how elements of a document are presented. A stylesheet is a list of rules. Each rule has a selector that chooses elements and declarations that set their properties. CSSMedia reads a CSS file to decide how to format an XML document in a PDF.

DTD

A Document Type Definition (DTD) is a grammar, defined by the XML specification, that declares the elements and attributes an XML document may use. A document points to its DTD in its document type declaration. When the DTD is not at the location the document names, an OASIS Catalog tells CSSMedia where to find it.

Font face

One style of a font family, such as its bold italic style. A font file holds one face, or several faces in a font collection. CSS Fonts 4 defines how a stylesheet declares faces.

Glyph

The shape that a font draws for a character or for a group of characters. One character can have several glyphs, such as a plain letter and a small capital.

Kerning

The adjustment of the space between two letters, so that pairs such as "AV" do not look too far apart.

Leader

A row of dots, or of another repeated pattern, that fills the space between two parts of a line. A table of contents uses a leader to join the title of an entry to its page number. CSS Content 3 defines leaders.

Ligature

A single glyph that replaces two or more letters, such as the glyph for "fi". Fonts use ligatures to avoid collisions between letters.

MathML

Mathematical Markup Language (MathML) is a W3C XML language for writing mathematical notation. A MathML formula describes the structure of an expression, such as fractions, roots and subscripts. CSSMedia draws MathML found in a document as an image.

Namespace

An XML namespace is a URI that identifies the vocabulary an element or attribute belongs to. An XML document declares it with an `xmlns` attribute, so elements with the same name in different vocabularies do not collide.

OASIS Catalog

An OASIS Catalog is an XML file, defined by the OASIS XML Catalogs standard, that maps the identifiers used in documents to the locations where the files are stored. A catalog lets a parser find a DTD, an XML Schema or a RelaxNG schema without downloading it. Pass a catalog to CSSMedia with the `-g` option.

OpenType

The font format that most fonts use today. An OpenType font holds glyphs and tables of features, such as ligatures, alternate shapes and kerning. Each feature has a four-letter tag. The OpenType specification defines the format.

PDF

Portable Document Format (PDF) is the file format, standardized as ISO 32000, for documents that look the same on any device. A PDF holds pages with text, fonts, images and vector graphics at fixed positions, and may also hold links and bookmarks. CSSMedia writes a PDF file from an XML file and a CSS file.

RelaxNG

RelaxNG is a schema language for XML, standardized as ISO/IEC 19757-2. A RelaxNG schema describes the structure of a document in an XML syntax or in a compact non-XML syntax. It is an alternative to a DTD and to an XML Schema, and it can be located with an OASIS Catalog.

Replaced element

An element whose content is replaced by something that is not text, such as an image. Its size comes from the image, unless the stylesheet sets one.

Variable font

A font whose design changes along axes, such as weight, width and slant. One variable font file can replace many ordinary font files. Each axis has a four-letter tag and a range of values. The OpenType specification describes variable fonts.

SVG

Scalable Vector Graphics (SVG) is a W3C format for two-dimensional vector graphics, written as XML. An SVG drawing describes shapes, paths, text and images that can be scaled without losing quality. CSSMedia draws SVG found in a document as an image.

XML

Extensible Markup Language (XML) is a W3C standard for writing structured documents. An XML document is a tree of elements, with attributes and text, whose names are chosen by the author or by a vocabulary such as DITA or DocBook. CSSMedia reads an XML file as the content of the PDF.

XML Schema

XML Schema is a W3C language for describing the structure of XML documents and the data types of their content. A schema is itself an XML file, and it is more expressive than a DTD. Like a DTD, a schema can be located with an OASIS Catalog.